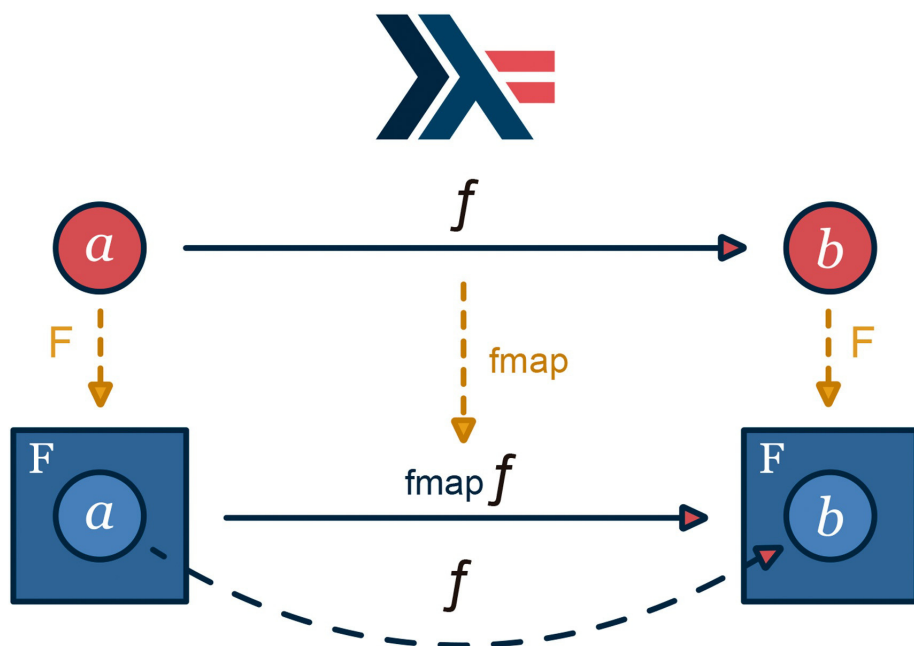


Haskell

Uma introdução à programação funcional

Edição Atualizada



© Casa do Código

Todos os direitos reservados e protegidos pela Lei nº 9.610, de 10/02/1998.

Nenhuma parte deste livro poderá ser reproduzida, nem transmitida, sem autorização prévia por escrito da editora, sejam quais forem os meios: fotográficos, eletrônicos, mecânicos, gravação ou quaisquer outros.

Edição

Vivian Matsui

Mariana Cerigatto

Gerente comercial e administrativa

Regiane Nunes

Capa

Design Alura

[2026]

Casa do Código

Rua Vergueiro, 3185 - 8º andar

04101-300 – Vila Mariana – São Paulo – SP – Brasil

casadocodigo.com.br



Este livro possui a curadoria da Casa do Código e foi estruturado e criado com todo o carinho para que você possa aprender algo novo e acrescentar conhecimentos ao seu portfólio e à sua carreira.

A Casa do Código é a editora da Alura, escola online de tecnologia que nasceu da vontade de criar uma plataforma de ensino com o objetivo de incentivar a transformação pessoal e profissional através da tecnologia.

O ecossistema da Alura, que inclui Fiap e PM3, constrói uma verdadeira comunidade colaborativa de aprendizado em tecnologia, programação, gestão, produtos e muito mais, oferecendo inovação na evolução dos seus alunos e alunas através de uma verdadeira experiência de encantamento.

Venha conhecer os cursos da Alura e siga-nos em nossas redes sociais.

 alura.com.br

 [@casadocodigo](https://www.instagram.com/casadocodigo)

 [casa-do-código](https://www.linkedin.com/company/casa-do-codigo)

ISBN

Impresso e PDF: 978-85-5519-273-9

EPUB: 978-85-5519-274-6

Caso você deseje submeter alguma errata ou sugestão, acesse
<http://erratas.casadocodigo.com.br>.

AGRADECIMENTOS

A Deus, pela força e pela perseverança que sempre me concedeu.

À minha família, pelo suporte emocional.

À minha namorada, Rosanne, por me aguentar sempre.

Aos meus professores da USP e da Fatec, que contribuíram para eu chegar até aqui.

À comunidade Haskell, por me fazer crescer na linguagem.

Aos meus alunos, que contribuíram de alguma forma com suas dúvidas e/ou orientação de TCCs.

À editora, Vivian Matsui, por tirar minhas dúvidas nas horas mais inoportunas.

SOBRE O AUTOR

O autor é formado como tecnólogo em Processamento de Dados pela Faculdade de Tecnologia da Baixada Santista Rubens Lara (FATEC-RL), desde 2004, e bacharel em Matemática pelo Instituto de Matemática e Estatística da Universidade de São Paulo (IME-USP), desde 2012, onde também obteve o título de mestre em Matemática Aplicada em 2015.

Leciona há 7 anos no Centro Estadual de Educação Tecnológica Paula Souza (CEETEPS), dos quais há 5 anos atua nas Fatecs, tendo passado pelos campi de Santos, Praia Grande e São Caetano do Sul.

Programa em Haskell há cerca de 4 anos e conheceu a linguagem por meio da orientação de um TCC na Fatec de São Caetano. Também é apaixonado por Álgebra, pelo Santos Futebol Clube, pela série *Final Fantasy* (especialmente *Final Fantasy VII*), por viagens de navio e por séries e animes.

PREFÁCIO

Haskell é uma linguagem de programação que traz novas formas de escrever programas corretos, eficientes e fáceis de manter. Com um modelo mental novo e uma caixa de ferramentas de programação extremamente moderna, essa linguagem pura e funcional nos dá *insights* valiosos para a construção de aplicações, mesmo quando precisamos aplicar esses *insights* em outras tecnologias e contextos.

Este livro foi escrito como uma introdução à linguagem Haskell para leitores familiarizados com ao menos uma linguagem de programação imperativa e dispostos a dedicar tempo para entender o que a programação puramente funcional tem a nos oferecer. Interessante e divertida, a linguagem é única por ser funcional, pura, estaticamente tipada, com avaliação preguiçosa por padrão e versões compiladas e interpretadas.

Com a crescente necessidade da indústria de escrever *software* altamente concorrente e paralelo, e capaz de ser executado na nuvem — *software* que precisa, sem grandes mudanças, suportar um ou N contextos de execução paralelos para escalar —, o modelo de programação sem o uso implícito de estado está em alta. Esse modelo, aqui tomando a forma de programação funcional ou declarativa, nos incentiva a modularizar as menores unidades possíveis da computação: funções. Usando a composição de declarações do que uma computação é, em vez de sequências de instruções de como uma computação ocorre, quebramos problemas em partes mais determinísticas. Assim, em vários casos, ganhamos suporte ao paralelismo e à concorrência de graça.

Além disso, nossos programas são cada vez maiores e mais complexos, e o trabalho de manutenção de *software* é cada vez maior e mais suscetível a falhas humanas. Por ser puro e estaticamente tipado, Haskell nos permite estabelecer garantias teóricas (a partir de provas e/ou modelos matemáticos expressos no código, bem como de propriedades de como expressões são avaliadas) e estáticas (a partir de mais metadados em seu rico sistema de tipos, programas finais sem casos não tratados).

Haskell é muito usado em pesquisa em linguagens de programação e tem um dos sistemas de tipos mais avançados disponíveis. A promessa é a de entregar programas sólidos mais rapidamente ao mercado, mesmo que um primeiro protótipo demore mais a ser escrito. O professor Alexandre convida-os a descobrir a sintaxe da linguagem Haskell, um pouco do *ethos* da programação funcional e alguns dos conceitos formais baseados na teoria das categorias, que proporcionam essas garantias, possibilidades e permeiam essa tecnologia. E ele faz isso com a iniciativa improvável e bem-sucedida de dois anos de aulas na FATEC-RL, ensinando seus alunos a beleza e a aplicação do paradigma puramente funcional e do Haskell.

Este livro é um primeiro pé em um longo caminho, no qual eu ainda tenho muito a percorrer; uma nova empreitada na literatura de linguagens de programação brasileira que espero ser tão útil para vocês quanto é animadora para mim.

Pedro Tacla Yamada — colaborador na HaskellBR, desenvolvedor e consultor na Beijafloor Software.

INTRODUÇÃO

Este livro destina-se a alunos(as) interessados(as) em aprender sobre este paradigma, que vem sendo cada vez mais adotado pelo mercado. Eu geralmente não recomendo um curso específico como pré-requisito, porém um semestre de lógica de programação e estruturas de dados não machuca e serve para entender um ou dois exemplos do livro. Um(a) aluno(a) que tem facilidade com matemática e paixão pelo assunto consegue seguir o livro sem ter feito um curso formal nos assuntos indicados também.

O livro constrói o conhecimento na linguagem Haskell, começando do zero, desde a sua instalação até o conceito de e a construção de uma aplicação web de verdade. Ele possui onze capítulos, um apêndice e referências, e foi escrito com base nas notas de aula do meu curso de Programação Funcional, na Faculdade de Tecnologia da Baixada Santista Rubens Lara (FATEC-RL). Depois de quase dois anos ministrando esta disciplina e tendo notado a dificuldade dos alunos em encontrar material em nossa língua materna, eis que surgiu a ideia do presente trabalho.

O primeiro capítulo aborda questões gerais da linguagem e do paradigma. No segundo, vemos a instalação do ambiente, alguns exemplos preliminares, manipulação de listas e tuplas. No terceiro, introduzimos os tipos (um dos maiores pontos fortes da linguagem) e suas manipulações.

Já no quarto, vemos como a linguagem trata as funções por meio dos conceitos de *lambdas*, *currying* e funções de alta ordem.

No quinto, alavancamos o poder dos tipos usando o conceito de polimorfismo paramétrico, classes de tipos e um exemplo muito especial: os monoides.

No sexto, há uma breve introdução informal sobre Teoria das Categorias. O sétimo explora os funtores, um dos conceitos-chave para o entendimento do assunto a ser abordado no oitavo capítulo: as Mônadas. No nono, são dados vários exemplos práticos de Mônadas por meio do IO, que é a representação da computação com efeitos no Haskell.

Os dois últimos capítulos põem a mão na massa: no décimo, construímos um pequeno interpretador de uma minilinguagem e usamos o sistema de tipos (com uma espiada em GADTs) para garantir que só programas bem formados rodem; e no décimo primeiro, colocamos uma aplicação web no ar com a biblioteca Servant, descrevendo uma API como um tipo. Confira ainda o apêndice com todo o código do miniprojeto e as referências para seguir seus estudos.

O QUE HÁ DE NOVO NESTA EDIÇÃO

Olá! Que bom ter você por aqui. Esta é uma versão revisada e ampliada do livro e eu quero te contar, rapidinho, o que mudou.

O livro está em dia com o Haskell de hoje

Revisei o livro inteiro para acompanhar a versão atual do Haskell. A forma de instalar a linguagem ficou mais simples (agora usamos o GHCup), e todos os exemplos de código foram testados de novo, um por um, para rodarem direitinho no compilador mais recente. Sempre que algo estava desatualizado ou com um errinho, foi corrigido.

Dois capítulos novos

A maior novidade são **dois capítulos extras**, no fim do livro, para você colocar a mão na massa:

- **Um pequeno interpretador:** você vai criar uma minilinguagem de programação dentro do Haskell. É um jeito divertido de ver o sistema de tipos trabalhando a seu favor.
- **Haskell na web com Servant:** aqui a gente fecha com chave de ouro, colocando uma aplicação web de verdade no ar, do zero até deixar o servidor rodando.

Mais exercícios e exemplos atualizados

Espalhei **novos exercícios** por vários capítulos, para você praticar mais. Alguns exemplos também foram modernizados, e as

referências do fim do livro ganharam novas indicações para você seguir estudando.

Em resumo

Você tem em mãos um livro mais completo, com o código funcionando na versão atual do Haskell e duas aventuras novas para explorar. Espero que aproveite. Bons estudos e bem-vindo ao mundo Haskell!

Sumário

1 Programação funcional	1
1.1 Linguagem Haskell	3
1.2 Haskell na web	5
1.3 Conclusão	6
2 Primeiros exemplos	7
2.1 Primeiro contato com os tipos de dados e funções	10
2.2 Operação com listas	11
2.3 Compreensão de listas	15
2.4 Tuplas	16
2.5 Exercícios	17
2.6 Conclusão	19
3 Declarando novos tipos de dados	20
3.1 Pattern matching	21
3.2 Campos de um construtor	26
3.3 Record syntax	28
3.4 Miniprojeto: RH de uma empresa de TI	29
3.5 Exercícios	32

3.6 Conclusão	37
4 Um pouco mais sobre funções	38
4.1 Lambdas	38
4.2 Funções de alta ordem	39
4.3 Currying	41
4.4 Exemplos de funções de alta ordem	42
4.5 Sintaxe em funções	51
4.6 Recursão	52
4.7 Miniprojeto: RH de uma empresa de TI	56
4.8 Exercícios	58
4.9 Conclusão	60
5 Polimorfismo paramétrico	61
5.1 Tipos com parâmetros	61
5.2 Restrição de tipos em funções	65
5.3 Classes de tipos	66
5.4 Outras classes	71
5.5 Semigrupos	76
5.6 Monoides	77
5.7 Miniprojeto: trabalhando com parsers	83
5.8 Exercícios	85
5.9 Conclusão	88
6 Teoria das Categorias	90
6.1 Categorias	90
6.2 Noção matemática de funtor	95
6.3 Função identidade em Haskell	96

6.4 Conclusão	98
7 Funtores	99
7.1 Funtor Maybe	101
7.2 Criando seu funtor	103
7.3 Funtores Aplicativos	104
7.4 Funtores Contravariantes	108
7.5 Miniprojeto: continuação usando funtores	113
7.6 Exercícios	114
7.7 Conclusão	115
8 Mônadas	117
8.1 Transformações naturais	118
8.2 Definição	120
8.3 Notação "do"	125
8.4 A Mônada []	129
8.5 Exercícios	130
8.6 Conclusão	132
9 Mônada IO	133
9.1 Compilando um programa "Olá Mundo"	134
9.2 Exemplos práticos	136
9.3 Manipulando arquivos	144
9.4 Miniprojeto final	147
9.5 Exercícios	157
9.6 Conclusão	159
10 Um pequeno interpretador	160
10.1 Representando programas como dados	161

10.2 Avaliando programas (com checagem em tempo de execução)	162
10.3 Tipos como exemplo de elegância: GADTs	164
10.4 Exercícios	167
10.5 Conclusão	168
11 Haskell na web com Servant	169
11.1 Seção 1: configurando o projeto com Cabal	170
11.2 Seção 2: módulos e como atualizar o arquivo .cabal	172
11.3 Seção 3: uma espiada em GADTs	173
11.4 Seção 4: o Servant e a explicação dos seus tipos	174
11.5 Seção 5: implementando os endpoints GET	177
11.6 Seção 6: o servidor e o ponto de entrada (main)	178
11.7 Exercícios	180
11.8 Conclusão	181
12 Apêndice	183
13 Referências	188

PROGRAMAÇÃO FUNCIONAL

A programação funcional é um paradigma de programação que trata apenas da aplicação de funções matemáticas, evitando a alteração de estado e a mutabilidade de dados. Ou seja, assim que uma variável é alocada na memória e um valor é associado a este local, tal valor não pode ser mudado, e sim transformado por uma aplicação de função.

Uma das características da programação funcional é o estilo declarativo, que se opõe ao estilo imperativo. Nela, não há descrição de estruturas de controle, e seu estilo descreve o que o programa faz (*what to do*), e não como ele deve ser feito (*how to do it*). O uso desse estilo visa minimizar os impactos dos efeitos colaterais (*side effects*). Esse paradigma é conhecido como forma funcional e faz uso de funções de ordem superior.

A fundamentação matemática rigorosa da programação funcional nos permite escrever testes de *software* mais precisos, baseados em propriedades matemáticas que possibilitam construir provas capazes de validar um código.

Em uma linguagem funcional, funções são tratadas como valores comuns, sendo que existe a possibilidade de uma função ser atribuída a uma constante localmente, ser passada via parâmetro ou até mesmo ser retornada por outra função. Atualmente, algumas das principais linguagens do mercado (por exemplo, Java e C#) incorporaram ferramentas da programação funcional, como o uso de *lambdas*, que consiste em tratar um método como se fosse um valor, podendo este ser passado como parâmetro ou retornado, conforme descrito anteriormente. Linguagens como JavaScript, Python, Ruby e muitas outras possuem algum suporte para esse paradigma, o que demonstra o crescente interesse das comunidades de desenvolvedores pelo tema.

Muitas linguagens que suportam apenas o paradigma funcional estão crescendo no mercado. Entre elas, podemos citar: Haskell, Erlang, Clojure, Scala, OCaml e algumas linguagens que compilam para JavaScript, como ClojureScript, Elm, PureScript, entre outras. Já em 2016, a empresa RedMonk elaborou um ranking baseado nas atividades das plataformas GitHub e Stack Overflow, no qual as linguagens funcionais Scala, Haskell e Clojure já apareciam, respectivamente, em 14ª, 15ª e 19ª colocações (O'GRADY, 2016). De lá para cá, a adoção da programação funcional só cresceu: ideias que eram "acadêmicas" — *lambdas*, imutabilidade, tipos algébricos, *pattern matching*, *Option / Maybe* — hoje são parte do dia a dia de praticamente toda linguagem moderna. O próprio Haskell segue firme, com lançamentos regulares do compilador (a série estável mais recente, à época da escrita deste livro, é a 9.14) e um ecossistema maduro de bibliotecas para web, dados e concorrência.

1.1 LINGUAGEM HASKELL

Esta linguagem foi concebida em 1987, de acordo com Peyton Jones (2007), durante uma conferência de programação funcional. Neste evento, um comitê de intelectuais se formou para criar um novo padrão de programação funcional. Além das características do paradigma funcional descritas anteriormente, outros fatores presentes são: *laziness*, o fato de ser uma linguagem de programação funcional pura e ser estaticamente tipada (PEYTON JONES, 2009).

O conceito de *laziness* (ou processamento preguiçoso) é o ato de a linguagem só calcular expressões quando realmente for necessário (HUGHES, 1990). Isto evita alguns processamentos desnecessários. Por exemplo, a função `++` em Haskell significa concatenação de listas. Se tivermos a seguinte expressão:

```
[3,6,7,3*10^89,0] ++ [-1, 9]
```

Ela produzirá a lista `[3,6,7,3*10^89,0, -1, 9]`, sem precisar calcular a expressão `3*10^89`, economizando tempo de processamento, neste caso. Em Hughes (1990) e Peyton Jones (2010), é possível ver que, antigamente, o conceito de computação preguiçosa e efeitos externos (leitura e escrita de arquivos, por exemplo) não poderiam coexistir.

"We have described lazy evaluation in the context of functional languages, but surely so useful a feature should be added to nonfunctional languages — or should it? Can lazy evaluation and side-effects coexist? Unfortunately, they cannot: Adding lazy evaluation to an imperative notation is not actually impossible, but the combination would make the programmer's life harder, rather

than easier. Because lazy evaluation's power depends on the programmer giving up any direct control over the order in which the parts of a program are executed, it would make programming with side effects rather difficult, because predicting in what order — or even whether — they might take place would require knowing a lot about the context in which they are embedded" (HUGHES, 1990).

Isso quer dizer que uma lista de ações com efeito externo, como imprimir um caractere na tela, não teria uma sequência aparente de execução dos comandos, tornando aparentemente impossível a existência de efeitos externos na linguagem, reduzindo-a a ambientes acadêmicos, inviabilizando o desenvolvimento de aplicações para o mercado de trabalho.

Felizmente, como vemos em Peyton Jones (2010), o conceito de Mônadas (ou **Monads**) tornou possível o trabalho conjunto desses dois conceitos e, além disso, obtém-se um sólido formalismo matemático, fundamentado na Teoria das Categorias.

A tipagem estática do Haskell é uma poderosa ferramenta da linguagem, permitindo usar o conceito de *type-safety*, que possibilita controlar erros de programação oriundos de conversões implícitas de tipos, como é comumente observado em outras linguagens (O'SULLIVAN et al., 2012).

O conceito de programação funcional pura nos diz que toda função aqui é pura, ou seja, deve ter o mesmo retorno a partir de um mesmo argumento. Isso indica que não há mudança em variáveis e pontos globais de acesso. Para se manter pura, a linguagem usa conceitos matemáticos, vistos aqui para a manipulação de dados que sofram interferência externa (*input* e *output*).

1.2 HASKELL NA WEB

Hoje, é possível desenvolver aplicações para a web usando a linguagem Haskell, e alguns *frameworks* e bibliotecas que usam essa linguagem são notórios, tais como: Servant, Yesod, Scotty, Happstack e Snap. Cada um deles possui características diferentes. No último capítulo deste livro, daremos ênfase ao **Servant**, uma biblioteca moderna em que a sua API web é descrita como um **tipo**, fazendo com que o compilador garanta que o servidor respeite exatamente o contrato definido. Antes dele, surgiu o Yesod, desenvolvido por Michael Snoyman (SNOYMAN, 2012).

A maior característica do Yesod é o fato de termos segurança de tipos (*type-safety*) nas URLs. Ou seja, uma aplicação web desenvolvida com este *framework* não terá problemas de imagens ou links quebrados, pois o conceito em foco garante que tais erros sejam checados em tempo de compilação e não de execução, garantindo aplicações com menos erros.

Conforme Snoyman (2012), o Yesod possui suporte também para *webservices* REST, padrão de internacionalização i18n, persistência de dados (dependendo do pacote usado, isto pode ser feito de modo *type-safe* também), interpoladores para criação de páginas dinâmicas e templatização usando os *shakespearean templates* que consistem em DSLs (*Domain Specific Languages*) para HTML, CSS e JavaScript chamadas de Hamlet, Lucius (Cassius) e Julius. Conclui-se que é possível programar usando Haskell e colocar aplicações em produção, o que antigamente não era possível.

1.3 CONCLUSÃO

Vimos algumas informações básicas sobre a linguagem Haskell e o paradigma funcional, bem como algumas características importantes da linguagem. A partir de agora, apresentaremos a configuração de um ambiente apropriado para programar em Haskell, além dos primeiros conceitos e trechos de código apresentados neste livro.

PRIMEIROS EXEMPLOS

A linguagem Haskell é uma linguagem de tipagem forte e estática, ou seja, toda expressão possui um tipo definido em tempo de compilação. Qualquer problema com tipos será detectado em tempo de compilação; por exemplo, se um parâmetro do tipo inteiro é esperado e um tipo Char é passado.

Será necessária a instalação do compilador `ghc` (*The Glasgow Haskell Compiler*). A antiga *Haskell Platform* foi descontinuada e **não deve mais ser usada**. Hoje, a forma recomendada de instalar o Haskell é por meio do **GHCup**, que gerencia, de uma só vez, o compilador `ghc`, o gerenciador de pacotes `cabal`, o `stack` e o servidor de linguagem `HLS`. Em Linux, macOS ou WSL, basta rodar:

```
curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

Depois da instalação, o comando `ghcup tui` abre um menu interativo no terminal, no qual você pode instalar, escolher (teclas de *set*) e remover versões do `ghc` e das demais ferramentas sem decorar comando algum. À época da escrita deste livro, a versão estável mais recente do compilador é a série **9.14**, mas qualquer versão a partir da **9.6** roda todos os exemplos aqui apresentados. Em caso de dúvida, use a versão marcada como

recommended pelo próprio GHCup (rode `ghcup list` para vê-la), que é a mais segura para quem está começando.

Para começarmos, usaremos a ferramenta `ghci`, que já vem junto com o `ghc` e é um REPL (*Read, Evaluate, Print, Loop*). Basta digitar na linha de comando a palavra `ghci` e teremos o seguinte *prompt*:

```
Prelude>
```

A palavra `Prelude` significa que os módulos básicos da linguagem foram carregados. Para carregar um módulo, basta executar o comando:

```
Prelude> :l Teste.hs
```

O `Teste.hs` é um arquivo na mesma pasta na qual o comando `GHCI` foi digitado. Um módulo começa da seguinte forma:

```
module Teste where
```

A partir daí, poderemos definir nossos tipos de dados e funções, e prosseguir com o desenvolvimento de um módulo. Observe também que, como chamamos o módulo de `Teste`, o seguinte comando também carregará este módulo:

```
Prelude> :l Teste
```

Isto suprime a extensão `.hs`. Se a declaração do início do módulo for suprimida, não haverá problema algum e o módulo será chamado de `Main` pelo `ghc`. Se o nome do arquivo e o nome do módulo forem diferentes, deveremos carregá-los pelo nome do arquivo apenas.

Após o módulo ser carregado com sucesso (e passar pelo processo de compilação), qualquer alteração feita precisará ser carregada novamente no módulo, usando o comando:

```
Teste> :r
```

Este dará um reload no módulo e compilará novamente, sem a necessidade de usar o `:l` com o nome do arquivo. Aqui você poderá também calcular valores de expressões que dependem ou não do módulo carregado.

```
Teste> 2+2  
4
```

```
Teste> 3<4  
True
```

```
Teste> "Ola " ++ "Mundo"  
"Ola Mundo"
```

A resposta aparecerá logo abaixo do indicador do prompt. Dessa forma, é possível testar as funções implementadas em um módulo de maneira fácil.

Finalmente, um dos comandos mais úteis do `ghci` é o `:t`, que inspeciona o tipo de dado de um valor:

```
Teste>:t True  
True :: Bool
```

A resposta obtida pode ser lida como `True`, e tem o tipo `Bool`. Note que o símbolo `::` foi lido como "tem o tipo".

2.1 PRIMEIRO CONTATO COM OS TIPOS DE DADOS E FUNÇÕES

O arquivo com extensão `.hs` é onde ficam as funções declaradas, como visto na seção anterior, de modo a serem usadas pelo GHCi. Toda função deve seguir o seguinte padrão:

`nomeDaFuncao p1 p2 p3 ... pN` = expressão que depende dos N parâmetros.

Note que a convenção utilizada para nomear funções é a *CamelCase* (equivalente para nomeação de métodos em Java) e, para nomear os parâmetros, todas as letras devem ser minúsculas. Funções também possuem tipo e podem ser declaradas explicitamente. Isto é considerado uma boa prática e, sempre que possível, devemos colocar tipos em nossas funções, como o seguinte exemplo:

```
maiorQue :: Int -> Int -> Bool
maiorQue x y = x > y
```

Esse exemplo mostra uma função que possui dois parâmetros inteiros, e sua expressão possui um retorno booleano. Portanto, o tipo da função é `Int -> Int -> Bool`.

Uma dica a seguir é que o número de flechas (`->`) acompanha o número de parâmetros que a função admite, que neste caso, são chamados de `x` e `y`. A função (`>`) também pode ser chamada de maneira infixa, como segue:

```
maiorQue :: Int -> Int -> Bool
maiorQue x y = (>) x y
```

Como a função (`>`) recebe dois parâmetros, o uso dos parênteses faz a chamada desta função de maneira infixa.

Observe agora esta função declarada a seguir:

```
u :: Int
u = 7
```

Neste exemplo, temos uma função `u` que não recebe parâmetros e possui um retorno inteiro constante. Um lembrete aos programadores imperativos é que, aqui em Haskell, o sinal de igual não representa atribuição como em muitas linguagens, e sim definição. Pelo fato de Haskell ter seus dados como sendo imutáveis, o trecho a seguir produzirá um erro de ambiguidade de definição.

```
u :: Int
u = 7

u = 6
```

Como pode ser visto, a partir do momento em que tentamos compilar o trecho anterior, não é possível distinguir as duas definições, causando, assim, um erro em tempo de compilação.

```
Teste.hs:6:1: error:
  Multiple declarations of 'u'
  Declared at: Teste.hs:4:1
               Teste.hs:6:1
```

2.2 OPERAÇÃO COM LISTAS

O `Prelude` vem recheado de funções para operar com listas, e nesta seção vamos ver algumas delas. Uma lista é um tipo de dado que assume um valor vazio `[]` ou, se possuir algum elemento, como um inteiro, escrevemos na forma `[1, 2, 3]`.

Listas possuem elementos do mesmo tipo e podem ter elementos adicionados ou removidos, fazendo com que seu

tamanho seja variável. Podemos inspecionar o tipo de uma lista no `ghci` desta forma:

```
Prelude>:t [True, False, True]
[True, False, True] :: [Bool]
```

A resposta obtida é lida da mesma forma que anteriormente. `[True, False, True]` tem o tipo `[Bool]`. A função `(++)` representa a concatenação entre duas listas, por exemplo:

```
Prelude> [1,2] ++ [3,4]
[1,2,3,4]
```

Também temos a maneira infixa:

```
Prelude> (++) [1,2] [3,4]
[1,2,3,4]
```

Observe que, se um dos parâmetros for uma lista vazia `[]`, obteremos os seguintes resultados:

```
Prelude> [1,2] ++ []
[1,2]
```

```
Prelude> [] ++ [1,2]
[1,2]
```

Estes resultados serão úteis mais à frente. Vale notar que a função `(++)` opera sob listas de qualquer tipo de dado:

```
Prelude> [True,False] ++ [True]
[True, False, True]
```

```
Prelude> "ABCD" ++ "EFG"
"ABCDEFG"
```

Não podemos nos esquecer que uma `String` (aspas duplas) é um apelido para lista de `Char` (aspas simples) ou, em Haskell, `[Char]`. O fato de esta função funcionar para listas de quaisquer tipos será explorado mais à frente.

As funções `head` , `last` e `tail` extraem o primeiro, o último e uma lista sem o primeiro elemento, respectivamente. Veja:

```
Prelude> head "ABCDEFGH"  
'A'
```

```
Prelude> last "ABCDEFGH"  
'H'
```

```
Prelude> tail "ABCDEFGH"  
"BCDEFGH"
```

Porém, se estas funções forem calculadas em uma lista vazia `[]` , obteremos:

```
Prelude> head []  
*** Exception: Prelude.head: empty list
```

```
Prelude> last []  
*** Exception: Prelude.last: empty list
```

```
Prelude> tail []  
*** Exception: Prelude.tail: empty list
```

Tais erros decorrem do fato de as três funções não estarem definidas na lista vazia. Tais funções são chamadas de funções parciais e devem ser evitadas, pois são fontes de erros de execução, os quais tentamos diminuir ao mínimo quando usamos a linguagem Haskell.

A função `reverse` inverte a ordem de uma lista de qualquer tipo:

```
Prelude> reverse "HASKELL"  
"LLEKSAH"
```

```
Prelude> reverse [1,2,3]  
[3,2,1]
```

```
Prelude> reverse []
```

```
[]
```

Observe que é possível deduzir que a função `last` é a composta de `head` com `reverse` :

```
Prelude> (head . reverse) "HASKELL"
'L'
Prelude> last "HASKELL"
'L'
```

Mais detalhes sobre a função que compõe duas funções `(.)` serão dados mais adiante. Não se preocupe.

A função `!!` , usada de maneira infixa, recebe uma lista e um número inteiro, e devolve o elemento na posição informada.

```
Prelude> [1,2,3,4] !! 2
3
Prelude> [1,2,3,4] !! 0
1
```

Tal função também é parcial, visto que:

```
Prelude> [1,2,3,4] !! (-2)
*** Exception: Prelude.!!!: negative index
Prelude> [] !! 2
*** Exception: Prelude.!!!: index too large.
```

A primeira exceção nos indica que houve uma ocorrência de índice negativo, no caso, `-2` . Já a última exceção indica que tentamos acessar uma posição maior que o tamanho da lista. Neste caso, temos uma lista de tamanho `0` e tentamos acessar a terceira posição `2` .

A função `cons` `(:)` recebe um elemento e uma lista, e devolve este elemento na frente da lista:

```
Prelude> 3 : [3,4,5,6,-1]
[3,3,4,5,6,-1]
```

```
Prelude> 'A' : "BCDE"
"ABCDE"
Prelude> 2 : []
[2]
Prelude> 'A' : []
"A"
```

Também é possível chamar esta função sucessivas vezes. Veja:

```
Prelude> 3 : [4,5,2]
[3,4,5,2]
Prelude> 3 : 4 : [5,2]
[3,4,5,2]
Prelude> 3 : 4 : 5 : [2]
[3,4,5,2]
Prelude> 3 : 4 : 5 : 2 : []
[3,4,5,2]
```

Guarde bem esse exemplo, pois será de suma importância quando o conceito de *pattern matching* for introduzido.

Finalmente a função `length` recebe uma lista e retorna a quantidade de elementos contida nela.

```
Prelude> length [1,2,3]
3
Prelude> length ['a']
1
Prelude> length "a"
1
Prelude> length []
0
```

2.3 COMPREENSÃO DE LISTAS

Em Haskell, é possível construir listas de quaisquer tipos usando expressões que podem ser distribuídas a todos os elementos de um dado vetor, usando *compreensão de listas* (ou *list comprehensions*). De maneira geral:

```
[EXPRESSÃO(var) | var<-LISTA, FILTRO_1, FILTRO_2, ..., FILTRO_N]
```

A expressão é qualquer função que será distribuída nos elementos da lista, representados por `var`, com os elementos que passem na condição dos filtros. Veja um exemplo:

```
dobroLista :: [Int] -> [Int]
dobroLista xs = [2*x | x<-xs]
```

Essa função possui como parâmetro a lista de inteiros `x` e ela devolve uma lista de inteiros contendo o dobro de cada elemento `x`, contido em `xs`. Tendo em mente a descrição da função anterior, é possível enxergar o uso da sintaxe `[Int] -> [Int]`, que representa que a função possui um parâmetro do tipo lista de inteiros e esta retorna uma lista de inteiros.

O tipo definido após a última `->` representa o retorno da função. No exemplo anterior, a expressão é a função `2*x`, e a lista na qual a função será distribuída é `xs`.

```
lista :: [Int]
lista = [2*x+1 | x<-[0 .. 10], x/=5]
```

Neste exemplo, vemos que a função `2*x+1` se distribuirá a todos os elementos da lista `[0 .. 10]`, com exceção do número 5 que não passa no filtro indicado. Portanto, a lista tem como conteúdo `[1, 3, 5, 7, 9, 13, 15, 17, 19, 21]`.

2.4 TUPLAS

Diferentemente das listas, que só carregam dados de um tipo só com um número variável de elementos, as tuplas carregam diversos tipos ao mesmo tempo e possuem um número fixo de elementos. Não é possível usar a função `cons ( : )`, nem concatenar

(++) nada a elas. Tuplas são imutáveis. O número de elementos em uma tupla é fixo, e cada local no qual um elemento reside é chamado de coordenada.

```
Prelude>:t ('A',"AL0")  
('A',"AL0") :: (Char, [Char])
```

A resposta obtida indica que ('A',"AL0") tem o tipo (Char, [Char]), que implica que a primeira coordenada possui o tipo Char e a segunda [Char] ou String. É possível fazer funções usando tuplas:

```
foo :: Char -> Int -> (Int, String)  
foo x y = (y+9, x:[x])
```

Vamos testar no GHCi:

```
Prelude> foo 'E' 2  
(11,"EE")
```

As funções fst e snd projetam a primeira e a segunda coordenada de uma tupla, respectivamente.

```
Prelude> fst ('A',"AL0")  
'A'  
Prelude> snd ('A',"AL0")  
"AL0"
```

2.5 EXERCÍCIOS

2.1) Gere as listas:

a) [1, 11, 121, 1331, 14641, 161051, 1771561]

b) [1, 2, 3, 5, 6, 7, 9, 10, 11, 13, 14, 15, 17, 18, 19, 21, 22, 23, 25, 26, 27, 29, 30, 31, 33, 34, 35, 37, 38, 39]

c) ["AaBB", "AbBB", "AcBB", "AdBB", "AeBB", "AfBB", "AgBB"]

d) [5, 8, 11, 17, 20, 26, 29, 32, 38, 41]

e) [1.0, 0.5, 0.25, 0.125, 0.0625, 0.03125]

f) [1, 10, 19, 28, 37, 46, 55, 64]

g) [2, 4, 8, 10, 12, 16, 18, 22, 24, 28, 30]

h) ['@', 'A', 'C', 'D', 'E', 'G', 'J', 'L']

2.2) Crie uma função que verifique se o tamanho de uma String é par ou não. Use `Bool` como retorno.

2.3) Escreva uma função que receba um vetor de Strings e retorne uma lista com todos os elementos em ordem reversa.

2.4) Escreva uma função que receba um vetor de Strings e retorne uma lista com o tamanho de cada String. As palavras de tamanho par devem ser excluídas da resposta.

2.5) Escreva a função `head` como composição de duas outras.

2.6) Faça uma função que receba uma String e retorne `True` se esta for um palíndromo; caso contrário, `False`.

2.7) Faça uma função que receba um inteiro e retorne uma tupla, contendo: o dobro deste número na primeira coordenada, o triplo na segunda, o quádruplo na terceira e o quádruplo na quarta.

2.8) Usando *compreensão de listas*, gere todos os pares (x, y) com x em $[1..3]$ e y em $[1..3]$ tais que $x + y$ seja par. Qual é a lista resultante?

2.9) Escreva uma expressão que, usando `head` , `tail` e `reverse` , extraia o **penúltimo** elemento de uma lista. Teste com `[10, 20, 30, 40]` .

2.6 CONCLUSÃO

Neste capítulo, vimos o básico para a instalação do compilador do Haskell (GHC) e de seu **REPL** (GHCi), alguns comandos básicos e como calcular expressões no GHCi. A principal noção abordada foi a de tipos e funções, que são conceitos recorrentes em toda a linguagem.

Listas e tuplas são tipos de dados e foram introduzidos também aqui, pois são ferramentas básicas para conceitos mais complexos que serão vistos mais adiante. A manipulação de listas dá uma boa ideia de como é a escrita em programação funcional, e fornece uma noção ao leitor de como será daqui para a frente e o quanto difere das linguagens imperativas comumente usadas no mercado.

É sempre bom lembrar da importância da resolução dos exercícios e do acompanhamento dos exemplos dados no GHCi. Sem isso, o entendimento dos capítulos vindouros pode ser comprometido.

O próximo capítulo é consequência natural dos conceitos apresentados aqui. Para avançar, certifique-se de que você entendeu como construir sua função, como ler o seu tipo e como funciona sua escrita básica.

DECLARANDO NOVOS TIPOS DE DADOS

Este capítulo é uma introdução ao abrangente sistema de tipos da linguagem. Nele, abordam-se formas de criar e manipular tipos de dados na linguagem Haskell.

Uma das possibilidades de criar novos tipos em Haskell é utilizando a palavra reservada `data`. Considere o seguinte exemplo:

```
data Dia = Segunda | Terca | Quarta |  
         Quinta | Sexta | Sábado | Domingo
```

Lê-se que o novo tipo chamará `Dia` e possuirá *value constructors* `Segunda`, `Terca`, `Quarta`, `Quinta`, `Sexta`, `Sabado` ou `Domingo`. Logo, nota-se que `|` pode ser lido como *ou* e que o tipo `Dia` pode assumir um entre vários valores. O tipo `Dia` é comumente chamado na literatura de *sum type*.

Os *value constructors* são construtores de valores associados aos tipos (também chamados de *data constructors*). Se usarmos `:t` no GHCi, é possível enxergar o tipo de cada *value constructor*:

```
Prelude>:t Segunda  
Segunda :: Dia  
Prelude>:t Quinta
```

```
Quinta :: Dia
```

Podemos agora criar funções com o novo tipo criado:

```
agenda :: Dia -> String
agenda Domingo = "TV..."
agenda Sabado = "Festa"
agenda _ = "Trabalho"
```

Este exemplo toma como parâmetro um `Dia` e retorna uma `String` que representa a tarefa definida pela agenda. Observe que, nesta função, foi aplicado o conceito de *pattern matching*, que será estudado mais de perto na próxima seção, uma importante parte da sintaxe de funções do Haskell.

Neste exemplo, se o `Dia` passado for `Domingo`, o retorno será `"TV..."`; se for `Sabado`, `"Festa"`; e qualquer outro dia retornará `"Trabalho"`.

3.1 PATTERN MATCHING

Sempre que desejarmos analisar partes de um tipo em um determinado ponto do programa, usamos o *pattern matching*. Essa técnica permite a inspeção de partes menores de um *value constructor* contra um padrão determinado pela pessoa programadora.

É possível enxergar isto como aquele brinquedo que possui formas geométricas com buracos e suas respectivas peças. Se tentarmos encaixar a peça em forma de quadrado no buraco em forma de círculo, não será possível.

Vamos analisar, no exemplo a seguir, o tipo `(Int, Int)`, que é uma tupla que possui dois inteiros, um em cada coordenada. Serão

listadas algumas combinações válidas deste tipo, na entrada de uma função qualquer f :

```
f :: (Int, Int) -> Int
f (0,0) = 0
f (0,1) = 1
f (1,0) = 1
f (x,0) = x
f (0,y) = y
f (x,y) = x+y
```

A primeira coisa a se observar é a forma do tipo de entrada da função (Int, Int) com as entradas da função. Todas elas possuem o mesmo tamanho de tupla e os valores constantes que aparecem são do tipo Int , como 0 e 1 . Agora vamos executar tal função no GHCi:

```
Prelude> f (0,0)
0
```

Note que o argumento passado para a função foi $(0, 0)$, e este padrão foi encontrado na linha $f (0, 0) = 0$, logo o valor retornado será 0 . Observe:

```
Prelude> f (0,1)
1
```

O argumento passado foi $(0, 1)$, e o padrão encontrado está na linha $f (0, 1) = 1$, retornando então 1 . O padrão da linha seguinte, quarta linha da função f , é análogo a este.

Se calcularmos a expressão a seguir no GHCi:

```
Prelude> f (0, -90)
-90
```

O valor retornado é -90 , pois o padrão encontrado é o da sexta linha de f , ou seja, $f (0, y) = y$. O padrão $(0, y)$ pode

ser lido como: *valor 0 fixo na primeira coordenada e um valor qualquer na segunda coordenada, que chamaremos de y .*

O padrão $(x, 0)$ pode ser lido analogamente como *valor qualquer na primeira coordenada que chamaremos de x e um valor fixo 0 na segunda*. O nome para os valores variáveis podem ser repetidos em várias linhas, porém, não na mesma. Poderíamos ter o padrão $(0, x)$ em vez de $(0, y)$ sem problemas. Então, temos este outro exemplo:

```
Prelude> f (10,3)
13
```

Ele se encaixará no último padrão, pois, os únicos valores fixos são 0 e 1. Note que, no último padrão $f(x, y) = x+y$, teremos que x valerá 10, e y será igual a 3. Em suma, para tuplas, temos os seguintes padrões:

- $(0, 0)$ – valores fixos em ambas as coordenadas;
- $(x, 0)$ – valor variável na primeira e fixo na segunda;
- $(0, y)$ – valor fixo na primeira e variável na segunda;
- (x, y) – valores variáveis em ambas coordenadas (devem possuir nomes diferentes).

```
g :: (Int, Int) -> Int
g (7,7) = 7
g (_,_) = 0
```

Uma observação é que, sempre que o `_` (underline) aparecer, significa variável ignorada. Ou seja, tanto o valor da primeira

variável quanto o da segunda são variáveis, porém, com o `_`, o efeito é o de ignorar os valores. Neste caso:

```
Prelude> g (34,58)
0
```

Assim, 34 não é igual a 7, e 58 não é igual a 7, desviando a chamada da função para o padrão contido na segunda linha, `g (_,_) = 0`. Os valores 34 e 58 foram ignorados pelo `_`. É possível também ignorar o valor todo:

```
g :: (Int, Int) -> Int
g (7,7) = 7
g _ = 0
```

O efeito será o mesmo. Agora o exemplo será com listas:

```
h :: [Int] -> Int
h [] = 0
h (_:[]) = 1
h (_:x:[]) = 2+x
h (x:y:z:[]) = 3+x+y+z
h (x:_:_:w:[]) = 4+x+w
h (x:xs) = x
```

Do capítulo anterior, sabemos que `:` é o operador `cons`, e que `1:2:3:[]` se equivale a `[1,2,3]`. No GHCi, podemos calcular:

```
Prelude > h []
0
```

Ou seja, a função `h` calculada no valor fixo `lista vazia` produz o valor `0`. Note que o valor constante `[]` é fixo no primeiro padrão. Agora se calcularmos `h` com o argumento `[1,2,3]`:

```
Prelude > h [213]
1
```

Obteremos como retorno o valor `1` , pois o argumento é `[213] = 213:[]` e o padrão encontrado é o da linha `h (_,[])` `= 1` , que é o padrão para listas de **exatamente** um elemento. O elemento da lista `213` foi ignorado pelo `_` .

```
Prelude > h [5,6]
8
```

Nesse código temos que `[5,6] = 5:6:[]` , portanto, o padrão é o da linha `h (_,x:[]) = 2+x` , sendo que o elemento `5` foi ignorado pelo `_` , e o valor `6` atrelado à variável `x` . Assim, o valor de retorno é `2+6` , que é `8`. Este padrão `(_:x:[])` é para **exatamente** dois elementos.

```
Prelude > h [1,2,3]
9
Prelude > h [1,2,3,4]
9
```

Estes exemplos anteriores retornaram `9` em consequência de os argumentos terem sido `[1,2,3] = 1:2:3:[]` , que se encaixavam no padrão `h (x:y:z:[]) = 3+x+y+z` , como `x` valendo `1` , `y` valendo `2` e `z` valendo `3` . Já `[1,2,3,4] = 1:2:3:4:[]` se encaixa em `h (x:_:_:w:[]) = 4+x+w` , com `x` valendo `1` , `w` valendo `4` e os outros dois valores ignorados pelo `_` .

Esses dois padrões são para `3` e `4` elementos, respectivamente. Para testar o último padrão da função `h` , basta que a lista tenha mais de `4` elementos. Veja um exemplo:

```
Prelude > h [1,2,3,4,5]
1
```

Ou seja, como a lista tem cinco elementos, ela não bate com nenhum padrão de elementos fixos (aqueles que terminam com

[]), mas sim com o de elemento variável (que terminam com `xs` , qualquer outro nome ou `_`). Logo, temos `[1, 2, 3, 4, 5] = 1:[2, 3, 4, 5]` , com `x` valendo `1` e `xs` valendo `[2, 3, 4, 5]` .

Pode-se ler o padrão `(x:xs)` como **um ou mais elementos**, assim como `(x:y:xs)` **dois ou mais elementos**, `(x:_:_:_:a:as)` **cinco ou mais elementos**, e assim por diante. Só para fixar, um último exemplo: considere `[1, 2, 3, 4, 5, 6]` . Este se encaixa, por exemplo, em `(x:_c:b:xs)` , com `x=1` , `c=3` , `b=4` e `xs=[5, 6]` .

3.2 CAMPOS DE UM CONSTRUTOR

Todo *value constructor* pode possuir campos.

```
data Pessoa = Fisica String Int | Juridica String
```

Nesse exemplo, o tipo `Pessoa` possui dois *values constructors*: `Fisica` , que possui dois campos; e `Juridica` , que possui apenas um. Inspeccionando o novo tipo no GHCi, é possível notar que todo *value constructor* é, na realidade, uma função que retorna um valor do tipo `Pessoa` , e os campos são seus parâmetros.

```
Aula3> :t Fisica
Fisica :: String -> Int -> Pessoa
Aula3> :t Juridica
Juridica :: String -> Pessoa
```

Este conceito de chamar uma função com menos argumentos do que o esperado é chamado de *currying*, e será explorado em mais detalhes no próximo capítulo. Com a ajuda do *pattern matching*, vamos criar uma função teste que recebe uma `Pessoa` como parâmetro, e esta retornará uma tupla contendo duas Strings, informando nome e idade.

No caso de `Pessoa` do tipo `Fisica`, a função retornará uma mensagem que mostra o nome na primeira coordenada, assim como a idade na segunda. Para o *value constructor* `Juridica`, será mostrado o nome na primeira coordenada e uma mensagem informando que não há o campo idade na segunda.

```
teste :: Pessoa -> (String, String)
teste (Fisica x y) = ("Nome: " ++ x, "Idade: " ++ show y)
teste (Juridica x) = ("Nome: " ++ x, "Não há idade")
```

A primeira parte da função usa o *pattern matching* para encaixar os parâmetros `x` e `y` dentro do padrão encontrado no tipo `Pessoa String Int`, logo, `x` possui o tipo `String` e `y`, o tipo `Int`. A segunda parte procede da mesma forma, porém, o *value constructor* `Juridica` possui apenas um parâmetro `String`, logo, o *pattern matching* faz com que `x` seja `String`.

Uma outra maneira de se declarar um tipo é o uso da palavra `newtype`, porém, nesse caso, só podemos ter um *value constructor* e, no máximo, um campo. Por exemplo:

```
newtype Dado = Dado Int
```

Ele serve para uma espécie de apelido para um tipo existente (no caso, `Int`). Outra diferença é que `Dado` tem uma valoração preguiçosa, enquanto `newtype` não. Veja um exemplo:

```
newtype Dado = Dado Int
data Dado1 = Dado1 Int
```

Se tentarmos fazer alguma operação com o valor `Dado undefined`, teremos erro pela ausência da valoração preguiçosa, enquanto `Dado1 undefined` não causará erro algum, desde que você não acesse o campo. Note que o valor `undefined` está presente em todo tipo e representa a ausência de computação.

3.3 RECORD SYNTAX

Outra forma de declarar novos tipos com parâmetros é o *record syntax*, que permite escrever tais tipos usando nomes para cada parâmetro. O uso do *record syntax* permite extrair os valores contidos no campo (associado a *value constructor*) da mesma maneira que um *getter* da programação orientada a objetos (lembre-se de que aqui não há o conceito de modificadores de acesso ou coisa parecida).

O seu uso é simples. Apenas daremos nomes aos campos que os *values constructors* carregam e só. Fazendo isso, já estaremos usando este conceito. Por exemplo, o tipo `Ponto`, que pode ser definido como:

```
data Ponto = Ponto Double Double
```

Pode ser escrito assim:

```
data Ponto = Ponto {xval,yval :: Double}
```

A vantagem é a semântica de os campos terem nomes. E cada nome dado também pode ser usado como função de projeção de valores (algo parecido com o *getter* da programação orientada a objetos). Podemos inspecionar o tipo das funções de projeção `xval` e `yval` no GHCi.

```
Prelude> :t xval
xval :: Ponto -> Double
Prelude> :t yval
yval :: Ponto -> Double
```

Em suma, o *record syntax* provê, além de um nome aos campos de um *value constructor*, funções de projeção para obter o valor do campo correspondente.

Uma observação é que o *type constructor* `Ponto` pode ter o mesmo nome do seu *value constructor*, e isto não causa ambiguidade nenhuma ao compilador.

```
Aula3> Ponto 1.1 2
Ponto {xval = 1.1, yval = 2.0}
```

Para fazer uma função que calcula a distância do ponto à origem, podemos proceder de três formas.

1. Primeira forma:

```
distOrig :: Ponto -> Double
distOrig (Ponto x y) = sqrt(x**2 + y**2)
```

2. Segunda forma:

```
distOrig :: Ponto -> Double
distOrig (Ponto {xval=x, yval=y}) = sqrt(x**2 + y**2)
```

3. Terceira forma:

```
distOrig :: Ponto -> Double
distOrig p = sqrt(xval p**2 + yval p**2)
```

As duas primeiras maneiras usam o *pattern matching* para encaixar os parâmetros `x` e `y` no padrão que possui o tipo `Ponto`. Já a última usa `xval` e `yval`, funções de projeção adquiridas no uso do *record syntax*.

3.4 MINIPROJETO: RH DE UMA EMPRESA DE TI

Após a discussão dos conceitos ao final de alguns capítulos, um miniprojeto será mostrado de forma a "juntar" os conceitos vistos e motivar os leitores com aplicações um pouco mais práticas do que

os exemplos vistos. Esse miniprojeto aumentará de tamanho significativamente ao final de cada capítulo, até virar um minissistema pronto.

Lembre-se de que o foco deste livro não é o desenvolvimento web (que será pensado em próximas versões), mas sim plantar a semente da programação funcional. Acompanhar estes miniprojetos é de fundamental importância para o aprendizado. A história se passa em uma empresa de TI que quer fazer um sistema em Haskell para controle de seu RH, como experiência para a adoção da linguagem futuramente.

Então, temos empresa de TI com um pequeno sistema de RH e, nesse minúsculo sistema, são mantidos nomes e os cargos. O salário dos funcionários são constantes conforme o cargo (não há programadores que ganhem mais que os outros). Os cargos são: estagiário, programador, analista e gerente.

Há mobilidade de cargos na estrutura hierárquica e é gerada uma folha com os detalhes da empresa como um pequeno relatório, em formato JSON. Antes de começar, crie um arquivo chamado `Projeto.hs`.

```
module Projeto where
```

```
data Cargo = Estagiario | Programador | Coordenador | Gerente deriving Show
```

```
data Pessoa = Pessoa {cargo :: Cargo, nome :: String} deriving Show
```

```
verSalario :: Pessoa -> Double
verSalario (Pessoa Estagiario _) = 1500
verSalario (Pessoa Programador _) = 5750.15
verSalario (Pessoa Coordenador _) = 8000
verSalario (Pessoa Gerente _) = 10807.20
```

```

verFolha :: Pessoa -> String
verFolha p = "{nome: \"" ++ (nome p) ++
              "\", cargo: \"" ++ show (cargo p) ++
              "\", salario: " ++ show (verSalario p) ++ "}"

promover :: Pessoa -> Pessoa
promover (Pessoa Estagiario n) = Pessoa Programador n
promover (Pessoa Programador n) = Pessoa Coordenador n
promover (Pessoa Coordenador n) = Pessoa Gerente n
promover (Pessoa _ n) = Pessoa Gerente n

```

Foram criados os tipos `Cargo` e `Pessoa` (linhas 2 e 3 do programa). O tipo `Cargo` possui quatro *values constructors* (uso do `deriving Show` será analisado no capítulo 5, agora é só importante saber que essa cláusula deve ser colocada para a função `show` funcionar, e que a função `show` transforma qualquer tipo em uma `String`).

O tipo `Pessoa` possui um *value constructor* contendo dois campos (`nome` e `cargo`), escritos em formato de *record syntax*. A função `verSalario` usa o *pattern matching* contra o campo `cargo` do tipo `Pessoa`, e ignora o campo `nome`. Esta retorna um valor fixo de salário para cada *value constructor* do tipo `Cargo`.

A função `verFolha` recebe um parâmetro do tipo `Pessoa` e retorna uma representação deste tipo em formato `JSON`. Note que usamos o `++` para concatenar as strings e a função de projeção `cargo`, definida no *record syntax* do tipo `Pessoa`. Também usamos o retorno da função `verSalario` para montar esta estrutura.

Finalmente, a função `promover` recebe um parâmetro do tipo `Pessoa` e faz um *pattern matching* contra o campo `cargo`. Em

todos eles usou-se a letra `n` para representar o campo `nome` do tipo `Pessoa`. O último *pattern matching* de `promover` nos diz que não há promoção para cargos de `Gerente`.

3.5 EXERCÍCIOS

3.1) Crie o tipo `Pergunta` com os *values constructors* `Sim` ou `Nao`. Faça as funções seguintes, determinando seus tipos explicitamente.

- `pergNum` : recebe via parâmetro uma `Pergunta`. Retorna `0` para `Nao` e `1` para `Sim`.
- `listPergs` : recebe via parâmetro uma lista de `Perguntas`, e retorna `0`s e `1`s correspondentes aos constructores contidos na lista.
- `and'` : recebe duas `Perguntas` como parâmetro e retorna a tabela verdade do `and` lógico, usando `Sim` como verdadeiro e `Nao` como falso.
- `or'` : idem ao anterior, porém deve ser usado o `ou` lógico.
- `not'` : idem aos anteriores, porém usando o `not` lógico.

3.2) Faça o tipo `Temperatura` que pode ter valores `Celsius`, `Fahrenheit` ou `Kelvin`. Implemente as funções:

- `converterCelsius` : recebe um valor `double` e uma temperatura, e faz a conversão para `Celsius`.
- `converterKelvin` : recebe um valor `double` e uma temperatura, e faz a conversão para `Kelvin`.
- `converterFahrenheit` : recebe um valor `double` e uma temperatura, e faz a conversão para `Fahrenheit`.

3.3) Implemente uma função que simule o vencedor de uma partida de pedra, papel e tesoura usando tipos criados. Casos de empate devem ser considerados em seu tipo.

3.4) Faça uma função que retorne uma string, com todas as vogais maiúsculas e minúsculas eliminadas de uma string passada por parâmetro usando *list comprehension*.

3.5) Sabe-se que as unidades imperiais de comprimento podem ser Inch , Yard ou Foot (há outras ignoradas aqui). Sabe-se que $1\text{in}=0.0254\text{m}$, $1\text{yd}=0.9144\text{m}$, $1\text{ft}=0.3048$. Faça a função `converterMetros` que recebe a unidade imperial e o valor correspondente nesta unidade. Esta função deve retornar o valor em metros.

Implemente também a função `converterImperial` , que recebe um valor em metros e a unidade de conversão. Esta função deve retornar o valor convertido para a unidade desejada.

3.6) Faça um novo tipo chamado `Mes` , que possui como valores todos os meses do ano. Implemente:

- A função `checaFim` , que retorna o número de dias que cada mês possui (considere fevereiro tendo 28 dias).
- A função `prox` , que recebe um mês atual e retorna o próximo mês.
- A função `estacao` , que retorna a estação do ano de acordo com o mês e com o hemisfério.

Use apenas tipos criados pela palavra `data` aqui.

3.7) Faça uma função que receba uma `String` e retorne `True` se esta for um palíndromo; caso contrário, `False` .

3.8) Faça uma função que elimine todos os números pares, todos os ímpares múltiplos de 7 e negativos de uma lista de inteiros passada via parâmetro. Você deve retornar esta lista em ordem reversa em comparação a do parâmetro.

3.9) Faça uma função que recebe três Strings `x`, `y` e `z` como parâmetro. A função retorna uma tupla com três coordenadas contendo a ordem reversa em cada. A primeira coordenada deve conter string reversa do primeiro parâmetro, e assim por diante.

3.10) Faça uma função chamada `revNum`, que receba um `Int n` e uma `String s` (nessa ordem). Esta deverá retornar as `n` primeiras letras em ordem reversa e o restante em sua ordem normal. Exemplo:

```
revNum 4 "FATEC" = "ETAFC"
```

3.11) Crie o tipo de dado `Binario` que pode ser `Zero` ou `Um`. Faça outro tipo de dado chamado `Funcao` que pode ser `Soma2`, `Maior`, `Menor` ou `Mult2`. Implemente a função `aplicar` que recebe uma `Funcao` e dois `Binarios`. Seu retorno consiste em executar a operação desejada. Exemplo:

```
aplicar Soma2 Um Um = Zero
```

3.12) Faça uma função chamada `binList`, usando *list comprehension*, que recebe uma lista de `Binarios` (ver exercício anterior) e retorna outra lista com elemento somado `Um` e convertido para `Int`. Exemplo:

```
binList [Um, Zero, Zero, Um, Zero] = [0,1,1,0,1]
```

3.13) Faça um novo tipo chamado `Metros`, que possui um *value constructor* de mesmo nome, cujos parâmetros são: `Int`,

que representa a dimensão, `Double`, que representa o valor da medida e outro chamado `MetragemInvalida`. Implemente as funções:

- `areaQuadrado :: Metros -> Metros` : calcula a área de um quadrado.
- `areaRet :: Metros -> Metros -> Metros` : calcula a área de um retângulo.
- `areaCubo :: Metros -> Metros` : calcula a área de um cubo.

Exemplo:

```
Prelude> areaQuadrado (Metros 1 2.0)
Metros 2 4.0
```

Use *pattern matching* para ignorar metragens erradas (calcular a área de um quadrado com lado 4 não é válido).

3.14) Faça o novo tipo `Valido` que possui dois *value constructors* `Sim` e `Nao`. O *value constructor* `Sim` possui um parâmetro (campo) `String`. Implemente uma função `isNomeValido` que recebe um nome e retorna `Nao` caso a `String` seja vazia; caso contrário, `Sim`.

3.15) Refaça o exercício 3 do capítulo anterior usando *record syntax* e tipos com parâmetro (siga o exemplo da conversão de medidas SI para imperial).

3.16) Faça o tipo `Numero`, que possui um *value constructor* `Ok` com um campo `double` e outro *value constructor* `Erro` com um campo `String`. Faça a função `dividir` que divida dois números e, caso o segundo número seja 0, emita um erro (use o *pattern matching*).

Exemplo:

```
Prelude> dividir (Numero 6) (Numero 5)
Numero 1.2
```

3.17) Faça o tipo `Cripto` que possua dois *values constructors* `Mensagem` e `Cifrado`, ambos com um campo `String` e um *value constructor* `Erro`. Faça as funções `encriptar` e `decriptar`, seguindo cada exemplo a seguir.

```
Prelude> encriptar (Mensagem "FATEC")
Cifrado "GBUFD"
```

```
Prelude> decriptar (Cifrado "DBTB")
Mensagem "CASA"
```

Veja que a encriptação deve empurrar cada letra a frente e a decriptação faz o inverso, empurrando uma letra para trás. Use as funções `succ` e `pred`, e também *list comprehensions*. Não é possível encriptar mensagens cifradas e decriptar mensagens.

3.18) Faça uma função `encriptarTodos` que encripta (ou dá erro) todos os elementos de um vetor de `Cripto`.

3.19) Tendo como base o exercício de conversão de medidas, crie uma função que faça conversão de câmbio. Você deve criar o tipo `Cambio` contendo os *value constructors* `Euro`, `Real` e `Dollar`.

Crie também o tipo `Moeda` que possui os campos `val :: Double` e `cur :: Cambio`. Use *record syntax* e as taxas de conversão do dia no qual você fez o exercício.

3.20) Crie a função `converterTodosReal` que recebe uma lista de moedas e retorna outra lista de moedas com todos os seus elementos convertidos para `Real`. Use *list comprehension*.

3.21) Crie a função `maxMoeda` que recebe uma lista de moedas e retorna o valor máximo absoluto (sem conversão alguma) dentre os campos `val` desta lista. Exemplo:

```
Prelude> maxMoeda [Moeda 3 Real, Moeda 7 Dollar, Moeda 1 Euro]  
7
```

Use a função `maximum`.

3.6 CONCLUSÃO

Neste capítulo, vimos como manipular tipos de dados, o conceito de *record syntax* com suas funções de projeção semelhantes aos *getters* da programação orientada a objetos, e o importantíssimo conceito de *pattern matching*, que facilita escrever funções com comportamentos que dependem de *value constructors*.

Este capítulo possui vários exercícios que devem ser feitos, além de ser o primeiro a propor um miniprojeto de uma aplicação feita em Haskell. A compreensão deste capítulo é fundamental para o domínio do funcionamento desta linguagem. Afinal, temos uma linguagem com um sistema de tipos brilhante que pode ajudar a reduzir significativamente os custos com testes de *software*, como explicado na introdução deste livro.

UM POUCO MAIS SOBRE FUNÇÕES

Em programação funcional, funções são tratadas como valores comuns, assim como o 5 pertence ao tipo `Int` e "Olá" ao tipo `String`. Ou seja, funções podem ser passadas como parâmetro e retornadas por outras funções. Além disso, se uma função espera três parâmetros e você passar dois, não há problema nisso. Podemos criar funções sem corpo, como vimos até aqui.

Este capítulo é dedicado a entendermos mais a fundo como o Haskell trabalha a estrela maior da linguagem: funções. Trabalharemos também um pouco de sintaxe dentro de funções e recursão.

4.1 LAMBDA

Funções podem ser usadas como valores e não necessariamente em um contexto explícito, como fizemos até aqui. Por exemplo, não é preciso declarar a função em um contexto para usá-la. Em vez disso, é possível usá-las naturalmente como se estivessem declaradas. De uma maneira geral, *lambdas* têm a seguinte forma:

$$\lambda p_1 p_2 p_3 p_4 \dots p_n \rightarrow \text{EXPR}(p_1, p_2, p_3, p_4, \dots, p_n)$$

Isso pode ser lido como: receba os parâmetros p_1 , p_2 , p_3 , p_4 , ... , p_n , e devolva uma expressão que dependa destes n parâmetros. Veja um exemplo:

```
Prelude> (\x -> 2*x) 4
8
```

Ele indica que estamos usando um *lambda* que recebe um parâmetro x e devolve o dobro dele mesmo. Uma expressão *lambda* só pode ser usada em contextos locais. Veja outro exemplo:

```
Prelude> (\x xs -> x : reverse xs) 'A' "UOIE"
"AEIOU"
```

Este recebe um elemento x e uma lista de elementos xs , devolvendo o primeiro elemento na frente da lista xs em ordem reversa. Todos os conceitos apresentados neste capítulo valem sempre para expressões *lambda*.

4.2 FUNÇÕES DE ALTA ORDEM

Como dito na introdução, funções podem ser passadas como parâmetro ou retornar outras funções. Uma função que possua alguma das duas características citadas é chamada de **função de alta ordem**, ou *high-order function*.

```
ev :: (Int -> Int) -> Int
ev f = 1 + f 5
```

Isto é uma função de alta ordem. O tipo de `ev :: (Int -> Int) -> Int` consiste em uma entrada do tipo `Int -> Int` e uma saída inteira. Os parênteses indicam que a entrada tem o tipo de uma função em vez de dois inteiros.

A expressão de retorno de `ev` pode ser em um primeiro momento esquisita, porém é de fácil entendimento. Ela significa: calcule a função `f`, que foi recebida como argumento, ao valor `5` e depois some `1`. Considere as funções auxiliares:

```
dobro :: Int -> Int
dobro x = 2*x
```

```
triplo :: Int -> Int
triplo x = 3*x
```

Vamos testar a chamada de `ev` no GHCi:

```
Prelude> ev dobro
11
Prelude> ev triplo
16
```

Tal chamada nos mostra que, primeiramente, a função `dobro` foi argumento de `ev` e depois `triplo`. Vamos analisar de perto o que aconteceu com a primeira (a outra chamada segue o mesmo raciocínio).

$$\text{ev dobro} = 1 + \text{dobro } 5 = 1 + 2*5 = 1 + 10 = 11$$

A função `dobro` foi plugada em `f`, assim, ela é chamada com o argumento `5`, resultando em `10` como retorno. Neste retorno foi somado `1` ao final, dando `11`. Exemplos de funções que retornam outras terão mais sentido nas seções seguintes e serão suprimidos no momento.

Observe que é possível usar *lambdas* em vez das funções `dobro` e `triplo`, como segue:

```
Prelude> ev (\x -> 2*x)
11
Prelude> ev (\x -> 3*x)
16
```

Escrevendo desta maneira, é permitido uma maior produtividade, pois, não será necessário escrever o corpo das funções e recarregar o módulo novamente.

4.3 CURRYING

Currying é uma técnica que consiste em transformar a chamada de uma função (retorno valorado), que recebe múltiplos argumentos, em uma avaliação de uma sequência de funções. Você pode fixar uma quantidade de argumentos e deixar o restante variável.

```
somarTresNum :: Int -> Int -> Int -> Int
somarTresNum x y z = x+y+z
```

```
somarCurr :: Int -> Int
somarCurr = somarTresNum 4 5
```

A função `somarCurr` possui fixo os parâmetros `x` e `y` da função `somarTresNum`, deixando `z` livre para variar. Portanto, podemos inspecionar o tipo de `somarCurr`.

```
Prelude> :t somarCurr
somarCurr :: Int -> Int
```

Como esperado, o tipo de `somarCurr` é `Int`, pois, apenas uma variável foi mantida livre na definição de `somarCurr`. O interessante é que podemos agrupar as últimas duas setas usando parênteses no tipo de `somarTresNum`. Veja:

```
somarTresNum :: Int -> Int -> (Int -> Int)
```

Fazendo isso, fica claro que, a partir da função `somarTresNum`, se passarmos dois argumentos a ela, o retorno será uma função de um parâmetro `Int` e seu retorno `Int`,

fazendo com que ela se torne de **alta ordem**. Sempre que houver parênteses encobrindo flechas e tipos, consideraremos que o parâmetro ou o retorno é uma função.

Em suma, sempre que uma função tiver algum parâmetro suprimido, o retorno será uma função. Esta possuirá como parâmetro todas as variáveis livres, e sua avaliação levará em conta todos os parâmetros que foram deixados fixos e os que ficaram livres também.

```
Prelude> somarCurr 1  
10
```

Agora vamos analisar um exemplo mais trabalhado, usando tipos e a estrutura de um programa em Haskell.

4.4 EXEMPLOS DE FUNÇÕES DE ALTA ORDEM

Nesta seção, serão apresentados ao leitor alguns exemplos de funções de alta ordem que facilitam o dia a dia do programador que utiliza este paradigma. Também será demonstrada a utilidade dos conceitos de funções de alta ordem e *currying*.

Map/foldl/filter

A função `map` tem como objetivo aplicar uma função `f`, recebida via parâmetro, a todos os elementos de uma lista `l`, também recebida via parâmetro. O retorno desta função é uma nova lista, na qual seus elementos são as saídas da função `f`, tendo como argumento os elementos da lista.

```
Aula4> :t map
```

```
map :: (a -> b) -> [a] -> [b]
```

O tipo de entrada da lista deve ser o mesmo tipo da entrada da função e o tipo da saída será o mesmo da saída da função. Usando o conceito de *currying* e *funções de alta ordem*, podemos enxergar o `map` como:

```
map :: (a -> b) -> ([a] -> [b])
```

Ao suprimir a lista de tipo `[a]`, como parâmetro da função `map`, o retorno obtido é uma função do tipo `[a] -> [b]`. Ou seja, sua função foi levantada de `a -> b` para `[a] -> [b]`, isto é, a função que inicialmente trabalhava com estruturas simples passa agora a trabalhar com listas.

```
Aula4> map (+2) [1..5]  
[3,4,5,6,7]
```

Neste exemplo, a função `map` recebe via parâmetro a função `(+2)`, que é um *currying* da função `(+)`, tendo o valor `2` fixo em seu segundo argumento, e distribui essa função a todos os elementos da lista `[1,2,3,4,5]`. Assim, é efetuado: `1+2`, `2+2`, `3+2`, `4+2` e `5+2`. Perceba que o uso do `map` se assemelha ao *list comprehension* visto no primeiro capítulo.

Para a função `foldl`, devem ser passados uma função `f` com dois parâmetros e também um valor inicial. Essa função dobra a lista começando da esquerda, isto é, a função `f` é aplicada ao valor inicial e ao primeiro elemento da lista. O retorno dela mais o segundo elemento devem ser os parâmetros para a nova aplicação de `f` até acabarem os elementos da lista. Você pode pensar que a função `foldl` se comporta como um acumulador se comparada ao paradigma imperativo.

```
Aula4> :t foldl
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

O primeiro parâmetro (de `foldl`) deve ser uma função na qual sua primeira entrada seja o mesmo tipo `b` que o valor inicial (segundo parâmetro de `foldl`), e sua segunda entrada com o mesmo tipo `a` dos elementos contidos na lista do terceiro parâmetro (de `foldl`).

```
Aula4> foldl (+) 0 [1..4]
10
```

A função soma `(+)` terá como parâmetro o valor inicial `0` e o primeiro valor da lista `1`. A função soma será aplicada a estes dois parâmetros, resultando em `1`. O valor `1` do acúmulo (`0+1`) será um novo parâmetro, junto ao valor `2` da lista, e ambos sofrerão a aplicação da soma novamente, resultando em `3`. Esse valor `3`, junto ao valor `3` da lista, sofrerá a aplicação de `(+)` novamente, resultando em `6`. O valor `6` e o elemento `4` da lista sofrerão pela última vez a aplicação de `(+)`, resultando em `10`.

```
(+) 0 1 [2,3,4]
(+) 1 2 [3,4]
(+) 3 3 [4]
(+) 6 4 []
10
```

Veha que, em Haskell, `0 + 1` é a versão infixa de `(+) 0 1`, e ambas produzem o mesmo valor, `1`.

```
Aula4> foldl (\xs x -> x : xs) [] "FATEC"
"CETAF"
```

Isto produzirá as seguintes chamadas:

```
'F':[] "ATEC"
'A': 'F':[] "TEC"
'T': 'A': 'F':[] "EC"
'E': 'T': 'A': 'F':[] "C"
```

```
'C': 'E': 'T': 'A': 'F': [] ""  
"CETAF"
```

Observe que $(\lambda x. x : xs)$ é um *lambda*, que nada mais é do que uma função sem corpo. Esta função recebe uma lista xs e um valor x , e retorna o elemento x inserido à frente da lista xs .

O `filter` é uma função que recebe uma outra função f de retorno booleano e uma lista de elementos. Ele retorna uma outra lista contendo os elementos que foram argumentos de f e que tiveram `True` como retorno.

```
Aula4> filter (>0) [-4..4]  
[1,2,3,4]
```

Neste exemplo, é filtrado todos os elementos maiores que zero. A função recebida (>0) se equivale ao *lambda* $\lambda x. x > 0$, que recebe um valor x . Este retorna `True` caso seja maior que zero, e `False` caso contrário. Os elementos `[1,2,3,4]` são todos que satisfazem a condição da função (>0) e, no caso, são retornados pela função `filter`.

```
Aula4> :t filter  
filter :: (a -> Bool) -> [a] -> [a]
```

O tipo da função `filter` nos diz que a função do primeiro parâmetro precisa retornar uma expressão booleana. A entrada da função do primeiro parâmetro deve ter um tipo a , que deve ser o mesmo tipo dos elementos do segundo parâmetro, de tipo $[a]$.

Função .

A composição de funções, introduzida no segundo capítulo, é uma noção muito importante em programação funcional e merece

toda atenção aqui nesta seção. A composição de funções é algo cotidiano na vida do programador. É o simples ato de chamar duas funções, ou mais, sendo que o retorno de uma é o argumento da outra.

Para acompanhar os exemplos a seguir, vamos considerar as funções:

```
traseira :: String -> String
traseira [] = []
traseira (x:xs) = xs
```

```
contar :: String -> Int
contar = length
```

A primeira função retorna uma lista vazia, caso o argumento seja vazio, e o fim da lista `xs`, caso o argumento tenha pelo menos um elemento (ignorando assim o primeiro elemento e devolvendo o resto). A função `contar` é apenas uma renomeação da função `length`, que conta elementos de uma lista de qualquer tipo. Nesse caso, deixamos a entrada como `String` por questões didáticas.

Na expressão `contar(traseira "Haskell")`, a função `traseira` será executada em um primeiro momento e terá como retorno `"askell"`. Após este passo, obtemos a chamada `contar "askell"` - note que o retorno de `traseira` é o argumento de `contar` - que produzirá o retorno `6`.

O exemplo parece simples, porém, na programação funcional, tem um motivo teórico muito bem fundamentado. Antes de continuar, note que `contar(traseira "Haskell")` se equivaleria a `contar(traseira("Haskell"))`, que é um estilo corriqueiro nas linguagens de programação imperativa.

A expressão em destaque `contar(traseira "Haskell")` pode ser melhorada e escrita de uma forma mais próxima da matemática, usando a função infixa `.` (ponto). O nome desta função vem da notação matemática para composição de funções.

```
Prelude> (contar . traseira) "Haskell"  
6
```

Essa função `.` é de alta ordem. Basta checar seu tipo:

```
Prelude> :t (.)  
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

Ela diz o seguinte: essa função recebe duas outras de tipos `(b -> c)` e `(a -> b)`, retornando uma outra de tipo `(a -> c)`. Observe que `a`, `b` e `c` são qualquer tipo da linguagem (`Int`, `Double`, `[Int]`, `String`, `(Int, String)` etc.).

Retomando a frase "o retorno de uma função deve ser o argumento da outra", é possível ver que o tipo incumbido de realizar esta façanha é o `b`, pois é a saída do segundo parâmetro de tipo `(a -> b)` e a entrada do primeiro parâmetro de tipo `(b -> c)`. Portanto, o tipo da função composta (retorno de `.`) se dá pela entrada da primeira função `a` e pela saída da segunda `c`.

Vamos analisar agora o exemplo da expressão `(contar . traseira) "Haskell"`:

```
Prelude> :t traseira  
traseira :: String -> String  
Prelude> :t contar  
contar :: String -> Int
```

Podemos agora inspecionar a função composta dada pela expressão `contar . traseira`. O primeiro argumento de `.` é a função `contar`, portanto, `b=String` e `c=Int`, pois, o tipo de

`contar` é `String -> Int`, e a função `conta` com um argumento de tipo `b -> c`.

Analogamente, o segundo argumento de `contar` . `traseira` é a função `traseira`, que tem o tipo `String -> String`. Como a função `.` espera algo do tipo `a -> b`, temos que `a=String` e `b=String`, concluindo que o tipo do retorno de `contar . traseira` é `String -> Int`, já que o retorno de `.` é `a -> c`.

É bom observar `b=String`, nos dois casos, fazendo que com que a composição funcionasse corretamente (saída da segunda função é a entrada da primeira). Se tentássemos compor `traseira . contar`, obteríamos um erro de compilação, pois `traseira` espera uma `String` e o retorno de `contar` é um `Int`.

Função \$

A função infixa `$` recebe uma função e um valor, e aplica a função neste valor. Veja um exemplo:

```
Prelude> contar $ "01a"  
3
```

Aparentemente, não há diferença alguma com uma chamada de função simples:

```
Prelude> contar "01a"  
3
```

Porém, a diferença está quando usamos os parênteses para dar prioridade nas operações. Quando calculamos o tamanho de uma lista usando a concatenação, devemos, por exemplo, usar parênteses:

```
Prelude> contar ("Ola" ++ "Alo")  
6
```

Fazemos isso para indicar que a concatenação será efetuada e seu retorno entrará na função `contar` .

A função `$` é uma maneira fácil de se livrar da poluição causada pelo uso excessivo de parênteses - como é habitual em muitas linguagens -, deixando o código mais claro.

```
Prelude> contar $ "Ola" ++ "Alo"  
6
```

Como `$` possui uma precedência à direita maior que `++` , primeiro será calculado `"Ola" ++ "Alo"` , e depois `contar "OlaAlo"` , retornando `6` . Inspeccionando o tipo de `$` , temos:

```
Prelude> :t ($)  
($) :: (a -> b) -> a -> b
```

Ela recebe uma função que recebe uma função de tipo `a` e retorno `b` , e um valor de tipo `a` devolvendo um valor de tipo `b` . Em nosso exemplo, o primeiro argumento de `$` é a função `contar` . Desta forma, `a` é uma `String` e `b` um `Int` , pois o tipo de `contar` é `String -> Int` . Já o segundo argumento é a `String "OlaAlo"` , tendo `6` como retorno e este do tipo `Int` .

Tanto `.` quanto `$` são funções infixas definidas na base do GHC. O próximo exemplo será criado por mim e ajudará bastante em alguns conceitos que serão abordados nos próximos capítulos.

Função |>

Podemos nos valer do poder que a linguagem Haskell nos oferece para criar a seguinte função infix:

```
(|>) :: a -> (a -> b) -> b
(|>) x f = f x
```

```
infixl 9 |>
```

Vale ressaltar que o operador que acabamos de definir, existe na linguagem e está definido no módulo `Data.Function` com o nome `(&)`. Para não confundir o leitor, pois `(&)` pode ser facilmente confundido com `(&&)` (`and` lógico), usaremos este `(|>)` que acabamos de definir (ou seja, `(|>) = (&)`). Ela recebe um valor de tipo `a`, e uma função que recebe `a` e retorna `b`, retornando assim um valor de tipo `b`. Note que essa função é apenas `$` com os parâmetros trocados de ordem.

O `infixl` indica alta precedência à esquerda. A linguagem de programação funcional para **front-end** Elm usa muito esta função e, em seu ambiente, este conceito é chamado de **pipelining**. Aqui em Haskell, este operador terá mais utilidade mais à frente, porém usando-o é possível reescrever a seguinte função:

```
func :: String -> String
func x = x ++ (tail (take 3 (reverse x)))
```

Da seguinte forma:

```
funcI :: String -> String
funcI x = x
    |> reverse
    |> take 3
    |> tail
    |> (x ++)
```

Isso tem um estilo parecido com as linguagens imperativas, dando a impressão de que está sendo executada uma instrução por linha. Primeiramente, é importante frisar que `func` e `funcI` são as mesmas funções escritas, mas em estilos diferentes.

A função `func` foi escrita usando composição de funções, como de costume. Nela, concatenaremos a expressão `(tail (take 3 (reverse x)))` ao parâmetro recebido `x`. Vale ressaltar que `(tail (take 3 (reverse x)))` pode também ser escrito como `(tail . (take 3) . reverse) x`, ou `tail . (take 3) . reverse $ x`. Essa expressão nos diz que o retorno de `reverse` entrará na função `take 3`. Graças ao conceito de *currying*, o retorno dela entrará em `tail`, finalizando com a concatenação.

Em `funcI`, temos o mesmo fenômeno, porém em outra escrita, sendo esta mais parecida com linguagens imperativas. Nela, temos que o parâmetro `x` entra em `reverse`, e seu retorno é passado para `take 3`. Lembre-se sempre do *currying* aqui, depois para `tail` e depois para a expressão `(x ++)`, que é a concatenação de `x` com o valor passado em cada função desde o começo (note o uso do *currying* aqui também).

Este último exemplo foi muito rico e usou todos os conceitos discutidos neste capítulo. É de extrema importância o entendimento da função infixa `|>`. Com ela, o leitor estará preparado para encarar o conceito de **Monads**, que é tão importante neste linguagem. A partir dele, conseguiremos nos conectar com o mundo real, ou seja, aplicações web, arquivos, banco de dados etc.

4.5 SINTAXE EM FUNÇÕES

Os *guards* são uma maneira de testar várias condições em uma função, de maneira similar a um `if` encadeado. Por exemplo, se quisermos calcular o IMC de uma pessoa e, a partir deste valor,

mostrar uma mensagem na tela indicando se ela está acima do peso ou não, é possível usando *guards*. Podemos escrever as condições de uma maneira limpa.

```
imc p a
| p/(a*a) <= 18.5 = "Abaixo do peso"
| p/(a*a) < 25.0 = "Peso ideal"
| p/(a*a) <= 30 = "Acima do peso"
| otherwise = "Obesidade"
```

Um possível tipo da função anterior é `Double -> Double -> Double`. Os parâmetros `p` e `a` representam peso e altura. A expressão `p/(a*a)` representa o cálculo do IMC. A partir deste cálculo, as condições são verificadas em ordem até que alguma seja `True` e o retorno da função (mensagem) será executada.

Caso a condição seja `False`, a próxima condição será verificada até chegar ao `otherwise`, que sempre será `True`. A cláusula `where` pode ajudar a facilitar a escrita da função de IMC.

```
imc p a
| valorImc <= 18.5 = "Abaixo do peso"
| valorImc < 25.0 = "Peso ideal"
| valorImc <= 30 = "Acima do peso"
| otherwise = "Obesidade"
where
    valorImc = p/(a*a)
```

O uso do `where` ajuda a não escrever a expressão `p/(a*a)` em toda condição. Note que, para cada padrão do *pattern matching*, é possível ter *guards* próprios.

4.6 RECURSÃO

Recursão é um método de resolução de problemas que consiste na solução de pequenas instâncias do problema até achar a solução

global. A recursão precisa de uma condição de base (de parada ou inicial) para que não se caia em *loops* infinitos.

Em Haskell, é uma técnica fundamental para resolver problemas, pois, não há instruções de *loop* como `repeat` ou `for`.

```
fat n
| n <= 1 = 1
| otherwise = n*fat(n-1)
```

Essa expressão realiza o cálculo de um fatorial que é um problema recursivo. A condição base é quando temos `fat 1 = 1`. Como `fat 0 = 1` e não queremos que a função exploda com valores negativos, a condição `n <= 1` foi escolhida (por enquanto, é a melhor opção).

Um possível tipo para a função anterior é `Int -> Int`. Para se reverter a ordem de uma `String`, é possível proceder de forma recursiva, como mostra o trecho de código adiante.

As operações são realizadas conforme a ordem de chamada da função. Por exemplo, se calcularmos o fatorial de 5 :

```
Prelude> fat 5
120
```

A primeira expressão a ser calculada é `fat 5 = 5*fat 4`, e a próxima chamada será `fat 4`. O quadro seguinte mostra a ordem das operações:

```
fat 5 = 5*fat 4
fat 4 = 4*fat 3
fat 3 = 3*fat 2
fat 2 = 2*fat 1
```

Na memória, as chamadas são empilhadas, criando algo parecido como o quadro:

```
fat 1
fat 2
fat 3
fat 4
fat 5
```

Cada retorno não recursivo da função faz com que a chamada seja eliminada da memória. Em nosso caso, a chamada `fat 1` corresponde a um retorno não recursivo conforme definido, logo, `fat 1` retornará `1` e a próxima expressão a ser calculada está em `fat 2`. O quadro a seguir mostra a ordem de cálculo das expressões.

```
fat 1 = 1
fat 2 = 2*fat 1 = 2*1 = 2
fat 3 = 3*fat 2 = 3*2 = 6
fat 4 = 4*fat 3 = 6*4 = 24
fat 5 = 5*fat 4 = 24*5 = 120.
```

O próximo exemplo ilustra chamadas recursivas usando o conceito de *pattern matching*.

```
reverse' :: String -> String
reverse' [] = []
reverse' (x:xs) = reverse' xs ++ [x]
```

Se a função `reverse'` for chamada com um argumento `"Haskell"`, obteremos `"lleksaH"`. O padrão `(x:xs)` quebra a String `"Haskell"` em `'H':"askell"`. A expressão da função `reverse'` concatena a chamada da função novamente com o argumento `"atec"`, concatenando a String `"F"` (equivalente a `['F']`) ao final, trocando assim a ordem da palavra.

A função procede recursivamente até achar a condição de parada `reverse' []=[]`, que quer dizer que se a String vazia for encontrada, será retornada ela mesma, não modificando a palavra a ser revertida e encerrando o processo recursivo.

Analogamente ao exemplo anterior, podemos ver o mesmo processo para esta função:

```
reverse' "Haskell" = reverse' "askell" ++ ['H']
reverse' "askell" = reverse' "skell" ++ ['a']
reverse' "skell" = reverse' "kell" ++ ['s']
reverse' "kell" = reverse' "ell" ++ ['k']
reverse' "ell" = reverse' "ll" ++ ['e']
reverse' "ll" = reverse' "l" ++ ['l']
reverse' "l" = reverse' [] ++ ['l']
```

E na memória:

```
reverse' []
reverse' "l"
reverse' "ll"
reverse' "ell"
reverse' "kell"
reverse' "skell"
reverse' "askell"
reverse' "Haskell"
```

Como `reverse' []` é uma chamada não recursiva (caso base), obtemos o quadro a seguir contendo a ordem do cálculo das expressões:

```
reverse' [] = []
reverse' "l" = reverse' [] ++ ['l'] = "l"
reverse' "ll" = reverse' "l" ++ ['l'] = "ll"
reverse' "ell" = reverse' "ll" ++ ['e'] = "lle"
reverse' "kell" = reverse' "ell" ++ ['k'] = "llek"
reverse' "skell" = reverse' "kell" ++ ['s'] = "lleks"
reverse' "askell" = reverse' "skell" ++ ['a'] = "lleksa"
reverse' "Haskell" = reverse' "askell" ++ ['H'] = "lleksaH".
```

Observe que `reverse'` e `reverse` são iguais. O conceito de recursão que acabamos de ver é de importância na substituição de blocos de repetição (`do` , `while` , `for`) usados em outras linguagens de programação.

4.7 MINIPROJETO: RH DE UMA EMPRESA DE TI

Neste capítulo, continuaremos o projeto iniciado, mantendo as funções, o mesmo arquivo `Projeto.hs` e também o mesmo cabeçalho. Aqui serão implementadas funções para contratar um funcionário novo, calcular a média salarial de uma lista de pessoas, contratação de vários funcionários em cargos iniciais e uma rotina para promoção.

```
contratarInicial :: String -> Pessoa
contratarInicial = Pessoa Estagiario

mediaSalarial :: [Pessoa] -> Double
mediaSalarial ps = (foldl calculo 0 ps) / (fromIntegral $ length
ps)
                                where
                                calculo salario pessoa = salario + verSala
rio pessoa

contratarVariosEstag :: [String] -> [Pessoa]
contratarVariosEstag ps = map (contratarInicial Estagiario) ps

rotinaPromocao :: Pessoa -> String
rotinaPromocao p = p
                |> promover
                |> verFolha
```

A função `contratarInicial` age como se fosse um construtor – sim, aquele velho conhecido da programação orientada a objetos. O cargo sempre será `Estagiario`. Esta

função é de alta ordem, pois não recebe entrada alguma e retorna uma função do tipo `String -> Pessoa`.

Perceba que aqui está sendo usado o conceito de *currying*, já que `Pessoa Estagiario` não é um valor do tipo `pessoa`, mas sim `String -> Pessoa`. Ou seja, para a chamada ser completa e o tipo ser `Pessoa`, necessita-se do nome.

A função `contratarVariosEstag` usa o `map` para jogar a função `contratarInicial Estagiario`. Não esqueça do *currying*, para dentro desta lista de `String ps`. Pelo funcionamento do `map`, todo elemento `String` da lista `ps` será argumento de `contratarInicial Estagiario`, e seu retorno será uma `Pessoa` contratada pela empresa.

Na função `mediaSalarial`, a cláusula `where` é usada de forma a facilitar a sua escrita. O tipo da função interna `cálculo` é `Double -> Pessoa -> Double`, que é exatamente um tipo de função exigida pelo `foldl`. Lembre-se de que tal função deve receber algo do mesmo tipo que `0` e um elemento de mesmo tipo que a lista `ps`, e também deve retornar algo do mesmo tipo que `0`.

A função `calculo` soma o salário que vem do primeiro argumento do `foldl` com `verSalario pessoa`, que verifica o salário de uma pessoa que é elemento do segundo argumento do `foldl` (no caso, `ps`). A soma realizada pelo `foldl` é dividida pela quantidade de elementos dada pela expressão `fromIntegral $ length ps`. Esta converte o tamanho inteiro da lista para um `Double`, usando a função `fromIntegral`. Note o uso de `$` para evitar os parênteses.

Finalmente, a rotina de promoção simplesmente promove um funcionário e exibe sua nova folha. Como duas ações estão sendo realizadas, a composição é uma das alternativas de escrita, porém, usamos o `|>`. A função `rotinaPromocao` é lida como: "Promova `p` e depois veja sua folha". A mesma função pode ser escrita também usando `.` e `$`, como:

```
rotinaPromocao :: Pessoa -> String
rotinaPromocao p = verFolha . promover $ p
```

Isso reforça que `|>` dá um som mais parecido com as linguagens imperativas.

4.8 EXERCÍCIOS

4.1) Faça uma função que retorne a média de um `[Double]`, usando `foldl`.

4.2) Faça uma função que receba uma `[String]` e retorne todos os elementos palíndromos. Ver exercício 3.7.

4.3) Implemente uma função que filtre os números pares e outra que filtre os ímpares de uma lista recebida via parâmetro.

4.4) Filtre os números primos de uma lista recebida por parâmetro.

4.5) Implemente uma função que receba uma lista de inteiros e retorne o dobro de todos, eliminando os múltiplos de 4.

4.6) Faça uma função `func` que receba uma função `f` de tipo `(String -> String)`, e uma `String s` que retorna o reverso de `s` concatenado com aplicação da função `f` em `s`.

4.7) Crie um tipo `Dia` contendo os dias da semana. Faça uma função que receba uma lista de `Dias` e filtre as `Terças`.

4.8) Implemente o tipo `Dinheiro` que contenha os campos `valor` e `correncia` (`Real` ou `Dolar`), e uma função que converta todos os "dinheiros" de uma lista para dólar (e outra para real). Com isso, implemente funções para:

- Filtrar todos os `Dolares` de uma lista de `Dinheiro`.
- Somar todos os `Dolares` de uma lista.
- Contar a quantidade de `Dolares` de uma lista.

4.9) Usando a função `foldl`, crie *lambdas* para:

- Contar números negativos de uma lista de `Int`.
- Contar letras 'P' de uma `String`.
- Para contar `Sabados` em uma lista de um `[DiaSemana]`.
- Para, a partir de uma lista de `[DiaSemana]`, retornar a soma dos dias. Exemplo: `[Segunda, Segunda, Quarta]` deve retornar `5`. Use uma função auxiliar para converter `DiaSemana` para `Int`.

4.10) Reescreva os exercícios anteriores usando: `.`, `$` e `|>`.

4.11) Implemente a função `fatorial` usando `foldl` (ou `foldr`) em vez de recursão explícita, a partir da lista `[1..n]`.

4.12) Usando *currying*, defina a função `somar10 :: Int -> Int` a partir da função `(+)`, sem escrever *lambdas* nem o parâmetro. Em seguida, escreva `aplicarDuasVezes :: (a -> a) -> a -> a`, uma função de alta ordem que aplica uma função duas vezes ao seu argumento. Teste `aplicarDuasVezes somar10 5`.

4.9 CONCLUSÃO

Este capítulo foi fundamental para entendermos como escrever um programa usando o paradigma funcional. Nele, foram mostradas as armas para escrever código da maneira que lhe for mais conveniente. Também foi visto que podemos ter estruturas condicionais e de *loop* usando ***guards*** e ***recursão***.

Aprendemos também outra alternativa aos ***list comprehensions*** do capítulo 2 usando os famosos `map`, `filter` e `foldl`. Foi mostrado ao leitor também como a linguagem trata as funções no estudo dos conceitos de **funções de alta ordem** e ***currying***.

Pode-se dizer que, até aqui, se você compreendeu os conceitos, rodou os miniprojetos e fez os exercícios, você possui um conhecimento básico da linguagem. Você sabe ver um tipo, identificar tipos de funções, escrever funções e entender o funcionamento básico do paradigma. Os próximos capítulos lhe deixarão numa posição de nível intermediário na linguagem. Aperte os cintos!

POLIMORFISMO PARAMÉTRICO

5.1 TIPOS COM PARÂMETROS

Os tipos com parâmetros (ou em inglês, *type parameters*) são o equivalente aos *generics* do Java aqui no Haskell. É uma forma de implementar o conceito de polimorfismo paramétrico, ou seja, chamaremos de contêiner ou caixa um tipo que possui um *type parameter*, por causa de sua semelhança com uma caixa de tipos. Ele poderá guardar um ou mais elementos de vários tipos, sem a necessidade de especificar qual.

Um exemplo real são as listas, corriqueiras no dia a dia do(a) programador(a). Elas podem ser enxergadas como caixas contendo vários elementos de um mesmo tipo. Graças a este conceito, podemos ter no mesmo programa listas de inteiros `[Int]`, de booleanos `[Bool]`, caracteres `String` ou `[Char]` e afins. A ideia é que possamos operar listas (ou qualquer contêiner) sem a necessidade de olhar "o que há dentro".

Ou seja, a partir do momento em que um tipo vai para dentro de um contêiner, estaremos preocupados com o contêiner em si, e não com o que foi para dentro. Podemos também transformar o

que está dentro do contêiner, mas mesmo assim, não importa quem ou o que está ali. Vamos usar um exemplo mais simples ainda do que uma lista, de modo a fazer você se ambientar com o conceito.

Uma `Coisa` representará algo que podemos guardar nenhum, um ou dois elementos de um mesmo tipo:

```
data Coisa a = UmaCoisa a | DuasCoisas a a | ZeroCoisa
```

O tipo anterior possui um *type parameter* `a` que significa que qualquer tipo poderá ser passado ao contêiner `Coisa`. O *value constructor* `DuasCoisas` carrega dois campos do tipo `a`, enquanto o *value constructor* `UmaCoisa` carrega um, e o `ZeroCoisa` é apenas um *value constructor* sem campos.

```
:t (DuasCoisas "OLA" "Mundo")
(DuasCoisas "OLA" "Mundo") :: Coisa String

:t (UmaCoisa True)
(UmaCoisa True) :: Coisa Bool
```

Observe que poderemos ter contêineres de vários tipos ao mesmo tempo, por exemplo, `Coisa String`, `Coisa Bool`, `Coisa Int`, `Coisa Pessoa` etc., assim como temos com as listas. Aqui o que importa é o contêiner `Coisa` e não o que há dentro.

Ao trabalharmos com contêineres, o comando `:kind` do GHCi vai indicar quantos *type parameters* existem em seu tipo. Quanto mais existirem, mais complexo será trabalhar com ele.

```
Prelude> :kind Int
Int :: *
```

```
Prelude> :kind Coisa
Coisa :: * -> *
```

```
Prelude> :kind []  
[] :: * -> *
```

O `kind` enxerga os tipos como funções de tipos. Como `Int` não possui parâmetro de tipo, seu `kind` é `*` (podemos pensar como ***kind 1***, como abuso de linguagem) ao passo que `Coisa` possui um parâmetro de tipo, logo seu `kind` é `* -> *` (ou ***kind 2***, abusando mais um pouco da linguagem). Divagando um pouco, se declarássemos o seguinte tipo:

```
data Foo a b = Foo a b
```

E também inspecionássemos seu `kind`, obteríamos `* -> * -> *`, pois `Foo` possui dois parâmetros de tipo `a` e `b`. Fazendo um comparativo com o Java, a classe `HashMap` (estrutura similar às listas que possuem índices de qualquer tipo) é um exemplo de um tipo de `kind * -> * -> *`.

Em Haskell, há vários tipos de `kinds`, porém o foco maior deste livro será em tipos de `kind *` e `* -> *`. Isto é, nossas caixas poderão guardar elementos de mesmo tipo apenas (e não de tipos diferentes, como `Foo`).

Se tentarmos inspecionar o tipo de `DuasCoisas True 'A'`, obteremos erro de compilação, pois `DuasCoisas` possui dois campos genéricos de mesmo tipo. Em contrapartida, veja:

```
Prelude> :t (Foo True 'A')  
(Foo True 'A') :: Foo Bool Char
```

Esse código é compilado sem erros, por ser um contêiner de `kind * -> * -> *`.

Podemos usar o conceito de polimorfismo paramétrico para criar tipos recursivos (tipos que possuem algum campo contendo ele próprio) como listas ligadas e árvores. Uma árvore binária pode ser escrita usando tipos polimórficos, por exemplo:

```
data Arvore a = Nulo |  
              Leaf a |  
              Branch a (Arvore a) (Arvore a) deriving Show
```

O tipo `Arvore` possui três *value constructors*: `Nulo`, que não possui nenhum campo; `Leaf a`, que possui um campo de tipo `a`; e `Branch a (Arvore a) (Arvore a)`, que possui um campo `a`, um campo para representar o nó do filho esquerdo de tipo `Arvore a` (que pode ser novamente `Nulo`, `Leaf` ou `Branch`) e um campo para representar o nó do filho direito de tipo `Arvore a`.

Os campos de tipo `a` representam o elemento a ser guardado na árvore, enquanto os campos de tipo `Arvore a` fazem a continuação da estrutura, tanto para a esquerda quanto para a direita.

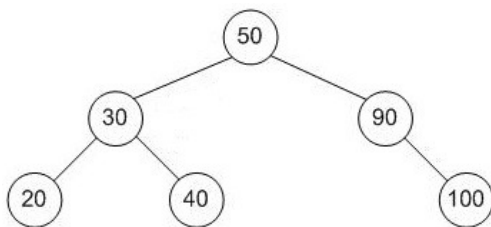


Figura 5.1: Árvore binária

A árvore anterior pode ser representada pela expressão:

```
Branch 50 (Branch 30 (Leaf 20) (Leaf 40)) (Branch 90 Nulo (Leaf 100))
```

O percurso em ordem, por exemplo, pode ser escrito com a ajuda da *recursão* e do *pattern matching*:

```
emOrdem :: Arvore a -> [a]
emOrdem (Branch x l r) = emOrdem l ++ [x] ++ emOrdem r
emOrdem (Leaf x) = [x]
emOrdem Nulo = []
```

Observe que a condição de parada da função anterior se baseia no fato de que o *pattern matching*, ao encontrar um `Leaf` ou `Nulo`, interrompe o processo recursivo dos `Branch` es. A função `emOrdem` é polimórfica e, como é possível notar, não utiliza o tipo `a` associado ao contêiner `Arvore`. Isto é, funções polimórficas não podem operar sobre as *type variables*.

No caso da função `emOrdem`, sua entrada é do tipo `Arvore a`, logo, essa função valerá para qualquer tipo `a`, e seu retorno `[a]` indica que será uma lista desse mesmo tipo. A conclusão maior que podemos tirar é que `emOrdem` troca o contêiner sem mexer nos elementos.

Pense em um presente que você coloca em uma caixa azul, porém você não gostou da cor da caixa e resolve trocar para uma caixa vermelha. Você apenas trocou a caixa — essa é a ideia com funções deste tipo.

5.2 RESTRIÇÃO DE TIPOS EM FUNÇÕES

Às vezes, é necessário operar com o elemento que possui um tipo genérico `a`, por exemplo. Em uma função, é necessário exibir um elemento dentro do contêiner. E se criássemos esta função da seguinte forma:

```
foo :: a -> String
```

```
foo x = "0 valor de tipo a é: " ++ show x
```

Teríamos um problema, pois a entrada aceita um tipo qualquer e este pode não ter uma instância de `Show` em sua declaração. Em outras palavras, o tipo precisa possuir uma instância de `Show`, normalmente obtida pela expressão *deriving Show*.

Para resolver isso, temos de avisar que não pode ser qualquer `a`, criando uma restrição que avisa ao compilador que tipos serão barrados fora desse contexto em tempo de compilação. Veja um exemplo:

```
foo :: Show a => a -> String
foo x = "0 valor de tipo a é: " ++ show x
```

Com esta função, conseguimos ter uma indicação do que `a` pode ser, conforme indicado pela restrição `Show a`. Ou seja, tipos que possuem a expressão `deriving Show` serão aceitos pela função `foo`, em tempo de compilação, ao passo que funções sem esta palavra não serão aceitas. Há um nome específico para `Show`, *typeclasses*, que será abordado a seguir.

5.3 CLASSES DE TIPOS

Um *typeclass* (classe de tipos) é uma estrutura do Haskell que habilita um operador (ou mais) ou uma função (ou mais) a ser usada de forma diferente, dependendo de um *type parameter*. Para cada tipo, uma instância deverá ser definida e, com ela, a definição do operador ou da função que o *typeclass* provê.

Fazendo uma analogia com o paradigma orientado a objetos, um *typeclass* se assemelha a uma interface, e o operador/função inerente a ele se assemelha a um método abstrato.

Se usarmos o tipo:

```
data Coisa a = Nada | UmaCoisa a | DuasCoisas a a
```

Não poderíamos mostrá-lo na tela nem comparar (usando a função `==`). Para mostrar qualquer tipo na tela, seria necessário o uso do *typeclass* `Show` e, para comparar, usaríamos o `Eq`. Reescrevendo:

```
data Coisa a = Nada | UmaCoisa a | DuasCoisas a a deriving (Show, Eq)
```

Temos agora que o tipo `Coisa a` pode ser mostrado na tela e é comparável. O Haskell provê para o tipo `Coisa a` a seguinte regra de igualdade: dois valores do tipo `Coisa a` são iguais se, e somente se, seus *value constructors* são iguais e os campos também.

```
DuasCoisas 5 5 == DuasCoisas 5 5
True
```

```
DuasCoisas 7 3 == DuasCoisas 3 7
False
```

Classe Eq

Esta regra de igualdade para o tipo em questão é obtida gratuitamente, apenas usando `deriving Eq` na definição. Se a regra de igualdade para este tipo for criada pelo leitor, a instância de `Eq` para o tipo `Coisa a` se faz necessária.

```
instance Eq a => Eq (Coisa a) where
  (DuasCoisas x1 y1) == (DuasCoisas x2 y2) = x1 == y2
  (UmaCoisa x) == (UmaCoisa y) = x == y
  Nada == Nada = True
  _ == _ = False
```

Note que o tipo `a` dentro da estrutura `Coisa` deve ser comparável (instância de `Eq`) como mostra o trecho `Eq a => Eq (Coisa a)`. Não pode ser qualquer `a`, sendo assim, faz-se necessária a restrição `Eq a`.

Se o tipo `a` não for uma instância de `Eq`, não há como comparar os elementos de dentro de `Coisa`, logo, o tipo `Coisa a` não pode ser comparável. Logo, é preciso apagar o `Eq` da palavra `deriving`. Veja:

```
data Coisa a = Nada | UmaCoisa a | DuasCoisas a a deriving Show
```

Ainda nesse exemplo, mudou-se a regra de igualdade apenas para o *value constructor* `DuasCoisas` e, com isso, temos:

```
DuasCoisas 7 3 == DuasCoisas 3 7
True
```

A operação binária diferente (`/=`) vem de graça com a negação da igualdade (`==`).

Podemos também escrever uma instância de `Show` para `Coisa a`, diferente da que foi provida pelo Haskell. Veja o exemplo:

```
Prelude> DuasCoisas 7 9
DuasCoisas 7 9
```

```
Prelude> Nada
Nada
```

```
Prelude> UmaCoisa True
UmaCoisa True
```

O valor, por exemplo, `DuasCoisas 7 9` apareceu porque `Coisa` possui a palavra `deriving Show`, que cria a instância de `Show` para `Coisa a` automaticamente.

Classe Show

A classe `Show` é usada para a saída de qualquer dado. Nos *frameworks web* em Haskell, usamos o `Show` para injetar saídas das funções em um template HTML. Há uma instância "padrão" de `Show` que fornece, na tela, `DuasCoisas 7 9`, por exemplo.

Podemos mudar isso removendo o `Show` do `deriving`:

```
data Coisa a = Nada | UmaCoisa a | DuasCoisas a a
```

E também criando a instância:

```
instance Show a => Show (Coisa a) where
  show Nada = "Nadinha..."
  show (UmaCoisa x) = "Coisa com o elemento " ++ show x
  show (DuasCoisas x y) = "Coisa com os elementos " ++ (show x)
  ++ " e " ++ (show y)
```

Perceba que a classe `Show` exige a definição da função `show` de tipo `a -> String`. Testando a instância anterior:

```
Prelude> DuasCoisas 7 9
Coisa com os elementos 7 e 9
```

```
Prelude> Nada
Nadinha...
```

```
Prelude> UmaCoisa True
Coisa com o elemento True
```

Desta forma, a instância padrão do Haskell foi modificada.

Classe Read

Esta *typeclass* é outro de extrema importância, pois converte uma `String` em um tipo `a` definido em sua instância. Ele ajuda muito na hora de fazer conversão de dados. Por exemplo, podemos

converter algo digitado pela entrada padrão, e transformar isto em qualquer tipo (é mais usual com números).

A criação de uma instância para este tipo pode ser bem complicada. Mostraremos o seu uso acompanhado do `deriving` apenas (instância padrão do Haskell). Vamos usar nosso tipo como teste:

```
module Cap5 where

data Coisa a = Nada | UmaCoisa a | DuasCoisas a a deriving Read

lerCoisa :: Coisa Int -> Coisa Int
lerCoisa Nada = UmaCoisa 0
lerCoisa (UmaCoisa x) = UmaCoisa (x+1)
lerCoisa (DuasCoisas x y) = DuasCoisas (2*x) (y-3)
```

E no GHCi teremos:

```
*Cap5> lerCoisa (read "Nada")
UmaCoisa 0
*Cap5> lerCoisa (read "UmaCoisa 8")
UmaCoisa 9
*Cap5> lerCoisa (read "DuasCoisas 1 2")
DuasCoisas 2 (-1)
```

Perceba que a entrada de `lerCoisa` foi obtida a partir de uma `String` após a conversão feita por `read`. O problema é que, se a `String` não estiver formatada corretamente, obteremos um erro de execução que não é bem-vindo por aqui.

```
*Cap5> lerCoisa (read "DuasCoisas 1 2 3")
*** Exception: Prelude.read: no parse.
```

A *exception* no `parse` indica que a conversão da `String` `"DuasCoisas 1 2 3"` em um tipo `Coisa Int` falhou.

5.4 OUTRAS CLASSES

Outras classes importantes são: `Num` , `Fractional` , `Real` , `Integral` , `Enum` , `Ord` e `Bounded` .

Classe `Num`

A classe `Num` é definida por:

```
class Num a where
  (+), (-), (*)      :: a -> a -> a
  abs                :: a -> a
  signum             :: a -> a
  fromInteger        :: Integer -> a
```

Ela provê uma interface para operações aritméticas básicas, como `+` (mais), `-` (menos) e `*` (vezes). Além disso, ela possui as funções:

- `abs` : para transformar valores negativos em positivos;
- `signum` : indica, em seu retorno, se um número é positivo, negativo ou zero;
- `fromInteger` : para converter um número inteiro (`Integer` representa números inteiros grandes) para o tipo `a` indicado na instância.

No caso de `Int` , por exemplo, temos que `signum` retorna `1` se o número for positivo, `0` caso seja o zero, e `-1` para número negativo.

Classe `Fractional`

A classe `Fractional` é definida por:

```
class (Num a) => Fractional a where
```

```
(/)      :: a -> a -> a
fromRational :: Rational -> a
```

Ela provê um jeito de efetuar a divisão para seu tipo, a partir da função `(/)`. A função `fromRational` converte uma fração, $3/5$ por exemplo, em seu tipo `a`.

Perceba que todo tipo que instancia `Fractional` deve ser também uma instância de `Num`, como indicado na restrição `Num a da linha (Num a) => Fractional a`.

Classe Real

A classe `Real` de tipo `(Num a, Ord a) => Real a` provê a função `toRational :: a -> Rational`, que converte seu tipo `a` em uma fração. Todo tipo instância de `Real` também deve, obrigatoriamente, ser instância de `Num` e `Ord`.

Classe Integral

A classe `Integral` determina que um tipo `a` é capaz de realizar divisões inteiras com quociente. As funções a serem implementadas são:

- `quotRem :: a -> a -> (a, a)`, que retorna o quociente na primeira coordenada da tupla e o resto na segunda.
- `toInteger :: a -> Integer`, que converte seu tipo `a` em um `Integer`.

Todo tipo instância da classe `Integral` deve ser instância também de `Real` e `Enum`, conforme a declaração `class (Real a, Enum a) => Integral a`.

Classe Enum

A classe `Enum` possui uma interface para enumerações de tipo, por exemplo:

```
data Dia = Domingo | Segunda | Terca | Quarta | Quinta | Sexta | Sabado
```

Isso pode ser associado a um inteiro, por exemplo, `Domingo = 1`, ..., `Sabado = 7`. Esta classe possui a definição:

```
class Enum a where
  toEnum :: Int -> a
  fromEnum :: a -> Int
```

Ela converte seu tipo `a` nesse inteiro representando uma contagem. Todo tipo instância de `Enum` ganha de graça as funções `succ` e `pred`, que indicam o sucessor e o predecessor de um valor. Se colocarmos `deriving Enum` no tipo `Dia`, teremos a enumeração anterior como:

```
Prelude> succ Domingo
Segunda
```

```
Prelude> pred Segunda
Domingo
```

O resultado foi como esperávamos. Os *typeclasses* podem ter funções já definidas previamente como `succ` e `pred` que valem para qualquer `a`. Note que `toEnum` e `fromEnum` não são funções previamente definidas, e elas esperam uma instância para ter sua definição, algo que lembra interfaces no Java.

Classe Ord

A classe `Ord` indica que seu tipo possui ordenação por meio do operador `<=`. A classe `Ord` é definida como:

```
class Eq a => Ord a where
    (<=) :: a -> a -> Bool
```

Note que basta definir como funciona `<=` , que `>` , `<` , `>=` , `max` e `min` são ganhos de graça. Note também que todo tipo ordenável (instância de `Ord`) deve ser instância de `Eq` . A classe `Ord` facilita, por exemplo, o trabalho com listas (`Strings` especificamente):

```
Prelude> "HASKELL" > "OLA"
True
```

Isso porque facilita a escrita ao ocultar a função `length` , provendo uma escrita mais limpa e eficaz.

Classe Bounded

Finalmente, vemos a classe `Bounded` :

```
class Bounded a where
    minBound, maxBound :: a
```

As funções que não possuem entrada `minBound` e `maxBound` representam os limites mínimo e máximo do seu tipo. Para o tipo `Dia` , poderíamos criar a instância:

```
instance Bounded Dia where
    minBound = Domingo
    maxBound = Sabado
```

Note que poderíamos usar o `Bounded` , assim como todas essas classes, no `deriving` . A classe `Bounded` cria listas com todos os valores de tipo, por meio da expressão `[minBound .. maxBound]` , de forma geral e sem precisar saber qual tipo está sendo trabalhado.

O intuito de usar os *typeclasses* é sempre o de criar uma função que valha para o tipo mais genérico possível e para o maior número de tipos possíveis. Obtém-se assim um nível de abstração maior em seu código, evitando uma quantidade enorme de linhas de código que fazem a mesma coisa.

Criando uma classe

Quando é necessário, por exemplo, ler diferentes tipos de arquivo cujo conteúdo deve ser representado por um tipo em Haskell, ou criar uma interface para validação de vários tipos de dados, precisamos criar uma classe.

```
class SimNao a where
    simnao :: a -> Bool
```

Esse *typeclass* provê uma função `simnao` que deve ser definida para todo `a` do qual desejarmos criar uma instância. Ela representa uma forma de validar dados de acordo com o tipo. Pode-se criar instâncias do *typeclass* `SimNao a` para qualquer tipo, como faremos para os seguintes:

```
instance SimNao Int where
    simnao x
        | x < 18 = False
        | otherwise = True
```

A instância de `SimNao` para `Int` pode conter validação de idades.

```
instance SimNao [a] where
    simnao [] = False
    simnao _ = True
```

Para listas (`String` , por exemplo), essa instância pode indicar que, ao digitar um nome, o campo foi deixado vazio.

```
Prelude> simnao (1::Int)
False
Prelude> simnao "João"
True
```

Note que `(1::Int)` foi usado para indicar que queremos um inteiro e não um `Num a`.

5.5 SEMIGRUPOS

Antes de chegarmos aos monoides, precisamos conhecer um irmão mais simples: o **semigrupo** (em inglês, *semigroup*). Um semigrupo é um conjunto (em Haskell, um tipo) `m` equipado com uma função infix `<>` de tipo `m -> m -> m` que satisfaz uma única lei:

- **Associatividade:** $(a \lt b) \lt c = a \lt (b \lt c)$ para quaisquer valores `a`, `b` e `c` de `m`.

Ou seja, um semigrupo é apenas um tipo que sabe **juntar dois de seus valores em um só**, de forma associativa (não importa de onde você comece a operação, pela esquerda ou pela direita, o resultado é o mesmo). A operação binária `<>` era, antigamente, chamada de *mappend*. Em Haskell, isso é expresso pela classe `Semigroup`, definida no `Prelude` (não é preciso importar nada):

```
class Semigroup a where
  (<>) :: a -> a -> a
```

Para criar uma instância, basta dizer como dois valores se combinam. Veja um exemplo com listas, em que `<>` é a concatenação `(++)`:

```
Prelude> [1,2,4] <> [0,9] <> [8,5]
[1,2,4,0,9,8,5]
```

```
Prelude> "Ola" <> " " <> "Mundo"  
"Ola Mundo"
```

Note que a concatenação é claramente associativa: $[1, 2, 4] \lt \gt ([0, 9] \lt \gt [8, 5])$ dá o mesmo resultado que $([1, 2, 4] \lt \gt [0, 9]) \lt \gt [8, 5]$, ou seja, $[1, 2, 4, 0, 9, 8, 5]$.

A associatividade é uma propriedade muito importante na área da programação paralela, daí a importância de se usar tal estrutura matemática. Pense em três processos rodando ao mesmo tempo: não há como saber qual deles termina primeiro. Se eles estiverem envolvidos em uma operação binária associativa, não há por que se preocupar com qual deles termina primeiro, pois o resultado será o mesmo. Se não houver associatividade, os resultados serão diferentes, causando muito prejuízo.

Existem tipos que são semigrupos, mas que **não** possuem um elemento neutro natural. Um bom exemplo é o tipo `NonEmpty` (listas que nunca são vazias): podemos concatenar duas listas não vazias, mas não existe uma "lista não vazia vazia" para servir de neutro. Esses tipos param no semigrupo e não sobem para monoide. Quando, além de combinar valores, existir também um elemento neutro, teremos um monoide, como veremos a seguir.

5.6 MONOIDES

Um **monoide** é um semigrupo que, além da operação $\lt \gt$, possui um **elemento neutro**. Em outras palavras, um monoide é um tipo `m` que satisfaz as seguintes leis:

- **Associatividade:** $(a \lt \gt b) \lt \gt c = a \lt \gt (b \lt \gt c)$ para quaisquer valores `a`, `b`, `c` de `m`.

- **Elemento neutro:** existe um elemento e em m , tal que $a <> e = a$ e $e <> a = a$ para todo valor a de m .

No Haskell, o *elemento neutro* é chamado de `mempty`. A classe `Monoid` vive no Prelude e, desde o GHC 8.4, **tem a `Semigroup` como superclasse**. Isso quer dizer que, para um tipo ser monóide, ele obrigatoriamente já deve ser semigrupo. Sua definição (simplificada) é:

```
class Semigroup a => Monoid a where
    mempty :: a
```

TOME CUIDADO!

Em livros e códigos antigos, você vai encontrar a operação binária sendo definida dentro do `> instance Monoid` com o nome `mappend`. Hoje isso mudou: a operação `<>` pertence à `Semigroup`, `>` e `mappend` virou apenas um *apelido* de `<>` (seu valor padrão é `mappend = (<>)`). Em código `>` novo, defina sempre `<>` na instância de `Semigroup` e apenas o `mempty` na instância de `Monoid`.

Essa classe deve ser usada toda vez que tivermos uma operação binária que recebe dois valores de mesmo tipo, retornando outro desse mesmo tipo, e quando existir um valor neutro para essa operação. Os monóides são interfaces para operações que juntam dois valores de um mesmo tipo em um, algo mais abstrato que somas e multiplicações.

Ao pensar em monoides, devemos identificar um elemento que não tenha efeito sobre essa operação (o `mempty`) e verificar se ela é associativa.

Se considerarmos o tipo `Int`, vemos que duas operações aritméticas são fortes candidatas a formar um monoide: a soma e a multiplicação. Observe que o elemento neutro da soma é o número `0`, pois $x + 0 = x$ para qualquer número x ; e o da multiplicação é o número `1`, pois $y * 1 = y$ para qualquer número y . Como a soma e a multiplicação são associativas, por exemplo, $(5+3)+2 = 5+(3+2) = 10$ e $(5*3)*2 = 5*(3*2) = 30$, ambas formam monoides.

Porém, em Haskell, só conseguimos ter **uma** instância de cada classe para cada tipo por vez. Se déssemos uma instância de `Monoid` direto para `Int`, teríamos de escolher entre a soma e a multiplicação, causando uma ambiguidade. Será que devemos escolher uma ou outra? Por isso o `Int` **não** tem instância de `Monoid`. Para resolver esse problema, há dois `newtype`s de kind `* -> *` no módulo `Data.Monoid` que embrulham um número e escolhem a operação para nós. São eles `Sum` e `Product`, declarados (de forma simplificada) como:

```
newtype Sum a      = Sum      { getSum      :: a }
newtype Product a = Product { getProduct  :: a }
```

Eles possuem as instâncias:

```
instance Num a => Semigroup (Sum a) where
    Sum x <> Sum y = Sum (x + y)

instance Num a => Monoid (Sum a) where
    mempty = Sum 0

instance Num a => Semigroup (Product a) where
```

```
Product x <> Product y = Product (x * y)
```

```
instance Num a => Monoid (Product a) where  
  empty = Product 1
```

Com isso, é possível somar ou multiplicar sem usar os sinais `+` e `*` e, em vez disso, usar `<>`, que realizará a operação de acordo com o `newtype` envolvido. Antes de dar o exemplo, é necessário carregar o módulo `Data.Monoid`, ou no programa usando `import Data.Monoid`, ou no GHCi com o comando `:m módulo`, que carrega um módulo externo pronto para seu uso. A função de projeção (`getSum` ou `getProduct`) serve para retirar o número de dentro do embrulho.

```
Prelude> :m Data.Monoid
```

```
Prelude Data.Monoid> Sum 5 <> Sum 6  
Sum {getSum = 11}
```

```
Prelude Data.Monoid> getSum (Sum 5 <> Sum 6)  
11
```

```
Prelude Data.Monoid> Product 5 <> Product 6  
Product {getProduct = 30}
```

Esse é o grande benefício de usar monoides. Podemos calcular ações representadas por uma operação binária usando uma única função, o `<>`.

TOME CUIDADO!

O compilador não verificará se seu operador tem realmente um `mempty` verdadeiro, ou se ele é associativo. Isso fica a cargo do programador. Porém, sugiro **fortemente** que siga essas regras, pois facilita na hora de fazer os chamados testes de propriedades.

Um teste de propriedade verifica sua função com base em alguma regra matemática, como, por exemplo, a associatividade. Se essa propriedade for satisfeita, não será necessário testar inúmeros casos específicos, reduzindo o custo dessa atividade.

Você pode estar se perguntando sobre a subtração e a divisão, porém elas não formam nem semigrupos, pois não são operações binárias associativas. A associatividade falha nesses casos: $3 - (2 - 1) = 2$ ao passo que $(3 - 2) - 1 = 0$.

A divisão possui um exemplo contrário análogo, $(10/5)/2 = 1$ enquanto $10/(5/2) = 4$. Não transforme em instâncias de semigrupo ou monoide operações que não são associativas, ou em que há um falso `mempty`, pois quem sairá perdendo será você.

Instâncias de monoide

Uma lista possui uma instância de monoide já implementada:

```
instance Semigroup [a] where
  (<>) = (++)
```

```
instance Monoid [a] where
    mempty = []
```

Seu `<>` é a concatenação e, com isso, seu `mempty` é a lista vazia, pois ao concatenarmos qualquer lista à lista vazia, não teremos efeito algum. Essa operação é claramente associativa, então, não importa por onde começamos a concatenar, o resultado será o mesmo. Por exemplo: `[1,2,4] <> ([0,9] <> [8,5]) = ([1,2,4] <> [0,9]) <> [8,5] = [1,2,4,0,9,8,5]` .

Uma das principais vantagens dessa instância é que `String` é apenas um sinônimo para `[Char]` . Acredite: isso facilita muito quando os tipos `Text` e `ByteString` são usados. Tanto um quanto o outro são usados em alguns pacotes de persistência de dados em um banco de dados e *frameworks web*, e são mais rápidos que a `String` convencional. O problema é que `String` , `Text` e `ByteString` possuem a função `concat` em comum, podendo dar ambiguidade em qual chamar. Considere o trecho:

```
import qualified Data.Text as T
import qualified Data.ByteString as BS
```

Então, usaríamos `concat` , `T.concat` e `BS.concat` para concatenarmos dois desses tipos: `String` , `Text` e `ByteString` , respectivamente. Com o uso das instâncias de monoide comum aos três, basta usar `<>` e estamos feitos, algo bastante útil quando estamos em um ambiente de produtividade.

Outro benefício do uso de uma instância de monoide é a função `mconcat` , que junta uma lista inteira de valores em um só e pode ser pensada como:

```
mconcat :: (Monoid a) => [a] -> a
mconcat xs = foldr (<>) mempty xs
```

A função recebe uma lista de valores do tipo `a` (que deve ser instância de `Monoid`, caso contrário, ocorrerá erro de compilação) e retorna um valor de tipo `a` que representa a sucessiva aplicação de `<>`, acumulando em um único valor e tendo como valor inicial da acumulação `mempty`.

Pense na soma dos elementos de um vetor: começamos do zero e vamos somando até acabar os elementos. Veja um exemplo:

```
Prelude Data.Monoid> mconcat [Sum 7, Sum 3, Sum 10]
Sum {getSum = 20}
```

```
Prelude Data.Monoid> mconcat ["Ola Mundo", " Haskell", "!!!"]
"Ola Mundo Haskell!!!"
```

5.7 MINIPROJETO: TRABALHANDO COM PARSERS

No *capítulo 3*, vimos que há uma função chamada `verFolha`, que transforma um valor do tipo `funcionário` em uma `String` no formato JSON. A função em questão é:

```
verFolha :: Pessoa -> String
verFolha p = "{nome: \"" ++ (nome p) ++
              "\", cargo: \"" ++ show (cargo p) ++
              "\", salario: " ++ show (verSalario p) ++ "}"
```

Embora não esteja errada, podemos generalizar sua implementação, usando um *typeclass*. Isto é, fazer uma rotina, de modo que funcione para todo tipo que necessitarmos transformar em JSON. Caso quisermos ter um tipo para cada entidade da empresa, e for necessário uma conversão para JSON, basta criarmos uma instância deste *typeclass* para cada um desses tipos.

O intuito aqui é mostrar como usar os *typeclasses* e também criar nossas próprias classes, além de saber quando usar esse conceito. Será criado um tipo chamado `Projeto` que carregará as informações de orçamento do projeto, nome e uma lista com índice das pessoas envolvidas (o índice será implementado no capítulo 7).

Como será necessário ver o total de pessoas em todos os projetos e o orçamento de todos eles, tornaremos `Projeto` instância de `Monoid`.

```
module Projeto where

-- funcionalidades implementadas previamente

data Projeto = Projeto {nomeProjeto :: String,
                        budget :: Double,
                        envolvidos :: [Int]} deriving Show

class ToJSON a where
    toJSON :: a -> String

instance ToJSON Pessoa where
    toJSON p = "{nome: \" " ++ (nome p) ++
                "\", cargo: \" " ++ show (cargo p) ++
                "\", salario: \" " ++ show (verSalario p) ++ "}"

instance ToJSON Projeto where
    toJSON p = "{nome: \" " ++ (nomeProjeto p) ++
                "\", orçamento: \" " ++ show (budget p) ++
                "\", envolvidos: \" " ++ show (envolvidos p) ++ "}"

instance Semigroup Projeto where
    (Projeto nome1 budget1 env1) <> (Projeto nome2 budget2 env2)
    =
        Projeto (nome1 ++ " " ++ nome2) (budget1 + budget2) (env1 ++ env2)

instance Monoid Projeto where
    mempty = Projeto "" 0 []
```

Perceba que agora não será mais necessária a função `verFolha`, e a classe `ToJSON` consegue trabalhar com cada tipo que quisermos transformar em um JSON. Com a instância de `Monoid` para `Projeto`, conseguimos juntar dois projetos em um só, de modo a concatenar seus nomes, somar seus orçamentos e juntar as duas listas de pessoas envolvidas.

Outro benefício de ter a instância de `Monoid` para `Projeto` é que a função `mconcat` juntará informações de vários projetos ao mesmo tempo.

5.8 EXERCÍCIOS

5.1) Crie o tipo `TipoProduto` que possui os *value constructors* `Escritorio`, `Informatica`, `Livro`, `Filme` e `Total`. O tipo `Produto` possui um *value constructor* — de mesmo nome — e os campos `valor` (`Double`), `tp` (`TipoProduto`) e um *value constructor* `Nada`, que representa a ausência de um `Produto`.

Deseja-se calcular o valor total de uma compra, de modo a não ter nenhuma conversão para inteiro e de forma combinável. Crie uma instância de `monoid` para `Produto`, de modo que o retorno sempre tenha `Total` no campo `tp` e a soma dos dois produtos em `valor`. Explique como seria o exercício sem o uso de monoides. Qual(is) seria(m) a(s) diferença(s)?

5.2) Crie uma função `totalGeral` que recebe uma lista de produtos e retorna o preço total deles usando o `monoid` anterior.

5.3) A função `min` no Haskell retorna o menor entre dois números, por exemplo, `min 4 5 = 4`.

- Crie um tipo `Min` com um campo inteiro, que seja instância de `Ord`, `Eq` e `Show` (*deriving*).
- Crie uma instância de `Semigroup` e de `Monoid` para `Min` (`maxBound` representa o maior inteiro existente no Haskell).
- Quanto vale a expressão `Min (-32) <> Min (-34) <> Min (-33)` ?
- Explique sua escolha para o `mempty`.

5.4) Crie uma função `minAll` que recebe um `[Min]` e retorna um `Min` contendo o menor valor.

5.5) A função `max` no Haskell retorna o maior entre dois números, por exemplo: `max 4 5 = 5`.

- Crie um tipo `Max` com um campo inteiro que seja instância de `Ord`, `Eq` e `Show` (*deriving*).
- Crie uma instância de `Semigroup` e de `Monoid` para `Max` (`minBound` representa o menor inteiro existente no Haskell).
- Quanto vale a expressão `Max 10 <> Max 13 <> Max 5` ?
- Explique sua escolha para o `mempty`.
- Crie uma função `maxAll` que recebe um `[Max]` e retorna um `Max` contendo o maior valor.

5.6) Crie o tipo `Paridade` com os *value constructors* `Par` e `Impar`. Crie o *typeclass* `ParImpar` que contém a função `decide` `:: a -> Paridade` e possui as instâncias:

- Para `Int` : noção de `Par/Impar` de `Int`.
- Para `[a]` : uma lista de elementos qualquer é `Par` se o número de elementos o for.

- `Bool` : `False` como Par , `True` como Impar .

5.7) Usando a estrutura de árvore, monte uma função `mapa` , que jogue uma função passada por parâmetro para todos os elementos de uma árvore. Deixe explícito o tipo desta função.

5.8) Usando o exercício anterior, some 5 a cada elemento de uma árvore de inteiros.

5.9) Usando a estrutura de árvore vista, faça uma função que some todos os elementos de uma árvore de números.

5.10) Implemente os percursos pós-ordem e pré-ordem. Via comentário, faça os "testes de mesa" para os dois percursos da árvore `Branch 15 (Branch 11 (Leaf 6) (Branch 12 (Leaf 10) Nulo)) (Branch 20 Nulo (Branch 22 (Leaf 21) Nulo))` .

5.11) Faça uma função para inserir um elemento em uma árvore de busca binária (use a mesma estrutura vista).

5.12) Faça uma função que, a partir de uma lista de elementos de um tipo, insira todos os elementos desta lista na árvore e retorne-a, usando o exercício anterior.

5.13) Uma lista ordenada é uma lista cujos elementos são inseridos de forma ordenada (crescente). Usando o tipo `data ListaOrd a = a :: (ListaOrd a) | Nulo deriving Show` , crie as funções:

- `inserir :: (Ord a) => a -> ListaOrd a -> ListaOrd a`
- `remove :: (Eq a) => a -> ListaOrd a -> ListaOrd a`

- `tamanho :: ListaOrd a -> Int`

Observação: a função `remover` deve buscar um elemento. Se não achar, a lista deve se manter intacta.

5.14) Mostramos que `Min` e `Max` formam monoides porque os tipos `Int` têm um maior e um menor valor (`maxBound` e `minBound`). Considere agora o tipo das listas **não vazias** `NonEmpty` (do módulo `Data.List.NonEmpty`), cujo `<>` é a concatenação.

- Por que `NonEmpty` é um `Semigroup` , mas **não** é um `Monoid` ? (Dica: pense em qual seria o `mempty` .)
- Dê outro exemplo de um tipo que seja semigrupo, mas não monoide, e justifique.

5.15) Crie o tipo `data Mediana = Mediana {soma :: Double, qtd :: Int}` , que acumula a soma de vários números e a quantidade deles. Crie instâncias de `Semigroup` e `Monoid` para `Mediana` (some as somas e some as quantidades; o `mempty` deve ser o elemento neutro). Em seguida, crie a função `media :: Mediana -> Double` e use `mconcat` para calcular, em uma só passada, a média de uma lista de `Mediana` .

5.9 CONCLUSÃO

Neste capítulo, foram apresentados conceitos mais avançados, como polimorfismo paramétrico, classes de tipos e monoides. Você também viu a estrutura dos números em Haskell a partir de seus *typeclasses*.

Os próximos capítulos que completam este livro dependem completamente dos conceitos apresentados aqui. De todos, este é o que mais apresenta conceitos que o leitor já está habituado, caso tenha estudado algumas linguagens orientadas a objetos (há uma semelhança entre os conceitos).

O próximo capítulo é de caráter opcional ao leitor, pois será uma introdução para explicar de onde vem os conceitos explicados nos capítulos subsequentes. Caso queira uma continuação sem se preocupar com a matemática, pule para o *capítulo 7*.

TEORIA DAS CATEGORIAS

Neste capítulo, será introduzido, de maneira informal, o conceito matemático de categoria, no qual a linguagem Haskell se baseia para nortear alguns conceitos de programação. Ainda que seja uma exposição breve, é importante o(a) leitor(a) ter este embasamento para captar a ideia do que vamos utilizar nos próximos capítulos.

Uma pessoa programadora Haskell não precisa ser um matemático ou um especialista em Teoria das Categorias, porém um entendimento (mesmo que básico) do assunto permitirá uma compreensão mais profunda da linguagem, proporcionando um diferencial poderoso no mercado pelo aumento da capacidade de abstração. O uso da Teoria das Categorias em Engenharia de Software é uma tendência fora do país e provavelmente logo sairá da escuridão aos olhos do mercado.

6.1 CATEGORIAS

Uma categoria C é uma estrutura composta de objetos e morfismos. Um objeto é uma representação abstrata do que se quer estudar (conjuntos, tabelas, palavras, pessoas, animais etc.) e um morfismo é uma relação entre objetos, e os morfismos podem

ser compostos de forma associativa. Existe também o morfismo neutro, chamado identidade. São seus três componentes:

- Coleção de todos os objetos, chamada de $\text{ob}(C)$.
- Coleção de todos os morfismos, chamada de $\text{hom}(C)$: um morfismo relaciona dois objetos A e B . O morfismo $f :: A \rightarrow B$ relaciona um objeto de entrada A com um de saída B (generalização do conceito de função).
- Noção de composição dos morfismos: $(.)$: se $g :: A \rightarrow B$ e $f :: B \rightarrow C$, então $f.g :: A \rightarrow C$.

Em toda categoria, é necessário existir a função identidade. Em outras palavras, para cada objeto X de uma categoria C , existe um morfismo $\text{id}_X :: X \rightarrow X$, tal que, para cada morfismo $f :: A \rightarrow B$, satisfaça:

$$f . \text{id}_A = f = \text{id}_B . f$$

Ou seja, a função identidade não faz absolutamente nada. Ela age como elemento neutro, assim como o 0 na soma e o 1 na multiplicação.

E, finalmente, a composição deve ser associativa. Isto é, se $f :: A \rightarrow B$, $g :: B \rightarrow C$ e $h :: C \rightarrow D$, obtemos:

$$(h . g) . f = h . (g . f)$$

Observe que a notação usada para representar a função composta é em termos da função $(.)$, apresentada no *capítulo 4*. Vale lembrar que $(f.g) x = f(g(x))$, ou seja, primeiro calculamos g tendo como argumento x e, após o retorno, calculamos f .

Um exemplo de categoria

Banco de dados, conforme Spivak (2014), pode ser visto como uma categoria, em que seus objetos seriam as tabelas e seus morfismos, as colunas. Uma declaração de regra de negócio é dada pela composição dos morfismos. Considere, a seguir, as tabelas Funcionário e Departamento.

Tabela de Funcionário

IdFunc	NomeFunc	SobrenomeFunc	Chefe	Dpt
101	João	Silva	103	10
102	José	Santos	102	02
103	Ernesto	Oliveira	103	10

Tabela de Departamento

idDept	NomeDept	Secr
10	Compras	101
02	Vendas	102

O campo `Secr` indica um secretário de departamento. Analisando esta situação sob a ótica de uma categoria, temos:

- `ob( C )` é composto de Funcionário, Departamento (as tabelas) e os tipos de dados `Int` (inteiros) e `String` (palavras).
- `Hom( C )` é composto por: `IdFunc :: Funcionário -> Int`, `NomeFunc :: Funcionário -> String`, `Chefe :: Funcionário -> Funcionário`, `Dept :: Funcionário -> Departamento`, `idDept :: Departamento -> Int`,

```
NomeDept :: Departamento -> String e Secr ::
Departamento -> Funcionário .
```

Foram escolhidos `IdFunc :: Funcionário -> Int` e `idDept :: Departamento -> Int` pelo fato de não relacionarem tabelas. As chaves estrangeiras relacionam tabelas, logo, a autorrelação e as duas chaves estrangeiras são definidas como `Chefe :: Funcionário -> Funcionário`, `Dept :: Funcionário -> Departamento` e `Secr :: Departamento -> Funcionário`. Os outros campos não citados relacionam as tabelas com `String` ou `Int`.

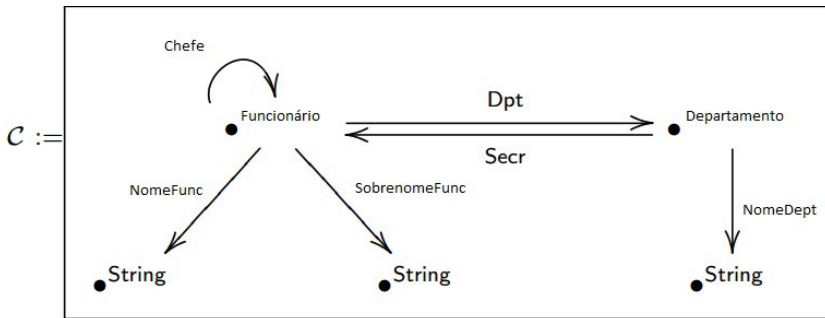


Figura 6.1: Representação de um banco de dados sob a ótica de uma categoria Spivak (2014), traduzido pelo autor

A figura anterior mostra os objetos e os morfismos identificados. Os círculos são os objetos, e as flechas, morfismos. Um exemplo de regra de negócio seria `Dpt . Chefe`, que indica a qual departamento um chefe está associado.

A categoria Hask

A categoria mais importante aqui é a `Hask`, que representa os tipos contidos na linguagem Haskell. Em outras palavras:

- `Ob(Hask)` é constituída dos tipos, por exemplo: `Int` , `Bool` , `[a]` , `Char` , `Double` etc.
- `Hom(Hask)` é constituída das funções, por exemplo: `length :: [a] -> Int` , `show :: a -> String` , `dobro :: Double -> Double` etc.
- A noção de composição é atribuída ao operador `.` (visto no capítulo 4).
- O operador `.` é associativo.
- Para todo tipo, há a função `id :: a -> a` , representando a identidade (a ser explicada mais a fundo nas próximas seções).
- A função `id` se comporta como elemento neutro.

Há uma classe que possui a noção de categoria chamada `Category` no módulo `Control.Category` :

```
class Category cat where
  -- | identidade
  id :: cat a a

  -- | composição
  (.) :: cat b c -> cat a b -> cat a c
```

Note que esta classe trabalha com tipos de *kind* `* -> * -> *` . A categoria `Hask` acontece na instância de `Category` para `->` (das funções), que é *kind* `* -> * -> *` . O contêiner `->` tem esse *kind*, pois `a -> b` é equivalente a `(->) a b` , formando um tipo de dado com duas variáveis ***type variable***.

Tendo em mente o tipo de `id :: cat a a` , se substituirmos `cat` por `->` , obteremos `id :: (->) a a` , ou seja, `id :: a -> a` quando há a substituição de `cat` por `->` .

Analogamente, $(.) :: \text{cat } b \text{ } c \rightarrow \text{cat } a \text{ } b \rightarrow \text{cat } a \text{ } c$ se torna $(.) :: (->) \text{ } b \text{ } c \rightarrow (->) \text{ } a \text{ } b \rightarrow (->) \text{ } a \text{ } c$, nos dando $(.) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$ como esperado.

A instância em questão é definida por:

```
instance Category (->) where
  id = GHC.Base.id
  (.) = (GHC.Base..)
```

Isso indica o que já sabíamos: `id` é a identidade, e `.`, a composição.

6.2 NOÇÃO MATEMÁTICA DE FUNTOR

Um funtor é uma relação entre duas categorias C e D , e mapeia objetos em objetos e morfismos em morfismos. Ou seja, todo objeto da categoria C terá um correspondente em D , e todo morfismo da categoria C terá um correspondente em D .

Pense em bolas brancas de papel ligadas por fios brancos. Se você trocar as bolas brancas por vermelhas e os fios brancos por vermelhos, entenderá o intuito do funtor.

A categoria C teria como objetos as bolas brancas, e a categoria D , as vermelhas. A categoria C teria como morfismos os fios brancos, e a categoria D , os vermelhos. O funtor seria o ato de trocas das bolas e dos fios, sem bagunçar o desenho que elas formam. Esse é o pensamento. Matematicamente, isso pode ser representado como:

- **Associação de valores:** para todo A de $\text{ob}(C)$, é associado $F(A)$ em $\text{ob}(D)$.

- **Associação de morfismos:** $f :: A \rightarrow B$ é associado a $F(f) :: F(A) \rightarrow F(B)$.

Isso deve satisfazer:

```
F(idX) = idF(X)
F(f.g) = F(f) . F(g)
```

Todo funtor em Haskell é um endofuntor, ou seja, a categoria de entrada dos funtores é **Hask**, assim como a sua saída. Um funtor em Haskell possui *kind* $* \rightarrow *$ e deve ser uma instância do *typeclass* **Functor**, que será visto a fundo no próximo capítulo.

6.3 FUNÇÃO IDENTIDADE EM HASKELL

Em Haskell (categoria Hask), temos a função `id`, que representa a função identidade apresentada anteriormente. Seu tipo segue a seguinte definição:

```
Prelude> :t id
id :: a -> a
```

Pela definição de polimorfismo paramétrico, isso significa que existe uma `id` para cada tipo `a`, ou seja, `id :: Int -> Int`, `id :: Bool -> Bool`, e assim por diante. Como dito, a função identidade age como elemento neutro, logo:

```
Prelude> id 5
5
```

```
Prelude> id True
True
```

Note que uma função de tipo `a -> a` só pode executar essa tarefa: nada. É difícil pensar em uma função que valha ao mesmo tempo para **todos** os tipos sem restrição alguma, por isso a única

tarefa possível para `id` é não fazer nada. A função `id` é usada, na prática, para, por exemplo, ignorar um argumento (e o *currying* também) de uma função de alta ordem. Isto é, uma função que pede uma função, e você precisa que este argumento não realize operação alguma. Veja um exemplo:

```
module Cap6 where
```

```
data Produto = Produto {preco :: Double, desconto :: Double} deriving (Show, Eq, Ord)
```

```
buscaProd :: (Produto -> a) -> [Produto] -> ([a] -> a) -> a
buscaProd f ps g = g (map f ps)
```

A função `buscaProd` possui um tipo esquisito, porém bem útil. Ela possui como parâmetro uma função de tipo `Produto -> a` (qualquer função de projeção de `Produto` entra aqui, `preco` ou `desconto`, pois `preco :: Produto -> Double` e `desconto :: Produto -> Double`), uma lista de produtos e uma função do tipo `[a] -> a`.

Aqui podemos realizar operações típicas de um *fold*, como somar os elementos da lista, obter o maior, obter o menor etc. Se quisermos o maior desconto do `Produto`, faremos:

```
Cap6> buscaProd desconto [Produto 20.99 0.2, Produto 4.99 0.7, Produto 1.99 0.4] minimum
0.2
```

Ou seja, ele vai extrair o `desconto` de todos os produtos, deixando a lista como `[0.2, 0.7, 0.4]`, e vai extrair o menor, que é `0.2` (ou seja, 20% de desconto).

Se quisermos a soma dos `precos`, faremos:

```
Cap6> buscaProd preco [Produto 20.99 0.2, Produto 4.99 0.7, Produto 1.99 0.4] sum
```

27.969999999999995

Também se quisermos o Produto com maior valor e menor desconto, primeiramente implementaremos a função:

```
maiorPrecoMenorDesc :: Produto -> Produto -> Produto
maiorPrecoMenorDesc (Produto p1 d1) (Produto p2 d2) = Produto (max
x p1 p2) (min d1 d2)
```

Lembra-se de que o `foldl` transforma uma lista em um valor? Nesse caso, usaremos `foldl maiorPrecoMenorDesc (Produto 0 1)`, que executa a função `maiorPrecoMenorDesc` em todos os produtos, tendo como retorno o produto de maior preço e menor desconto. Note o valor inicial `Produto 0 1`: o `0` é o elemento neutro para o `max` do preço (nenhum preço é menor que zero), e o `1` (ou seja, 100%) é o maior desconto possível, servindo de elemento neutro para o `min` do desconto. Para ajudar nesta tarefa, implementaremos:

```
todos :: [Produto] -> Produto
todos ps = foldl maiorPrecoMenorDesc (Produto 0 1) ps
```

Para esse teste, não vamos extrair nenhum valor de `Produto`, ou seja, queremos ignorar a extração dos valores. Logo, a única função que pode nos ajudar é aquela que não faz nada: `id`.

```
Cap6> buscaProd id [Produto 20.99 0.2, Produto 4.99 0.7, Produto
1.99 0.4] todos Produto {preco = 20.99, desconto = 0.2}
```

6.4 CONCLUSÃO

Neste capítulo, abordamos, de maneira leve e informal, o conceito matemático de categorias. Se, de alguma forma, você achou pesado, é necessário ter em mente apenas as nomenclaturas: (endo)funtor, morfismo, objeto e categoria `Hask`.

FUNTORES

No capítulo anterior, foi introduzida a noção de *funtor* de acordo com a Teoria das Categorias. Neste capítulo, abordaremos o uso dos *funtores* na linguagem Haskell.

Em Haskell, um funtor é simplesmente uma classe (ou *typeclass*) que possui a função `fmap` a ser definida para toda instância. Ou seja, todo tipo de *kind* `* -> *` que seja instância de `Functor` deve "saber" como levar uma função `g` de tipo `a -> b` para dentro de um contêiner de tipo `f a`, resultando assim em algo do tipo `f b`.

A entrada de uma função `g` de tipo `a -> b` é claramente de tipo `a`, fazendo com que algo do tipo `f a` seja barrado com ***type mismatch*** pelo compilador, já que `a` e `f a` são valores de tipos diferentes. A classe `Functor` é definida como:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

A função `fmap` faz com que a função `g`, de tipo `a -> b`, seja "jogada para dentro" de `f a`, e esta é a intuição sobre os *funtores*. A partir deste raciocínio, é possível notar que a lista `[]` (que é um tipo de *kind* `* -> *`) possui uma instância de `Functor`, e seu `fmap` é a conhecida função `map`.

Podemos pensar também que os *funtores* aqui em Haskell generalizam de alguma maneira a ação do `map` em relação às listas. Um exemplo lúdico é o ato de fazer carinho em um gato. A ação do carinho é uma função. Se o gato entrar em uma caixa, grande de preferência, você não fará carinho na caixa (`f a` não pode ser entrada de `g`), então você deve entrar na caixa. O ato de entrar na caixa para fazer o carinho seria o tal `fmap` .

Vamos checar seus tipos:

```
Prelude> :t map
map :: (a -> b) -> [a] -> [b]
```

```
Prelude> :t fmap
fmap :: (Functor f) => (a -> b) -> f a -> f b
```

Nisso, notamos que, ao trocar `[]` na função `map` por um tipo `f` de *kind* `* -> *` , qualquer que seja instância de `Functor` , obtemos o tipo de `fmap` . O tipo de `fmap` nos diz que ela possui como entrada uma função de tipo `a -> b` e um valor dentro de um contêiner `f` de tipo `f a` para devolver algo de tipo `f b` .

Se usarmos o *currying*, podemos enxergar `fmap g` como uma função que recebe algo de tipo `f a -> f b` , ou seja, algo que recebe `f a` e devolve `f b` . Todo tipo que seja instância de `Functor` deve ser implementado segundo as leis dos *funtores* (assim como nos monoides).

```
fmap id x = x
fmap (g.h) x = (fmap g . fmap h) x
```

Estas regras valem para todo valor `x` de tipo `f a` . As leis acima indicam que se jogarmos para dentro a função `id` não acontecerá nada e que se houver uma composição de funções no

argumento de `fmap` equivalerá à composição listada na expressão `(fmap g . fmap h) x`, ou seja, será efetuado `fmap h x` primeiro e o resultado disso será o argumento de `fmap g`.

Ressalto que o compilador permite que um funtor não siga essas regras; porém, para efeito de testes por propriedades, é recomendado seguir.

7.1 FUNTOR MAYBE

O tipo `Maybe` é um tipo de *kind* `* -> *` definido como:

```
data Maybe a = Just a | Nothing
```

O *value constructor* `Just`, que possui um campo de tipo variável `a`, pode representar uma computação validada, ao passo que `Nothing` representa um erro ou ausência de uma computação. Este tipo é usado para validações de dados, como:

```
divisao :: Double -> Double -> Maybe Double
divisao x 0 = Nothing
divisao x y = Just (x/y)
```

A função `divisao` recebe dois parâmetros que serão testados pelo *pattern matching*. No caso de o segundo ser `0`, o retorno será `Nothing`, pois não há divisão por zero.

O último padrão indica que o segundo parâmetro é diferente de zero, então a divisão será colocada dentro do *value constructor* `Just`, indicando uma operação válida. A função está correta e possui uma boa regra de validação, porém ela não opera com a função `2*`, por exemplo:

```
Prelude> 2 * (divisao 10 2)
```

```
<interactive>:1:1: error: [GHC-39999]
  • No instance for 'Num (Maybe Double)' arising from the liter
al '2'
  • In the first argument of '(*)', namely '2'
    In the expression: 2 * (divisao 10 2)
    In an equation for 'it': it = 2 * (divisao 10 2)
```

Isso fará com que o compilador procure uma instância de `Num` para `Maybe a` (que não há) e, mesmo que houvesse, a expressão anterior é errada e não compilaria, já que `Double` é diferente de `Maybe Double`. O tipo `Double` representa um número que é tangível e pode ser usado, já `Maybe Double` pode não conter um número e, assim, representar uma operação malsucedida (uma divisão por zero, por exemplo). Por isso, é necessária uma maneira de operarmos um `Double` com um `Maybe Double`, genericamente `a` com `Maybe a`. Essa "mistura" se dá através de uma instância de `Functor` (já implementada):

```
instance Functor Maybe where
  fmap g Nothing = Nothing
  fmap g (Just x) = Just (g x)
```

Quando o parâmetro de `fmap g` (lembre-se do *currying*) é `Nothing`, nada deve ser feito, pois é um contêiner vazio e não há como fazer a função passar para dentro. Quando o parâmetro for `Just x`, passamos a função `g` para dentro do contêiner `Just (g x)`, e nada mais.

Agora é possível calcular a expressão anterior usando o `fmap`:

```
Prelude> fmap (2*) (divisao 10 2)
Just 10
```

Em vez de ficarmos usando estruturas condicionais (como em outras linguagens), o funtor `Maybe` nos dá uma solução prática para lidar com erros, como no exemplo a seguir:

```
Prelude> fmap (2*) (divisao 10 0)
Nothing
```

Assim, nos dá um controle melhor sobre situações indesejadas, semelhantes a dividir por zero.

Há também um operador que representa o `fmap`, chamado `<$>`, que nada mais é do que sua forma infixa:

```
Prelude> (2*) <$> (divisao 10 2)
Just 10
```

A escolha entre um e outro é apenas uma questão de estilo de código. O operador `<$>` é usado frequentemente junto aos Funtores Aplicativos a serem discutidos nas próximas seções desse mesmo capítulo.

7.2 CRIANDO SEU FUNTOR

Os contêineres podem ser vistos como um rótulo. No exemplo anterior, `Maybe` a rotula um tipo `a` a qualquer, indicando que pode haver um valor ou um erro lá. As listas `[a]` indicam que pode haver vários valores dispostos em forma de lista. Há também o `IO a`, que indica que o tipo `a` é oriundo de uma computação com efeitos (leitura e escrita de arquivos, por exemplo, que será estudado a fundo no *capítulo 9*).

Os *funtores* ajudam as funções a operar mais facilmente com tipos similares aos citados. É possível criar nosso contêiner contendo dois espaços para um mesmo tipo `a`, como por exemplo, o tipo:

```
data Dupla a = Dupla a a deriving Show
```

Este tipo é análogo a (a, a) . O tipo `Dupla` nos ajuda a trabalhar uma instância de funtor diferente do que a provida pela linguagem. Vamos analisar:

```
instance Functor Dupla where
    fmap g (Dupla x y) = Dupla (g x) (g y)
```

Nesse código, vemos que `Dupla x y` é um valor de tipo `Dupla a`, assim sendo `x :: a` (`x` tem o tipo `a`) e `y :: a` também. Logo, o resultado de `fmap g` calculado em `Dupla x y` nos dá `Dupla (g x) (g y)`, resultando em algo do tipo `Dupla b`, que é exatamente a definição de `fmap`.

Moral da história: não importa quantos campos de mesmo tipo seu contêiner possua, a função `g` pode se distribuir por mais de um campo, se necessário.

7.3 FUNTORES APLICATIVOS

Quando temos uma função dentro de um funtor (`f (a -> b)`), e queremos operar em cima de um valor `f a`, temos a necessidade de usar o operador `<*>` definido na classe `Applicative`, que rege os Futores Aplicativos. Outro problema é que o `fmap` sozinho funciona apenas para funções com um parâmetro, como o `2*` usado anteriormente.

Um funtor aplicativo ajuda a fazer uma função receber um valor como parâmetro quando tanto a função como o(s) parâmetro(s) estão dentro de tipos de *kind* `* -> *`. A classe `Applicative` pode ser definida como:

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

Note que `<*>` está envolvido com funções `f (a -> b)` e valores `f a`, dentro de um funtor `f`. Preste bem atenção agora e compare as expressões a seguir:

```
($) :: (a -> b) -> a -> b
<*> :: f (a -> b) -> f a -> f b
```

Notou alguma semelhança? Se sim, você está muito bem! Se não, experimente apagar os `f`'s de `<*>`. Recairemos em `$`. Mágico, certo? Você pode pensar em `<*>` como um nome *gourmet* para o `$`. Eles fazem a mesma coisa (aplicam uma função sobre um valor), porém, no caso do primeiro, há um funtor envolvido. A função `pure` simplesmente coloca um valor dentro de um funtor aplicativo, e nada mais.

Para testar os exemplos, devemos importar o módulo `Control.Applicative`. Para os testes, vamos usar as instâncias de `Applicative` de nossos dois *funtores* favoritos: `Maybe` e `[]`.

```
instance Applicative Maybe where
  pure x = Just x
  (Just f) <*> (Just x) = Just (f x)
  _ <*> _ = Nothing

instance Applicative [] where
  pure x = [x]
  fs <*> xs = [f x | f <- fs, x <- xs]
```

Na instância de `Applicative` para `Maybe`, vemos que o `pure` joga um valor `x` qualquer de tipo `a` dentro do `Just`, como `pure 5 = Just 5` nesse caso. O operador `<*>` provê a aplicação de `f` em `x`, sendo que tanto `f` de tipo `a -> b` quanto `x` de tipo `a` estão presos dentro do funtor `Maybe`. Com isso, temos `(Just f) <*> (Just x) = Just (f x)`, ou seja, `f x` dentro do funtor `Maybe` (*value constructor* `Just`).

No caso de `[]`, o `pure` coloca um valor dentro de uma lista, neste caso, `pure 5 = [5]`. No caso do operador de aplicação, serão aplicadas todas as funções dentro da lista `fs` em todos os valores da lista `xs`.

A *list comprehension* `[f x | f <- fs, x <- xs]` diz que, para todo `f` dentro de `fs` e todo `x` dentro de `xs`, ocorrerá a aplicação (execução da função) `f x`.

```
Prelude> Just (2*) <*> Just 10
Just 20
```

```
Prelude> [(2*), id, (3*)] <*> [1,2]
[2,4,1,2,3,6]
```

Uma conclusão possível de se tirar é `fmap f x = pure f <*> x`. Outra aplicação legal dos Funtores Aplicativos é que se, por exemplo, quisermos dividir `Just 10` por `Just 2`, teríamos de extrair o 10 e o 2, e depois dividir.

Lembre-se de que extrair valores de tipos com instância `funtor` nem sempre é legal. Pense em como extrair algum valor de uma lista vazia `[]` ou de um erro `Nothing`. Para executar a divisão anterior, bastaria fazer:

```
Prelude> pure (/) <*> Just 10 <*> Just 2
Just 5
```

Ou mais comumente:

```
Prelude> (/) <$> Just 10 <*> Just 2
Just 5.0
```

A primeira expressão indica o uso do `pure` e a segunda do `fmap` (`<$>`). As expressões se associam à esquerda e fazem o uso do *currying* para funcionar. Por exemplo, na primeira expressão:

```

pure (/) <*> Just 10 <*> Just 2 =
(pure (/) <*> Just 10) <*> Just 2 =
(Just (/) <*> Just 10) <*> Just 2 =
Just (10/) <*> Just 2 =
Just (10/2) =
Just 5

```

Note que a primeira linha representa a expressão a calcular, e a segunda foi usada na associação à esquerda. Já na terceira, temos a definição do `pure` para o funtor aplicativo `Maybe`; na quarta foi usado o *currying* (lembre-se de que `10/` é o *currying* da função divisão, pois falta o segundo argumento) e a definição de `<*>` para `Maybe`; e na quinta, temos a mesma definição e, finalmente, é computado `10/2`.

Na segunda expressão, temos:

```

((/) <$> Just 10) <*> Just 2 =
Just (10/) <*> Just 2 =
Just (10/2) =
Just 5

```

Da primeira para a segunda linha, foi usado o `fmap` e o *currying* e, na próxima, a definição de `<*>`.

No *framework* para desenvolvimento de aplicações web `Yesod`, são usados Funtores Aplicativos para ler formulários de um HTML e fazer uma requisição `GET` ou `POST` para uma rota economizando muitas linhas. O exemplo usado lá é parecido com isso.

Suponha haver um funtor aplicativo `Form`, que representa um valor digitado em campo `input type=text`, por exemplo. Considere o tipo `Pessoa = Pessoa String Int`, que possui um nome `String` e uma idade `Int`. O tipo `Pessoa` é na verdade uma função `String -> Int -> Pessoa` pelo *currying*. Logo, se

digitarmos João e 40 , teremos Form João e Form 40 no Haskell.

```
Pessoa <$> Form "João" <*> Form 40
Form (Pessoa "João") <*> Form 40
Form (Pessoa "João" 40)
```

Ao passar o tipo Pessoa construído "para dentro" de Form , poderíamos salvar o registro digitado no formulário em uma tabela do banco de dados usando o Persistent .

7.4 FUNTORES CONTRAVARIANTES

Esta seção é opcional e você pode pulá-la sem problema algum no aprendizado do restante do livro. Mas se você é um leitor curioso e que realmente quer ir além na linguagem, e ter um *boost* em sua capacidade de pensamento e abstração, esta seção é para você.

Primeiramente, a classe Functor vista aqui, na realidade, tem um "sobrenome" chamado **Covariante**. Uma pergunta que pode ter surgido é: "será que qualquer tipo pode ser instância de Functor?".

Em um primeiro momento, somos levados a crer que sim, a partir do momento em que criamos o tipo Dupla (que carrega dois valores de tipo a) e vimos que o tipo [] carrega vários valores de tipo a . Vamos analisar, então, o tipo:

```
module Cap07 where
```

```
data Predicado a = Predicado {runPred :: a -> Bool}
```

Ele possui um *value constructor*, chamado Predicado , e um campo de tipo a -> Bool , chamado runPred . A função

`runPred` recebe um valor de um tipo qualquer `a` e retorna um valor `Bool`.

Esse tipo generaliza a noção de validação de dados, ou seja, tem várias regras de validação "dentro" deste tipo de *kind* `* -> *`. A pergunta que fica é: podemos ter uma instância de `Functor` para `Predicado`, dado que ele tem *kind* `* -> *`? Para respondê-la, vamos entender o tipo `Predicado`.

```
ehMenor4 :: Predicado Int
ehMenor4 = Predicado (\x -> x < 4)
```

A função monta um predicado que checa se um número é menor que 4 ou não. Perceba que o tipo `Predicado` carrega uma função dentro e esta tem o tipo `Int -> Bool` (por causa do tipo de `ehMenor4` ser `Predicado Int`). Portanto, o parâmetro `x` do lambda `\x -> x < 4` é um `Int`.

Veja um outro pequeno exemplo:

```
tamanhoOito :: Predicado String
tamanhoOito = Predicado (\x -> length x == 8)
```

Ele verifica se uma `String` tem tamanho 8 ou não. Como o tipo da função é `Predicado String`, a entrada do lambda dentro de `Predicado` será uma `String`. Para usar as funções anteriores, fazemos:

```
Cap07> let func = runPred ehMenor4
Cap07> func 8
False
Cap07> func (-5)
True
```

Primeiramente, utilizamos uma expressão usando o `let`, para podermos usar `func` sempre que precisarmos na sessão ativa do

GHCi. A função `func` contém `runPred ehMenor4`, que extrai a função de dentro de `Predicado`. Logo, `func = \x -> x < 4` ou, em outras palavras, `func x = x < 4`, e ela possui o tipo `Int -> Bool`.

As expressões `func 8` e `func (-5)` testam se os números passados no argumento são menores que 4 ou não:

```
Cap07> let func = runPred tamanho0ito
Cap07> func "Haskell"
False
Cap07> func "HASKELL "
True
```

O mesmo ocorre com a função `tamanho0ito`. Entendido o tipo `Predicado` (que é realmente um pouco diferente), vamos agora recapitular o tipo de `fmap`.

```
Cap07> :t fmap
fmap :: (Functor f) => (a -> b) -> f a -> f b
```

A pergunta que fica no ar é: por conta do `Predicado` possuir *kind* `* -> *`, é possível torná-lo instância de `Functor`? A resposta é um grande **não**! E tal resposta causa estranheza, pois até agora qualquer tipo mostrado era possível tornar um *functor*.

Tendo em mente `(a -> b) -> f a -> f b`, como nossa tentativa de *functor* é o `Predicado`, precisaríamos ter `(a -> b) -> Predicado a -> Predicado b`. Teríamos então duas possibilidades de instância de `Functor` para `Predicado`. A primeira seria:

```
instance Functor Predicado where
    fmap g (Predicado p) = Predicado (g . p)
```

E a segunda seria:

```
instance Functor Predicado where
    fmap g (Predicado p) = Predicado (p . g)
```

Nos dois casos, só poderíamos ter uma composição, pois $(a \rightarrow b) \rightarrow \text{Predicado } a \rightarrow \text{Predicado } b$ possui dois parâmetros que envolvem: uma função de tipo $a \rightarrow b$ e um `Predicado a` que possui dentro dele também uma função, que no caso é de tipo $a \rightarrow \text{Bool}$. O *pattern matching* `Predicado p` extrai a nossa função `p` de tipo $a \rightarrow \text{Bool}$ de dentro.

Agora, vamos analisar `g . p`. O `p` tem entrada `a` e saída `Bool`, enquanto `g` tem entrada `a`, portanto, temos um erro de compilação. Analisando `p . g`, temos que `g` tem entrada `a` e saída `b`, porém a entrada de `p` é `a`, ocorrendo um erro novamente.

Logo, não é possível criarmos uma instância de `Functor` para `Predicado`. Porém, eis que há uma classe que nesse caso nos salva, a `Contravariant`, do módulo `Data.Functor.Contravariant` (que hoje já faz parte da base, então basta importá-lo):

```
class Contravariant f where
    contramap :: (a -> b) -> f b -> f a
```

Um `Functor Contravariante` é um `Functor` na categoria oposta de `Hask`. A categoria oposta é aquela que inverte todas as direções das funções $\rightarrow$. Ou seja, se temos uma função de tipo $\text{Int} \rightarrow \text{String}$ em `Hask`, a mesma terá $\text{String} \rightarrow \text{Int}$ na categoria oposta de `Hask` (que chamaremos de `Hask*`, por conveniência).

Voltemos ao `contramap`. A intuição é a mesma de um *functor*: jogar uma função para dentro do contêiner, só que dessa vez a função jogada tem de ter a "flecha invertida". Pense que $a \rightarrow b$

vira `b -> a` quando jogada para dentro de `f b`, para ter o tipo de dentro de `f` batendo com a entrada de `b -> a`. No caso de `Predicado`, conseguimos que ele tenha uma instância de `Functor Contravariante`:

```
instance Contravariant Predicado where
    contramap g (Predicado p) = Predicado (p . g)
```

Como a função `contramap` tem tipo `(a -> b) -> Predicado b -> Predicado a` (no caso do `Predicado`), temos que `p` da expressão `Predicado (p . g)` tem tipo `b -> Bool`, então temos que `g` possui entrada `a` e saída `b`. Esta saída entra em `p` que tem entrada `b` e saída `Bool`, nos dando `p . g :: a -> Bool` e, por fim, `Predicado (p . g) :: Predicado a`, como queríamos.

Agora, toda vez que quisermos trocar as entradas das funções de dentro de `Predicado`, usamos o `contramap`. Por exemplo, se tivermos uma função `String -> Int`, podemos jogar para dentro de `ehNegativo :: Predicado Int`, nos retornando algo de tipo `Predicado String`. Testando no GHCi:

```
Cap07> let func = contramap length ehMenor4
Cap07> let g = runPred func
Cap07> g "Haskell"
False
Cap07> g "ABA"
True
```

Podemos observar que `length` é "jogada para dentro" de `ehMenor4 :: Predicado Int` através do `contramap`. Na segunda linha, a nova função é extraída a partir de `runPred`, logo, `g` possui o tipo `String -> Bool` (`func` possui o tipo `Predicado String`) e verifica se uma `String` possui o tamanho menor que 4 e não mais um `Int`.

A partir daqui, você já tem um conhecimento um pouco mais profundo sobre *funtores* e está preparado para encarar **Bifuntores** e **Profuntores**, que são *funtores* para tipos de *kind* $* \rightarrow * \rightarrow *$. Os Bifuntores jogam duas funções "sem mudar a seta" para dentro, enquanto os Profuntores jogam uma função "sem mudar a seta" e outra "mudando a seta" para dentro do contêiner.

Apesar de meu profundo interesse em Bifuntores e Profuntores, a abordagem destes conceitos pode elevar a complexidade do livro que tem como objetivo ser uma introdução à programação funcional utilizando a linguagem Haskell.

7.5 MINIPROJETO: CONTINUAÇÃO USANDO FUNTORES

Para facilitar buscas de funcionários e projetos em arquivos (buscas em qualquer tipo de arquivo, não só para eles), devemos usar uma espécie de índice (assim como uma chave primária). Esse tipo que representa índice será um tipo de *kind* $* \rightarrow *$ e instância de `Functor`.

```
module Projeto where

data Indice a = Indice {indice :: Int, dados :: a}

-- rotinas previamente implementadas

instance Functor Indice where
    fmap f (Indice i dados) = Indice i (f dados)
```

O tipo `Indice` possui um *value constructor* chamado `Indice`, com dois campos de tipos `Int` e `a`. Esse tipo representa um valor indexado. `Indice Pessoa` e `Indice Projeto` indicam que estamos criando campos indexados sem

mexer nos tipos já implementados, além de separar índice de dados (o pacote `persistent` propõe algo similar ao mexer com dados oriundos de uma tabela de banco de dados).

Como `Índice` é algo de *kind* `* -> *`, aproveitamos para criar uma instância de functor. Fazendo isso, não precisaremos ficar extraindo os dados do índice toda vez que for necessário operar com `Pessoa` ou `Projeto`. Por exemplo, a função `promover :: Pessoa -> Pessoa` não pode ter a entrada como `Índice Pessoa`, então temos de "jogar para dentro" a função `promover` através da expressão `Índice i (f dados)`. Note que o parâmetro `i` é ignorado, pois não é de tipo variável e, portanto, fica fora da ação do `fmap`.

Com a instância de `Functor` para índice finalizada, é possível operar com o contêiner `Índice` sem a necessidade de ficar extraindo o valor de dentro.

7.6 EXERCÍCIOS

7.1) Faça uma instância de `Functor` para o tipo `Coisa`, definido no início do capítulo 5. A função `g` deve "ir para dentro" em todas as coordenadas de `Coisa`. No caso de `ZeroCoisa`, o `fmap` deve retornar `ZeroCoisa`.

7.2) Aproveitando o exercício anterior, faça uma instância de `Applicative Functor` para o tipo `Coisa`.

7.3) Crie a função `mult234 :: Double -> Coisa Double` que multiplica por 2 a primeira coordenada, por 3 a segunda, e por 4 a terceira. Use a instância de `Applicative` feita no exercício anterior.

7.4) Escreva uma instância para `Functor` e `Applicative` para o tipo `Arvore`, visto no *capítulo 5*.

7.5) Escreva uma instância de `Functor` para o tipo `data Fantasma a = Fantasma .`

7.6) Escreva uma possível instância de `Functor` para o tipo `data Dupla a = Dupla a Int a .`

7.7) É possível criar uma instância de `Functor` para o tipo `Derp a = Derp {runDerp :: Bool -> a} ?` Justifique e, em caso positivo, crie a instância de `Functor` e `Applicative`.

7.8) (Opcional) Implemente:

- O tipo `NovoPred` possui um *value constructor* de mesmo nome, contendo o campo `runNovoPred` de tipo `Maybe a -> Bool` ;
- Crie uma instância de `Functor` Contravariante para o tipo `NovoPred` .

7.7 CONCLUSÃO

Este capítulo foi dedicado a mais uma classe (*typeclass*) especial, os *funtores*. Diferentemente do capítulo anterior, que mostrou uma noção matemática, aqui foi abordado o uso prático de um funtor.

Todo tipo de *kind* `* -> *` pode ser pensado como um rótulo para o tipo, uma caixa rotulante, por exemplo: `[]` representa uma estante onde podem ser guardadas várias coisas de tipo `a` ; `Maybe` representa um valor `a` ausente ou não; um *funtor* é apenas como vamos jogar uma ação dentro dessas caixas.

O Funtor Aplicativo ajuda nos casos em que uma função f está dentro desses tipos de *kind* $* \rightarrow *$, e que $\langle * \rangle$ é o análogo de $\$$ para tipos de *kind* $* \rightarrow *$. O entendimento deste capítulo prepara o aluno para o entendimento do último conceito teórico do livro, as famosas **Monads**.

MÔNADAS

Monad (ou Mônada, em português) é um conceito oriundo da Teoria das Categorias e serve para representar matematicamente a computação com efeitos colaterais (*side-effects*), como leitura e escrita de arquivos, aplicações **web**, banco de dados, entre outros. A partir deste capítulo, você será capaz de compreender e desenvolver aplicações em Haskell, além de entender os benefícios do uso deste conceito para a construção de aplicações mais robustas.

Para começar, precisamos introduzir uma nomenclatura presente na literatura sobre a linguagem Haskell quando se pesquisa o termo **Monad**. Tal nomenclatura será útil para o entendimento do conceito, mas no dia a dia sua presença é meramente opcional. A intenção é analisar a categoria dos Endofuntores do **Hask**, e não **Hask** em si, pois Mônadas são estruturas que moram nesta nova categoria.

Ressalta-se ao leitor que não é necessário ser um *expert* em Teoria das Categorias. Entretanto, ter um conhecimento básico do que está por trás dos conceitos aqui em Haskell lhe proporcionará um diferencial incrível frente ao mercado de trabalho, já que você poderá enxergar tais estruturas em sua linguagem de trabalho e escrever um código mais fácil de manter, que precise de menos

testes e esteja em um alto nível de abstração. Basta olhar para os promises do AngularJS após o entendimento deste capítulo para sentir o poder destes conceitos.

8.1 TRANSFORMAÇÕES NATURAIS

Uma Mônada, assunto principal deste capítulo, é composta por um *functor* e duas transformações naturais. Para entender melhor o conceito em foco, será introduzida primeiramente a definição de transformações naturais.

Uma **transformação natural** é outro conceito emprestado da Teoria das Categorias. Aqui podemos enxergá-la como uma função entre dois *funtores*. Como o formalismo matemático está além do escopo deste livro, podemos pensar em transformações naturais como funções de tipo $F\ a \rightarrow G\ a$, em que a é um tipo de *kind* $*$ qualquer (novamente lembrando `Int`, `String`, `Char`, `Bool` etc.), enquanto F e G são tipos de *kind* $*$ $\rightarrow *$ quaisquer (`[]`, `Identity`, `Maybe`, `IO` etc.).

Uma observação quanto ao *functor* `Identity`, do módulo `Control.Monad.Identity`, é que ele não faz nenhuma mudança de contexto, ou seja, `Identity Int` é apenas `Int`, assim como `Int` pode ser visto como `Identity Int`. Em instantes, veremos o motivo de pensarmos neste *functor* que não faz nada.

As transformações naturais são usadas quando precisamos de funções entre contêineres (tipos de *kind* $*$ $\rightarrow *$) sem nos preocuparmos com o que há dentro. Vamos agora analisar alguns exemplos de transformações naturais:

```
maybeLista :: Maybe a -> [a]
maybeLista (Just x) = [x]
maybeLista Nothing = []
```

Esta transformação natural troca o *functor* `Maybe` pelo *functor* `[]`. A entrada desta função possui um *pattern matching* contra `Maybe`. Caso tenhamos o *value constructor* `Just x` — lembrando que $x :: a$ —, este elemento é colocado dentro de uma lista usando o símbolo `[]`; caso seja `Nothing`, a lista vazia é retornada.

Os próximos exemplos serão mais importantes para continuarmos.

```
toLista :: a -> [a]
toLista x = [x]
```

```
Prelude> toLista 6
[6]
```

A função `toLista` pode ser pensada como uma transformação natural que troca o *functor* `Identity` (por conta de `Identity a = a`) pelo *functor* `[]`, simplesmente colocando um elemento $x :: a$ dentro desta lista. Essa função nos mostra que, ao criarmos uma lista em um programa, estamos lidando com uma transformação natural entre `Identity` e `[]`.

Como dito na introdução deste capítulo, é importante analisarmos a categoria dos Endofuntores, cujos objetos são os *funtores* (qualquer tipo de *kind* $* \rightarrow *$ que seja instância de `Functor`). Seus morfismos são as transformações naturais que acabamos de ver.

8.2 DEFINIÇÃO

Matematicamente, uma Mônada na categoria *Hask* consiste, basicamente, em três "coisas":

- Um Endofuntor m , ou seja, um tipo de *kind* $* \rightarrow *$, instância de *Functor*.
- Uma transformação natural que troca *Identity* por m , chamada *return*. Note que $\text{return} :: a \rightarrow m\ a$.
- Uma transformação natural que troca $m\ m$ por m , chamada *join*. Note que $\text{join} :: m\ (m\ a) \rightarrow m\ a$.

E devem seguir as leis das Mônadas. Essas leis são as mesmas válidas para monoídes, pois é sabido que uma Mônada em *Hask* é um monoíde na categoria dos Endofuntores de *Hask* (os objetos dessa categoria são *funtores*, os morfismos são as transformações naturais, e a noção de composição não é dada por $(.)$, mas por $(<=>)$, que é um caso especial de $(>=>)$, visto abaixo. A estrutura de monoíde vem do fato de *return* agir como *mempty* e *join* como *mappend*).

Em Haskell, as Mônadas são representadas pela *typeclass* **Monad**, definida no módulo `Control.Monad`, e possuem *kind* $* \rightarrow *$. Isso significa que um tipo m de *kind* $* \rightarrow *$, desde que seja uma instância de *Functor*, pode se tornar uma Mônada ao definirmos as transformações naturais *return* e *join*.

Porém, em Haskell, *join* é uma função que vem de graça, pois ela é escrita em termos da função infixa $>=>$, chamada ***bind***.

Conclusão: para usarmos instâncias de *Monad* para qualquer *functor* m , é necessário sabermos como implementar *return* e

>>= para esse *functor* m , já que cada um possuirá suas peculiaridades.

Os próximos tópicos nos trarão informações sobre `return` , `>>=` e `join` .

return

A transformação natural `return` representa como devemos pegar um valor de tipo a e colocar dentro de um contêiner de *kind* $* \rightarrow *$ m , que possui uma instância de *functor*. Ludicamente, pense que a tem o tipo `Livro` e m o tipo `Caixa` , então o ato de pôr seu livro na caixa chama-se `return` .

Para fixar, em vez de falar "Colocarei o livro na caixa", fale "return livro", que dará no mesmo! Outro exemplo é se estivermos focados no *functor* `[]` e no *functor* `Maybe` , então:

```
Prelude> return 6 :: [Int]
[6]
```

```
Prelude> return 6 :: Maybe Int
Just 6
```

Lembre sempre de que `::` indica o tipo que queremos resolver na expressão anterior. A primeira expressão coloca o número `6` no *functor* `[]` e, na seguinte, coloca o mesmo número dentro do *functor* `Maybe` . Se não colocássemos `::` , teríamos o seguinte tipo para a expressão `return 5` :

```
Prelude> :t return 5
return 5 :: (Monad m, Num a) => m a
```

Isso é uma restrição para a classe `Num` em a , e outra para a classe `Monad` em m , ou seja, m é alguma Mônada não

especificada ainda. Lembre-se, `return` nunca realiza operação alguma com o valor de dentro, ele só coloca dentro de um *functor*.

join e bind (>>=)

A transformação natural (de agora em diante, chamaremos de função mesmo) `join` e a função infixa `>>=` estão intimamente ligadas. Se o Haskell pedisse para definirmos `join`, ganharíamos de graça `>>=`. Porém, a linguagem nos pede para definir sempre `>>=`, logo, ganhamos de graça `join`. O "ganhar de graça" quer dizer que basta definir um que o outro já fica definido automaticamente, sem qualquer intervenção humana.

A função `join` simplesmente junta dois *funtores* em um só, ou seja, `join` transforma `[[[]]]` em `[]`, `Maybe Maybe` em `Maybe`, `IO IO` em `IO`, e assim por diante (note que os *funtores* devem ser iguais). Ludicamente, vamos supor que sua mãe pegue um livro seu, coloque-o em uma caixa, e depois pegue esta caixa e coloque-a em outra. O que você teria? Seu livro dentro da caixa da caixa. O que o `join` faz? Jogaria uma caixa fora e seu livro ficaria dentro de uma caixa apenas. Veja um exemplo:

```
Prelude> :m Control.Monad

Prelude Control.Monad> join [[4]]
[4]

Prelude Control.Monad> join (Just (Just True))
Just True

Prelude Control.Monad> join [[[20]]]
[[20]]

Prelude Control.Monad> join ["w"]
"w"
```

Esses códigos nos mostram uma camada de *functor* sendo ignorada nos três exemplos. A pergunta que vem na cabeça é: e se tivermos mais de um elemento nas listas? A resposta será dada na próxima seção.

Entendida a função `join`, é necessário o entendimento de `>>=`, que talvez, até aqui, seja a função mais importante vista. Primeiramente, vamos inspecionar seu tipo:

```
Prelude> :m Control.Monad

Prelude Control.Monad> :t (>>=)
(>>=) :: Monad m => m a -> (a -> m b) -> m b
```

Primeiro, note que há uma restrição em `m`, dizendo que este deve ser instância de `Monad` (obviamente). A função de alta ordem `>>=` possui como entrada *um valor* `m a` e *uma função de tipo* `a -> m b`, e na saída temos um valor de tipo `m b`. Vamos relembrar da função `|>` vista no *capítulo 4*; ela foi definida como:

```
(|>) :: a -> (a -> b) -> b
(|>) x f = f x
```

Agora compare, sem a restrição para ativarmos nossa habilidade de reconhecer padrões.

```
(>>=) :: m a -> (a -> m b) -> m b
(|>) :: a -> (a -> b) -> b
```

Notou algo de similar? Claro que sim. Se retirarmos o `m` de `>>=`, caímos em `|>`, logo `>>=` é o `|>` para Mônadas! Só para lembrar, há o operador `=<<`, que não vamos focar aqui, mas ele possui o tipo:

```
(<<=) :: Monad m => (a -> m b) -> m a -> m b
```

Compare este com a função `$`, vista no *capítulo 4*:

```
($) :: (a -> b) -> a -> b
```

Logo, `=<<` é o `$` para Mônadas! Só um detalhe: se escolhermos `m` como o *functor* `Identity`, ou seja, `Identity a = a`, obteríamos:

```
(>>=) :: Identity a -> (a -> Identity b) -> Identity b
```

Isso nos daria `(>>=) :: a -> (a -> b) -> b`, ou seja, `>>=` e `|>` são os mesmos sob o *functor* `Identity` (assim como `=<<` e `$`)! Como `>>=` recai em algo que já vimos, só precisamos entendê-lo nesse novo contexto e estaremos prontos para trabalhar em casos mais concretos.

Suas entradas são um valor de tipo `m a` e uma função de tipo `a -> m b`, temos então uma caixa `m` e um item `a`. Note que o tipo `a -> m b` possui entrada `a`, ou seja, poderíamos jogar tal função, via `fmap`, dentro da caixa `m a` e esta transformaria o elemento de dentro em `m b`. O resultado seria algo de tipo `m (m b)` que, com a ajuda do `join`, transformaria em `m b` novamente, que é o retorno da função `>>=`. Logo, teríamos:

```
x >>= f = join (fmap f x)
```

Isso é exatamente o processo descrito. Note que o `join` se faz extremamente necessário, pois é preciso acabar com um dos `m`. Vale ressaltar, mais uma vez, que tanto `>>=` quanto `|>` trabalham com o **valor na função, e não a função no valor**. Basta lembrar:

```
x |> f = f x
```

Note a ausência de `join` e `fmap` para este caso sem Mônadas. Vamos a um exemplo. Vamos analisar a seguinte expressão:

```
Prelude Control.Monad> Just "Olá" >=> (\x -> Just (x ++ " Mundo"))
)
Just "Olá Mundo"
```

O valor de entrada de `>=>` tem tipo `Maybe String`, e a função de entrada tem tipo `String -> Maybe String`, logo `a = String`, `b = String` e `m = Maybe`. Para entender melhor o resultado `Just "Olá Mundo"`, vamos analisar friamente o cálculo da expressão anterior.

```
Just "Olá" >=> (\x -> Just (x ++ " Mundo")) =
join (fmap (\x -> Just (x ++ " Mundo")) (Just "Olá")) =
join (Just ((\x -> Just (x ++ " Mundo")) "Olá")) =
join (Just (Just "Olá Mundo")) =
Just "Olá Mundo"
```

Primeiramente, o *lambda* `(\x -> Just (x ++ " Mundo"))` entra para dentro de `Just` presente no valor `Just "Olá"`, com ajuda da função `fmap`. Depois, `"Olá"` vira argumento do *lambda* `(\x -> Just (x ++ " Mundo"))`, retornando `Just (Just "Olá Mundo")`, que é argumento de `join`, finalizando o cálculo.

8.3 NOTAÇÃO "DO"

A escrita de funções que usam os operadores monádicos `join`, `return` e `>=>` pode ser bastante complicada para novatos na linguagem. Por isso, existe um facilitador na sintaxe chamado notação `do`, que permite o uso das funções anteriores de uma forma mais convencional e parecida com linguagens imperativas.

Primeiramente, vamos relembrar alguns conceitos para depois introduzir a notação. Vimos que `>=>` e `|>` são "irmãos" e, com

isso, temos que ambos se associam à esquerda com a maior prioridade. Lembre-se do exemplo do *capítulo 4*:

```
module Monadas where

import Control.Monad

funcI :: String -> String
funcI x = x
        |> reverse
        |> take 3
        |> tail
        |> (x ++)
```

Uma função análoga, usando o *funtor* `Maybe`, pode ser escrita e, em vez de usarmos `|>`, usaremos `>=>`.

```
funcMaybe :: String -> Maybe String
funcMaybe x = return x
              >=> \y -> return (reverse y)
              >=> \z -> return (take 3 z)
              >=> \w -> return (tail w)
              >=> \u -> return (x ++ u)
```

Com as chamadas:

```
Prelude> :l Monadas.hs

Monadas> funcI "Haskell"
"Haskellle"
Monadas> funcMaybe "Haskell"
Just "Haskellle"
```

Isso nos mostra as similaridades das funções em questão e como elas se associam à esquerda com alta prioridade. Para entender `funcMaybe`, devemos entender a instância de `Monad` para `Maybe` já implementada na linguagem.

```
instance Monad Maybe where
  (Just x) >=> f = f x
  Nothing >=> f = Nothing
```

TOME CUIDADO!

No Haskell moderno (a partir do GHC 8.4), a classe `Monad` não vive sozinha: ela tem > a `Applicative` como superclasse, que por sua vez tem a `Functor` como superclasse. > Isto é, **todo `Monad` é também um `Applicative` e um `Functor`** . Por isso, para > transformar um tipo em Mônada, é preciso fornecer as três instâncias. Para o `Maybe` , a tríade completa é:

```
instance Functor Maybe where
    fmap f (Just x) = Just (f x)
    fmap _ Nothing  = Nothing

instance Applicative Maybe where
    pure x = Just x
    (Just f) <*> (Just x) = Just (f x)
    _ <*> _ = Nothing

instance Monad Maybe where
    (Just x) >>= f = f x
    Nothing >>= f = Nothing
```

O antigo `return` continua existindo, mas agora ele é apenas um *apelido* de `pure` (seu valor padrão é `return = pure`). Em código novo, prefira `pure` . É comum encontrar em livros e exemplos antigos a definição `return x = Just x` dentro do `instance Monad` ; hoje esse papel pertence ao `pure` , na instância de `Applicative` .

Observe que `(Just x) >>= f = f x` ignora o primeiro `Just` e faz com que `x` seja a entrada da função `f` , pois ambas possuem o mesmo tipo `x :: a` e `f :: a -> m b` . Voltando a

`funcMaybe` , o primeiro cálculo é `return x >=> \y -> return (reverse y)` , que se traduz em `Just "Haskell" >=> \y -> Just (reverse y)` .

Como visto na definição de `Monad` para `Maybe` , temos que "Haskell" vira argumento do *lambda* `\y -> Just (reverse y)` , logo `y = "Haskell"` nos dando `Just "lleksaH"` .

No próximo `Just "lleksaH" >=> \z -> return (take 3 z)` , teremos o retorno `z = "lleksaH"` , portanto, o resultado `Just "lle"` . A expressão continua com `Just "lle" >=> return (tail w)` , assim temos `w = "lle"` e retornamos `Just "le"` .

Por fim, temos `Just "le" >=> \u -> return (x ++ u)` com `x = "Haskell"` e `u = le` , retornando, finalmente, `Just "Haskellle"` . Lembre-se de que tudo isso foi graças à alta prioridade à esquerda, que `>=>` possui. Graças a isso, podemos transformar expressões como as que vimos em `funcMaybe` na notação do .

```
funcMaybe :: String -> Maybe String
funcMaybe x = do
  y <- return x
  z <- return (reverse y)
  w <- return (take 3 z)
  u <- return (tail w)
  return (x ++ u)
```

Esta notação transforma a escrita funcional, que estávamos habituados, em uma escrita parecida com a das linguagens de programação imperativas! Dê uma olhada: `<-` lembra a atribuição, ausente aqui graças a um truque com os *lambdas*. Tendo isso em mente, quando vemos `return x >=> \y -> return (reverse y) >=> \z ...` , é só ignorarmos `->` e

transformar `>>=` em `<-` , e pronto! Podemos de agora em diante escrever de uma maneira "imperativa" aqui no Haskell, sem nos preocuparmos com *side-effects* ou valores impuros.

Algo a ser frisado: se o retorno de uma função não for monádico (algo como `m a`), então sempre devemos chamar a função `return` ou a notação `do` quando usarmos `>>=` . Por isso, em `funcMaybe` , as expressões `x` , `reverse y` , `take 3 z` , `tail w` e `x ++ u` são argumentos de `return` (nenhuma possui algum retorno monádico).

Outra coisa: nunca confunda o `return` do Haskell com o do Java, por exemplo. São duas coisas completamente diferentes.

8.4 A MÔNADA []

Finalmente, vamos mostrar a instância de `Monad` para listas (já implementadas na linguagem).

```
instance Applicative [] where
    pure x      = [x]
    fs <*> xs = [f x | f <- fs, x <- xs]    -- visto no capítulo 7

instance Monad [] where
    xs >>= f = [y | x <- xs, y <- f x]
```

Seguindo a hierarquia moderna que vimos na seção sobre o `Maybe` , o `pure` (da instância de `Applicative`) simplesmente coloca um elemento `x` na lista, enquanto `>>=` (da instância de `Monad`) pega todos os valores `x<-xs` da lista e coloca-os como entrada de `f` . Veja um exemplo:

```
Prelude Control.Monad> [2,3] >>= \x -> [x+1]
[3,4]
```

Caso a função tenha várias expressões em cada posição, os valores de entrada serão os mesmos. Observe:

```
Prelude Control.Monad> [2,3] >= \x -> [x+1,x-1]
[3,1,4,2]
```

O valor de retorno `[3,1,4,2]` representa o cálculo de `[2+1,2-1,3+1,3-1]`. Ou seja, a função `\x -> [x+1,x-1]` é aplicada inteira ao primeiro elemento (`x=2`, gerando `[3,1]`) e só depois ao segundo (`x=3`, gerando `[4,2]`), e as listas resultantes são concatenadas na ordem. Como listas são Mônadas, a notação `do` pode ser usufruída também seguindo as mesmas regras vistas na seção anterior.

```
expr :: [Int]
expr = do
  x <- [2,3]
  [x+1,x-1]
```

8.5 EXERCÍCIOS

8.1) Faça um tipo `Caixa` com um *type parameter* `a` e três construtores chamados `Um`, `Dois` e `Tres` possuindo um, dois e três campos de tipo `a`, respectivamente.

- Faça uma instância de `Functor` para o tipo `Caixa`. A função deve ser aplicada em todas as coordenadas dos valores (`Um`, `Dois` ou `Tres`).
- Crie uma instância de `Monad` para o tipo `Caixa`. Seu `return` deve ser o *value constructor* de `Um`.

Observação: quando definir `>=` para `Caixa`, o valor `a` entrar em `f` segue as regras:

- Um : o único campo entra na função (análogo ao Maybe);
- Dois : o segundo campo entra;
- Tres : o terceiro campo entra.

8.2) Crie uma função `mult234 :: Double -> Caixa Double` que receba um parâmetro `x` e devolva o dobro de `x` na primeira coordenada, o triplo na segunda e o quádruplo na terceira usando o operador `>=>` .

8.3) Determine o valor das expressões a seguir (caso seja possível), sem usar o GHCi:

- `Tres 1 2 3 >=> mult234 >=> mult234`
- `Dois 2 4 >=> mult234`
- `:kind Caixa`
- `Dois 2 3 >=> \_ -> Dois 7 7`

8.4) Faça um exemplo, usando a notação `do` , de um trecho qualquer de código usando sua `Monad Caixa` .

8.5) Lembre-se de que, no Haskell moderno, todo `Monad` é também `Applicative` e `Functor` . Escreva as **três** instâncias (`Functor` , `Applicative` e `Monad`) para o tipo `Caixa` do exercício 8.1, definindo `pure` na instância de `Applicative` (e não mais `return`).

8.6) Determine, sem usar o GHCi, o valor de `[10,20] >=> \x -> [x, x+1]` . Em seguida, confira no GHCi. A ordem do resultado bate com a sua intuição? Explique com base na instância de `Monad` para listas.

8.6 CONCLUSÃO

Neste capítulo finalizamos o ferramental teórico necessário para programar em Haskell em plena produtividade. O conceito de Mônada é fundamental para ir além na linguagem e fazer aplicações reais.

Vimos aqui os conceitos de transformações naturais, Mônadas e sua tríade de funções. Por último, vimos o importantíssimo `>=>` e os benefícios de sua existência, dando origem ao uso da notação `do` que permite ao programador sentir-se em um ambiente similar ao encontrado nas linguagens imperativas.

No próximo capítulo, veremos apenas a parte prática com a Mônada `IO` e como ler dados da entrada padrão e leitura e escrita em arquivos de texto. A leitura e compreensão deste capítulo situam o leitor em uma posição quase avançada na linguagem, e permitirão uma maior tranquilidade quando o leitor se deparar com aplicações web e banco de dados.

MÔNADA IO

Neste capítulo, apresentaremos a Mônada `IO`, talvez a mais importante da linguagem, pois é ela que realiza a conexão da linguagem Haskell com o complicado "mundo real". Esta Mônada é responsável pela leitura e escrita de arquivos e pela conexão com a entrada padrão, por exemplo.

Os *frameworks* web e as ferramentas de persistência da linguagem Haskell são consequência dessa Mônada. Este capítulo será totalmente prático, pois a teoria para entender o que há por trás da Mônada `IO` foi construída previamente.

O tipo `IO` possui *kind* `* -> *` e instâncias para as importantes classes `Monad`, `Functor` e `Applicative`. Convém lembrar que não é permitido ao programador retirar o valor de dentro de `IO`.

Por exemplo, se o número `5 Int` for lido do teclado, este terá o tipo `IO Int`, implicando que não podemos somar diretamente este valor lido do teclado com um valor `7` fixo e puro (sem ser lido de fonte externa), ou até mesmo somá-lo a outro valor lido pelo teclado. O rótulo `IO` impedirá essas manipulações de dados e manterá um estado imutável por toda a aplicação.

9.1 COMPILANDO UM PROGRAMA "OLÁ MUNDO"

Graças à Mônada `IO`, finalmente será possível compilar um programa "Olá Mundo". Basta nomearmos o módulo como `Main` e definirmos a função `main` que terá o tipo `IO ()`, análogo ao `void` do Java.

O tipo `()` é algo com um *value constructor* apenas chamado `()`, algo como `data () = ()`. Ele é importante para representar funções "sem retorno", por exemplo, imprimir um caractere na tela.

Com isso em mente, podemos escrever nosso primeiro programa (até agora usamos um REPL pelo GHCi). Precisamos criar um arquivo chamado `Principal.hs` e, em seguida, escrever a função `main`, como vemos no exemplo a seguir.

```
module Main where

main :: IO ()
main = putStrLn "Olá Mundo!"
```

A função `main` representa o início do programa, e é ela que o compilador procurará para gerar o executável. Para compilar, basta digitar em um terminal a linha:

```
$ ghc Principal.hs -o Principal
```

Então, aparecerá a seguinte mensagem:

```
[1 of 1] Compiling Main                ( Principal.hs, Principal.o )
Linking Principal ...
```

E para executar, digite no terminal (usando aqui *bash*):

```
$ ./Principal
```

Olá Mundo!

Pronto, nosso primeiro programa está concluído. Note que é bem simples a escrita dele, porém a teoria foi importante para entender o que há por trás do `IO`.

A função `putStrLn` possui o tipo `String -> IO ()`, recebe um valor `String` e imprime seus caracteres na tela retornando `()` para simular o `void`. Ou seja, nenhum valor útil será retornado após a execução da função.

Essa função imprime caracteres linha a linha. Se quiséssemos outra linha, poderíamos nos beneficiar da notação `do`, vista no capítulo anterior.

```
module Main where

main :: IO ()
main = do
    putStrLn "Olá Mundo!"
    putStrLn "Haskell"
```

Após os passos de compilação, teríamos duas linhas de impressão. Vale notar que podemos escrever essa versão do `main` em termos do `bind` (`>>=`), mais precisamente, usando `>>` que é a versão do `bind` para funções que retornam `()`. Algo do tipo `valor >>= \_ -> função`.

Note que o valor pode ser oriundo de uma computação sem retorno e, assim, é ignorado pelo *pattern matching* `_` na entrada da função. Desta forma, a expressão `valor >>= \_ -> função` pode ser reescrita como `valor >> função`, imitando o sequenciamento visto no quadro anterior. Reescrevendo essa função, obteremos:

```
module Main where
```

```
main :: IO ()
main = putStrLn "Olá Mundo!" >> putStrLn "Haskell"
```

Apesar de `>>` ser amplamente usado, manteremos a preferência pela notação `do` .

9.2 EXEMPLOS PRÁTICOS

Nesta seção, estudaremos a fundo as funções `putStrLn` , `print` , `readLn` e `getLine` , sendo as duas primeiras de impressão na tela e as duas últimas para leitura da entrada padrão.

A função `putStrLn` só funciona com `String` , enquanto `print` funciona com qualquer tipo que seja instância de `Show` (como visto no *capítulo 5*).

```
Prelude> :t putStrLn
putStrLn :: String -> IO ()
Prelude> :t print
print :: Show a => a -> IO ()
```

O `getLine` lê uma `String` da entrada padrão, enquanto `readLn` é mais geral, executando a conversão de uma `String` para o tipo desejado, desde que este tipo seja instância de `Read` .

```
Prelude> :t getLine
getLine :: IO String
```

```
Prelude> :t readLn
readLn :: Read a => IO a
```

Em casos de ambiguidade, teremos de forçar a conversão para o tipo em questão usando `:: IO Tipo` . Por exemplo, `readLn :: IO Int` indica a leitura de algo a ser convertido para inteiro. O cuidado que devemos ter é que se precisarmos ler um inteiro e

outro tipo é encontrado, como `Char`, teremos problema no `read`, causando uma exceção **Prelude.readIO: no parse**.

Agora mostraremos vários exemplos de programas feitos em Haskell para que o leitor se acostume com a escrita da notação `do` e também aprenda a usar a Mônada `IO`.

Para todos os exemplos, lembre-se da primeira linha `module Main where`. Você pode compilar os programas um a um, ou renomear todas as funções e usá-las no GHCi (este executa funções de IO sem nenhum problema).

Exemplo 1: soma de dois números

```
main :: IO ()
main = do
    putStrLn "Digite um número: "
    x <- readLn
    putStrLn "Digite outro número: "
    y <- readLn
    putStrLn "Resultado: "
    print $ x + y
```

Note que a função `putStrLn` exibe na tela apenas `String`, enquanto o `print` exibe a soma `x + y`, pois toda instância de `Num` possui instância para `Show`. Se quiséssemos ler `x` e `y` como `Double`, por exemplo, poderíamos usar `readLn :: IO Double`, como mencionado anteriormente.

Exemplo 2: loop while funcional

```
main :: IO ()
main = do
    let loop = do
        putStrLn "Qual seu nome?"
        nome <- getLine
        if (nome == "") then do
```

```

        putStrLn "Erro... "
    loop
else
    putStrLn $ "Olá " ++ nome
loop
putStrLn "Fim"

```

O programa anterior executa um `loop` , similar ao `while` , de uma forma funcional utilizando o conceito de recursão. Observe que, dentro da função `main` , definimos a função auxiliar `loop` (similar ao `where` , a cláusula `let` permite-nos definir funções localmente, antes de serem usadas).

A função `loop` imprime na tela a mensagem "Qual seu nome?" , e pede ao usuário que digite uma `String` lida pelo `getLine` e associada a `nome` . A função `if then else` é similar às estruturas condicionais de uma linguagem imperativa. Se o `nome` for vazio, será impresso na tela uma mensagem de erro e a função `loop` será chamada novamente (recursão); caso contrário, será mostrado um cumprimento de Olá, seguido do valor em `nome` .

A cláusula `let` apenas definiu a função `loop` localmente. Como dito anteriormente, a função só é chamada na linha 11 com a instrução `loop` , fazendo a primeira chamada dela. Após o `loop` ser completado, a mensagem "Fim" é exibida. Esse exemplo realça a semelhança da notação `do` com as linguagens imperativas.

Exemplo 3: loop for funcional

```

import Control.Monad

main :: IO ()
main = do

```

```

z <- readLn
form_ [1..z] $ \i -> do
    print i

```

Nesse exemplo, um inteiro `z` será lido na entrada padrão, e ele será o limite máximo para a expressão `[1..z]`, que criará uma lista de números de 1 até `z`.

A função `form_` repetirá a execução do *lambda* `\i -> print i` por um total de `z` vezes. Nesse caso, a variável de entrada do *lambda* `i` receberá cada valor da lista `[1..z]`, primeiramente 1, depois 2 e assim por diante. Para cada valor da lista, esse será impresso na tela conforme a expressão `print i`. Esse exemplo mostra na tela cada elemento da lista em linhas diferentes.

Exemplo 4: for funcional

```

import Control.Monad

main :: IO ()
main = do
    z <- readLn
    let dentro i = do
        putStrLn $ "Número " ++ (show i)
        readLn
    ns <- mapM dentro [1..z]
    putStrLn $ "Resultado: " ++ (show $ sum ns)

```

Nesse programa, é mostrada na tela, em cada linha, cada elemento da lista `[1..z]` (números de 1 a `z` de um em um), em que `z` foi informado pelo usuário. Ao final, é mostrada a soma dos números.

A função `mapM :: Monad m => (a -> m b) -> [a] -> m [b]` joga uma função de ação monádica de tipo `a -> m b` dentro de uma lista de tipo `[a]`, retornando uma lista de tipo `[b]`

dentro da Mônada `m` , especificada na função passada como parâmetro.

O intuito é o mesmo do `map` , porém com uma Mônada envolvida. A cláusula `let` define uma função local de nome `dentro` que recebe um `Int` e executa as ações da Mônada `IO` : mostrando cada número na saída padrão e lendo um número da entrada padrão. Assim, teríamos algo parecido com tipo `Int -> IO Int` , entrada `i` de tipo `Int` , e saída `IO Int` por causa do `readLn` .

A função `mapM` faz com que a função `dentro` seja chamada, tendo como argumento todos os elementos da lista `[1..z]` e, ao final, mostra na saída a soma dos números digitados.

```
import Control.Monad

main :: IO ()
main = do
  z <- readLn
  ns <- forM [1..z] $ \i -> do
    putStrLn $ "Numero " ++ show i
    readLn
  putStrLn $ "Resultado: " ++ (show $ sum ns)
```

Aqui é feita a mesma ação, porém escrita de outra forma. Neste caso, temos algo mais próximo das linguagens imperativas (lembra mais ou menos o `for` do Python). Note que a função `forM` é a mesma que `mapM` , porém as duas possuem os parâmetros trocados de posição. Vamos inspecionar os tipos no GHCi:

```
Prelude> :t mapM
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
Prelude> :t forM
forM :: Monad m => [a] -> (a -> m b) -> m [b]
```

Então, vemos a troca de posições dos parâmetros. Isso é recorrente na linguagem Haskell e, dependendo da posição dos argumentos, podemos escrever uma função de várias formas (lembremos de `$` e `|>` do *capítulo 4*). Há uma função no Haskell, chamada `flip`, que troca dois parâmetros de posição. Um leitor perspicaz identificará que `form = flip mapM` e que `|> = flip ($)`.

Exemplo 5: adivinhando uma carta do baralho

Vamos construir um jogo simples de cartas feito em programação funcional pura. Esse exemplo nos mostra que jogos também podem ser desenvolvidos nesse tipo de paradigma. Nele mostraremos o uso de números aleatórios usando o pacote `random`.

O exemplo a seguir usa a função `randomRIO`, do módulo `System.Random`, que vive no pacote `random` (ele não vem junto com o compilador). A forma moderna de disponibilizá-lo é criar um projeto com `cabal init` e adicionar `random` à lista de `build-depends` do arquivo `.cabal` (veremos esse fluxo em detalhes no capítulo sobre `Servant`). Para um teste rápido no `GHCi`, também funciona instalá-lo no ambiente global com `cabal install --lib random`.

```
import System.Random (randomRIO)

data Naipe = Ouros | Espadas | Copas | Paus deriving (Show, Enum,
Read, Eq, Bounded)

data Valor = Dois | Tres | Quatro | Cinco | Seis |
Sete | Oito | Nove | Dez | J | Q | K | A
deriving (Show, Enum, Read, Eq, Bounded)

data Carta = Carta {valor :: Valor, naipe :: Naipe} deriving (Sho
```

```

, Read, Eq)

main :: IO ()
main = do
    let acertou True = "Você acertou"
        acertou False = "Errou..."
    let baralho = [Carta x y | x<-[Dois .. A], y<-[Ouros .. Paus]
]
    cartaNum <- randomRIO (0, length baralho - 1)
    let carta = baralho !! cartaNum
    putStrLn "Escreva a carta para adivinhar: "
    palpite <- readLn
    putStrLn $ "Sua carta foi " ++ show (valor carta) ++ " de " +
+ show (naipe carta)
    putStrLn $ acertou $ carta == palpite

```

Repare nas cláusulas `deriving` : além de `Show` e `Enum` , agora pedimos `Eq` (para comparar a carta sorteada com o palpite usando `==`) e `Read` (para que o `readLn` consiga converter o que o usuário digitou em um valor do tipo `Carta`). Note ainda que `baralho` e `carta` são valores puros, então usamos `let` em vez de `return` — só `cartaNum` e `palpite` vêm de ações de `IO` . O índice aleatório vai de `0` até `length baralho - 1` , pois a função `!!` começa a contar do zero.

Primeiramente, definimos a função `acertou` , localmente com o `let` , de tipo `Bool -> String` , usando *pattern matching* no parâmetro booleano. A `[Carta x y | x<-[Dois .. A], y<-[Ouros .. Paus]]` cria uma lista com as 52 cartas de um baralho.

Perceba que a expressão `Carta x y` representa o tipo `Carta` a ser montado com o `x` (tipo `Valor`) e o `y` (tipo `Naipe`). O `x` varia entre todos os *value constructors* de `Valor` , e o `y` , entre os de `Naipe` . Lembre-se de que isto é possível graças ao `..` e à instância de `Enum` (o operador `..` depende da função `succ` definida em uma instância de `Enum`).

A lista montada ficaria da forma [Carta Dois Ouros, Carta Dois Espadas, Carta Dois Copas, Carta Dois Paus, Carta Tres Ouros, ... , Carta A Paus] . Como baralho é um valor puro (não vem de uma ação de IO), usamos let .

Um índice aleatório entre 0 e 51 (length baralho - 1) é gerado a partir de randomRIO (0, length baralho - 1) . A função randomRIO é do módulo System.Random e recebe uma tupla com os limites mínimo e máximo para gerar um número aleatório. Note que esta ação é monádica (IO), por isso usamos <- aqui.

A expressão let carta = baralho !! cartaNum seleciona, com esse índice, uma carta do baralho sob o nome de carta (novamente um valor puro, daí o let).

O usuário deve escrever a carta usando a *record syntax*, e a instância de Read dos três tipos nos garante isso. Mas o leitor pode optar por fazer o tipo Carta sem o *record syntax* para evitar isso. Basta usar a linha data Carta = Carta Valor Naipe deriving Show no lugar do tipo Carta definido no trecho anterior. O programa mostrará a carta escolhida usando a instância de Show dos três tipos, e verificará se são iguais graças ao Eq dos três tipos. Veja um exemplo de execução:

```
Escreva a carta para adivinhar:
Carta {valor = A, naipe = Ouros}
Sua carta foi Sete de Espadas
Errou...
```

Note como o usuário teve de escrever a carta. A função read da classe Read , em caso de existência de *record syntax*, lerá a entrada apenas desta forma.

9.3 MANIPULANDO ARQUIVOS

Vamos estudar três funções básicas para manipulação de arquivos:

```
readFile :: FilePath -> IO String
writeFile :: FilePath -> String -> IO ()
appendFile :: FilePath -> String -> IO ()
```

Observe que o tipo `FilePath` é um sinônimo para `String` (`type FilePath = String`). Para ler arquivos, usamos a função `readFile` ; para escrever um novo arquivo, `writeFile` ; e para escrever ao fim de um arquivo, o `appendFile` . Assim como na seção anterior, vamos aprender a manipular tais funções por meio de alguns exemplos.

Exemplo 6: salário total e maior de uma lista de funcionários

Vamos supor que precisamos calcular os salários a partir de uma lista de funcionários de uma empresa, dispostos em um arquivo texto. Para isto, precisamos do arquivo `func.dat` que deverá possuir o formato:

```
Func1 3000.00
Func2 2300.00
Func3 9999.99
Func4 23000.00
```

Para realizar este cálculo, considere o trecho:

```
import Text.Printf

main :: IO ()
main = do
    lista <- fmap (map words . lines) $ readFile "func.dat"
    salarios <- return $ map \(_:v1:_) -> read v1) lista :: IO [
```

```
Double]
    printf "%.2f\n" $ sum salarios
    print $ maximum salarios
```

Observe que, na primeira linha após o `do`, temos a leitura do arquivo a partir da expressão `readFile "func.dat"`. Tal expressão retorna em um `IO String` representando todo o arquivo e, por isso, há um `fmap` que joga "para dentro" do *functor* `IO` (note que estamos usando `IO` como *functor* aqui) a função composta de duas (`map words . lines`).

A função `lines` quebra a `String` toda do arquivo pelo separador `\n`, retornando em um `[String]` cada linha do arquivo. Por exemplo, `Func1 3000.00\nFunc2 2300.00` é transformado em `["Func1 3000.00", "Func2 2300.00"]`.

A função `map words` aplica a função `words` a cada elemento da lista acima, dividindo cada `String` pelo separador `" "`. Por exemplo, `"Func1 3000.00"` torna-se `["Func1", "3000.00"]`. Assim, `lista` passa a ser `[["Func1", "3000.00"], ["Func2", "2300.00"], ["Func3", "9999.99"], ["Func4", "23000.00"]]`, considerando que o arquivo lido seja o do exemplo anterior.

A segunda linha extrai o segundo campo das listas anteriores, restando só os valores que são `String` e devem ser convertidos em números. A expressão `map (\(_:v1:_) -> read v1)` lista faz com que o *lambda* `\(_:v1:_) -> read v1` seja jogado "para dentro" de uma lista de formato análogo ao de cima. Temos então que este *lambda* extrai a segunda posição via ***pattern matching*** `_:v1:_`, e converte para `Double` usando a função `read`.

Como o `map` trabalha fora do `IO`, devemos colocar a lista de salários dentro do `IO` usando o `return`. Como vimos no capítulo anterior, ou a ação é monádica ou devemos colocar um `return` para torná-la monádica.

O `:: IO [Double]` indica que o tipo que o `read` deve converter a `String` é `Double`. O `printf "%.2f\n"` faz o truncamento de duas casas decimais (esta função está no módulo `Text.Printf`). A soma e o máximo são realizados pelas funções `sum` e `maximum`.

Exemplo 7: uso do `write` e `append`

Vejam a diferença entre `write` e `append`, além de usar a função `doesFileExist`, que verifica se um arquivo existe ou não. O exemplo mostra como gravar uma `String` em um arquivo.

```
import System.Directory

main :: IO ()
main = do
  putStrLn "Digite o nome do arquivo. Será criado caso não exista"
  arq <- getLine
  putStrLn "Digite uma mensagem"
  mensagem <- getLine
  existe <- doesFileExist arq
  if existe then
    appendFile arq ("\n" ++ mensagem)
  else
    writeFile arq mensagem
```

O programa tem o intuito de verificar se um arquivo existe ou não. Se existir, a mensagem digitada será escrita em uma nova linha usando `appendFile`; se não existir, será criado um novo `writeFile`.

Note que `arq` se refere ao nome do arquivo em questão e `mensagem` ao conteúdo a ser salvo. A função `doesFileExist`, do módulo `System.Directory`, retorna um `IO Bool` (por isso que não foi direto no `if`), indicando se o arquivo cujo nome está em `arq` existe ou não. Para a mensagem ir para a linha subsequente, em um arquivo existente, usamos a expressão `"\n" ++ mensagem`.

Os dois exemplos dados ilustram como manipular arquivos físicos na linguagem Haskell, e ressaltam a estrutura de Mônada presente no tipo `IO`, de *kind* `* -> *`.

9.4 MINIPROJETO FINAL

Chegou o momento de aplicar tudo o que aprendemos para a conclusão do miniprojeto. Completaremos o sistema de RH que começou nos capítulos anteriores com as funcionalidades de buscas e cadastro de funcionários e projetos através da Mônada `IO`. Com a Mônada `IO`, podemos realizar buscas e cadastrar funcionários/projetos para nosso pequeno sistema de RH. Vamos mostrar o código aos poucos para facilitar o seu entendimento.

O código completo pode ser apreciado no apêndice. O leitor é encorajado a implementar as rotinas introduzidas nos capítulos anteriores.

Para a conclusão do sistema, começemos com a redefinição das instâncias dos tipos usados e com uma instância da classe `ToJSON`, criada no *capítulo 5*, para o tipo `Indice` criado no *capítulo 7*.

```
{-# LANGUAGE BangPatterns #-}
```

```

module Main where

import System.Directory
import Control.Monad
import Control.Applicative

data Cargo = Estagiario | Programador | Coordenador | Gerente deriving (Show, Read)

data Pessoa = Pessoa {cargo :: Cargo, nome :: String} deriving (Show, Read)

data Indice a = Indice {indice :: Int, dados :: a} deriving (Show, Read)

data Projeto = Projeto {nomeProjeto :: String,
                        budget :: Double,
                        envolvidos :: [Int]} deriving Show

instance ToJSON a => ToJSON (Indice a) where
    toJSON p = "{id: " ++ show (indice p) ++ ", dados: " ++ (toJSON $ dados p) ++ "}"

```

Precisamos estabelecer uma execução de código não preguiçosa, ou seja, não há uma computação sob demanda e ela deve ser executada com urgência. Essa extensão, vista na primeira linha com o pragma `{-# LANGUAGE BangPatterns #-}`, permite o uso de um `!` (exclamação) na frente de um parâmetro usado dentro de uma função. Se não usarmos o `!`, a computação sob demanda vai travar o arquivo `func.dat` por causa da preguiça, pois pode acontecer uma tentativa de leitura e de escrita ao mesmo tempo.

Repare que os tipos possuem agora instâncias de `Read` para serem usadas nas buscas. A instância de `ToJSON` para `Indice` possui uma restrição em `a`. Lembrando de que vimos no *capítulo 5* que não é todo `Indice a` que poderá ser instância de `ToJSON`, apenas os `a`s que forem instâncias de `ToJSON`. Vamos introduzir

agora a função `main` que representa o fluxo principal do programa.

```
main :: IO ()
main = do
    let projetos = "projetos.dat"
        pessoas = "func.dat"
    existeProj <- doesFileExist projetos
    existeFunc <- doesFileExist pessoas
    when (not existeProj) (writeFile projetos "")
    when (not existeFunc) (writeFile pessoas "")
    opcao <- menu
    case opcao of
        1 -> cadastroPessoa
        3 -> buscaPessoas
        6 -> todosSalarios
        7 -> exportarPessoas
        _ -> putStrLn "Opção ainda não implementada"
```

Primeiramente, vamos analisar a função principal `main`, pois ela é a mais modularizada possível e vai nortear o entendimento do código como um todo. A função começa com a cláusula `let`, que define duas `Strings` para representar o nome dos arquivos a serem usados. O primeiro será para os projetos e o último para os funcionários.

As quatro próximas linhas verificarão se existem ou não os arquivos. Se existirem, não faremos nada; caso contrário, os arquivos `projetos.dat` e `pessoas.dat` serão criados.

A função `when` é um `if` sem `else`, e seu primeiro argumento é um `Bool` que indica se a ação monádica de criação de arquivo `writeFile` será executada ou não. A expressão `opcao <- menu` indica a chamada da função `menu :: IO Int` que cria nosso menu e retorna a opção correta digitada pelo usuário.

O uso do `case` é análogo a qualquer outra linguagem: são testados os valores de `opcao` até encontrar o verdadeiro. A vantagem do `case` aqui no Haskell é que podemos usar o *pattern matching*, como por exemplo, usar a opção de ignorar `_`.

Cada opção de `case` chamará uma função de tipo `IO ()` para que todas tenham o mesmo tipo de `main`. Vamos analisar agora as funções `menu` e as que são chamadas no `case`.

```
menu :: IO Int
menu = do
    putStrLn "Lambda Systems v1.0"
    putStrLn "Digite uma opção... "
    putStrLn "1- Cadastro de Pessoas"
    putStrLn "2- Cadastro de Projetos"
    putStrLn "3- Busca de Pessoas"
    putStrLn "4- Busca de Projetos"
    putStrLn "5- Total gasto por projeto"
    putStrLn "6- Ver salários"
    putStrLn "7- Exportar Pessoas (JSON)"
    putStrLn "8- Exportar Projetos (JSON)"
    opcao <- readLn
    if opcao < 0 || opcao > 8 then do
        putStrLn "Opção inválida"
        menu
    else
        return opcao
```

A função `menu` segue a mesma abordagem da função `loop`, apresentada na seção de exemplos, utilizando recursão para tratar opções inválidas. Observe que seu tipo de retorno é `IO Int`, definido tanto pela chamada recursiva a `menu`, no ramo verdadeiro do `if`, quanto por `return opcao`, que encapsula o inteiro digitado na Mônada `IO`.

Cadastro de pessoas

Aqui será apresentado o cadastro de pessoas como previsto na função `main`. Essa função lerá dados do teclado e os salvará em um arquivo texto.

```
cadastroPessoa :: IO ()
cadastroPessoa = do
    !pessId <- fmap (length . lines) (readFile "func.dat")
    pess <- Pessoa <$> (putStrLn "Cargo: " >> readLn) <*> (putStrLn
Ln "Nome: " >> getLine)
    appendFile "func.dat" (show (Indice (1+pessId) pess) ++ "\n")
    putStrLn "Usuário cadastrado com sucesso!"
```

Para contar quantas pessoas foram cadastradas, de modo a incrementar o índice, usamos a expressão `fmap (length . lines) (readFile "func.dat")`. Ela joga a função `length . lines` para dentro de um `IO String` representado pelo `readFile`. Note que a função de cadastro de pessoas começa com um parâmetro de *lambda* com nome `!pessId`, que indica o uso do `BangPatterns` visto anteriormente.

A função `lines :: String -> [String]` quebra uma `String` em várias usando o separador `\n`, e o `length` dá o tamanho desta lista. Logo, esta linha conta os registros.

Precisamos montar o tipo `Pessoa` para salvarmos no arquivo. Esse dado fará parte do tipo `Indice`, de modo a termos como recuperá-lo de volta a partir de uma busca. Para realizar essa tarefa, usamos a expressão `pess <- Pessoa <$> (putStrLn "Cargo: " >> readLn) <*> (putStrLn "Nome: " >> getLine)`.

Utilizamos o `Applicative` para colocar `Pessoa` dentro da `Mônada IO`, como explicado no fim da seção sobre *funtores aplicativos*. As expressões `putStrLn "Cargo: " >> readLn` (que

é um `IO Cargo`) e `putStrLn "Nome: " >> getLine` (um `IO String`) entram como argumentos na função `Pessoa :: Cargo -> String -> Pessoa` (como visto no *capítulo 3*).

Usamos apenas `<$>` e `<*>`, não havendo necessidade de outra conversão. Lembre-se de que esse truque foi explicado usando `Form` em vez de `IO`, que no fundo são "farinha do mesmo saco".

Na linha `appendFile "func.dat" (show (Indice (1+peSSId) pess) ++ "\n")`, escrevemos no arquivo `func.dat` o `Indice`, contendo um autoincremento, e a `Pessoa`, aproveitando a instância de `Show` destes tipos. O `"\n"` ao final indica quebra de linha. A função termina com uma mensagem de sucesso.

Cadastro de projetos

O código a seguir mostra como cadastrar projetos lidos da entrada padrão, em um arquivo texto.

```
cadastroProjeto :: IO ()
cadastroProjeto = do
    !projId <- fmap (length . lines) (readFile "projetos.dat")
    proj <- Projeto <$> (putStrLn "Nome: " >> getLine) <*> (putStrLn
"Orçamento: " >> readLn) <*> (putStrLn "Envolvidos: " >> readLn)
    appendFile "projetos.dat" (show (Indice (1+projId) proj) ++ "\n")
    putStrLn "Projeto cadastrado com sucesso!"
```

Essa função é análoga ao cadastro de pessoas. O seu ponto fraco é que o usuário terá de digitar o índice das pessoas envolvidas, como `[3,4,9]` indica que o projeto possui os funcionários de índice 3, 4 e 9.

Buscas

A função a seguir funciona para buscar tanto pessoas como projetos. Ela foi feita para evitar a criação de duas funções idênticas (a menos de nomes dos *lambdas*) e deixar o programa o mais modularizado possível.

```
buscaPorId :: (Read a, Show a) => Int -> String -> IO (Maybe (Indice a))
buscaPorId tid arq = do
    todosReg <- fmap lines (readFile arq)
    todosConv <- return $ map read todosReg
    res <- return $ filter (\ips -> indice ips == tid) todosConv
    case res of
        [] -> return Nothing
        _ -> return $ Just (head res)
```

Essa função busca qualquer pessoa ou projeto por índice. Ela tem como parâmetro o `id` inteiro do projeto ou pessoa, e uma `String arq` representando o arquivo. O arquivo `arq` é lido e transformado em uma lista de `String` na expressão `todosReg <- fmap lines (readFile arq)`. Convertemos também todas as `String` para o tipo `Indice a` (qualquer `a` instância de `Show` e `Read`, pois o tipo da função é `(Read a, Show a) => Int -> String -> IO (Maybe (Indice a))`) como visto na expressão `todasPess <- return $ map read todasPessoas`.

Note que é jogado o `read` para dentro da lista, de `String`, `todosReg`, nos dando uma lista de `Indice a` chamada `todosConv`. Ao final, isso é colocado dentro da Mônada `IO` pelo `return` (note que `map read todosReg` não é da Mônada `IO` e necessita de `return`).

Filtramos da lista apenas o `Indice a` pelo inteiro `tid`, a partir de `res <- return $ filter (\ips -> indice ips ==`

tid) todosConv . Na expressão de filtragem, o *lambda* (\ips -> indice ips == tid) expressa a igualdade entre o índice do registro e o passado por parâmetro.

O `case` é usado para fazer *pattern matching* em `res` . Caso seja `[]` (vazio), retornará `Nothing` ; caso contrário, será retornado o registro desejado dentro de `Just` . Essa função busca apenas o registro, não mostrando para o usuário.

Gostaríamos de que o usuário pudesse fazer uma consulta no nosso arquivo por meio de um `id` . Para isto, precisamos de funções que pedem para ele digitar.

```
buscaPessoas :: IO ()
buscaPessoas = do
    putStrLn "Digite o id do funcionário"
    pessId <- readLn
    pess <- buscaPorId pessId "func.dat" :: IO (Maybe (Indice Pessoa))
    case pess of
        Nothing -> putStrLn "Erro..."
        Just p -> print p

buscaProjetos :: IO ()
buscaProjetos = do
    putStrLn "Digite o id do projeto"
    projId <- readLn
    proj <- buscaPorId projId "projetos.dat" :: IO (Maybe (Indice Projeto))
    case proj of
        Nothing -> putStrLn "Erro..."
        Just (Indice i (Projeto nm bd env)) -> do
            print $ "id :" ++ show i
            print $ "nome :" ++ nm
            print $ "orçamento :" ++ show bd
            forM_ env $ \envId -> do
                pess <- buscaPorId envId "func.dat" :: IO (Maybe (Indice Pessoa))
                case pess of
                    Nothing -> print "Pessoa inválida"
```

```
Just p -> print p
```

Essas funções são parecidas, porém a busca de projetos mostra todas as informações da pessoa simulando, assim como um `join` do SQL. Ambas as funções começam pedindo para o usuário digitar um `id`.

Após isto, a chamada da função `buscaPorId` é feita, como `buscaPorId projId "projetos.dat" :: IO (Maybe (Indice Projeto))` indica que queremos buscar um `Projeto`. Note que `:: IO (Maybe (Indice Projeto))` foi necessário pelo fato do tipo do retorno de `buscaPorId` ser `IO (Maybe (Indice Projeto))`, ou seja, forçamos que a seja `Projeto` aqui.

Como a busca pode não ser bem-sucedida, usamos o `case` para fazer *pattern matching* em `pess` ou `proj`. Caso o resultado seja `Nothing`, uma mensagem de erro será mostrada; caso seja `Just`, o dado de dentro dele será mostrado na tela.

No caso de `buscaProjetos`, usamos outro *pattern matching* dentro de `Just`, do qual obtemos o inteiro `i` do `Indice`, o nome do projeto `nm`, o orçamento `bd` e o envolvido `env`, que é um `[Int]`. Como ele é uma lista e queremos mostrar as pessoas a partir dos `id`s contidos, usaremos o `forM_` para jogar a ação monádica (de `IO`) para dentro dos `id`s.

```
\envId -> do
  pess <- buscaPorId envId "func.dat" :: IO (Maybe (Indice
Pessoa))
  case pess of
    Nothing -> print "Pessoa inválida"
    Just p -> print p
```

Logo, o *lambda* vai ter como entrada um `id` desta lista, e vai realizar a busca por `id` no arquivo de funcionários `pess <-`

`buscaPorId envId "func.dat" :: IO (Maybe (Indice Pessoa))` . E se achar, mostrará na tela; caso contrário, mostrará um erro.

Outras funcionalidades

Seria importante para o projeto uma funcionalidade para listar os salários dos funcionários e outra para exportar o nosso arquivo texto para outro formato, em nosso caso, o JSON .

```
todosSalarios :: IO ()
todosSalarios = do
    putStrLn "Listando salários..."
    todasPessoas <- fmap lines (readFile "func.dat")
    todasPess <- return $ map read todasPessoas :: IO [Indice Pessoa]
    forM_ todasPess $ \(Indice _ pessoa) ->
        print $ "Nome : " ++ (nome pessoa) ++ ", Salário: " ++ show (verSalario pessoa)

exportarPessoas :: IO ()
exportarPessoas = do
    writeFile "func.json" ""
    todasPessoas <- fmap lines (readFile "func.dat")
    todasPess <- return $ map read todasPessoas :: IO [Indice Pessoa]
    mapM_ (\x -> appendFile "func.json" (toJSON x ++ "\n")) todasPess
```

As funções `todosSalarios` e `exportarPessoas` são parecidas. No caso de exportar, primeiro criamos um arquivo `.json` . A busca é feita, assim como a conversão. Note que, nesse caso, não temos, intencionalmente, a generalização de `buscaPorId` , pois deixaremos a cargo do leitor como exercício fazer com que essas duas funções funcionem tanto para `Pessoa` quanto `Projeto` , assim como na função citada.

```
forM_ todasPess $ \(Indice _ pessoa) ->
```

```
print $ "Nome : " ++ (nome pessoa) ++ ", Salário: " ++ show (verSalario pessoa)
```

Em `todosSalarios`, a linha faz um *pattern matching* no tipo `Indice` para mostrar as pessoas na tela. Em `exportarPessoas`, o *lambda* `(\x -> appendFile "func.json" (toJSON x ++ "\n"))` representa uma inserção de registro em `func.json`, de acordo com a expressão `(toJSON x ++ "\n")` que converte `x` para JSON, e pula uma linha. Esse *lambda* será jogado na lista de `Indice Pessoa`, `todasPess`.

O leitor é convidado a implementar, por exemplo, a opção 5 e um cálculo da média salarial dos funcionários conforme a função apresentada anteriormente, como exercício prático. Aproveite os vários exemplos e ponha a mão na massa, só assim para você aprender com efetividade. O código completo do projeto estará disponível no apêndice.

9.5 EXERCÍCIOS

9.1) Faça um programa que faça o usuário digitar um número, e mostre na saída padrão se ele é par ou ímpar.

9.2) Faça um programa que mostre uma palavra em ordem reversa a partir de uma digitada pelo usuário.

9.3) Baseando-se no exemplo 5, faça um jogo de Pedra, Papel e Tesoura.

9.4) Faça um programa que calcule uma equação do segundo grau, a partir dos dados digitados pelo usuário.

9.5) Converta para o "estilo funcional" os 7 exemplos dados

neste capítulo. Basta escrever os trechos da notação do `>>=` (`bind`) e `>> .`

9.6) Modificando o exemplo 5, faça com que o programa mostre duas cartas aleatórias, retirando a adivinhação.

9.7) Faça um programa que peça para o usuário entrar com um número inteiro `n` e, a partir dele, o usuário deve digitar `n` linhas e estas devem ser gravadas em um arquivo.

9.8) Leia um arquivo que tenha o seguinte formato:

```
1 2
2 4
9 7
455 300
```

E verifique o maior número de cada linha e, ao final, o maior número entre todos. Estes números devem ser mostrados em um arquivo.

9.9) Uma senha é válida se há, pelo menos, 8 caracteres e, desses oito, pelo menos um deve ser numérico. Faça um programa que indique se a senha digitada pelo usuário é válida ou não.

9.10) A partir de um arquivo de senhas, mostre em outro arquivo apenas as senhas válidas e, em sua última linha, a quantidade delas.

9.11) Faça um programa que crie um arquivo contendo 6 números de 1 a 60, gerados aleatoriamente.

9.6 CONCLUSÃO

Neste capítulo, vimos como manipular entradas e saídas de dados em Haskell, tanto na saída padrão quanto em arquivos. Este capítulo deixou claro o uso da estrutura de uma Mônada. A partir daqui, você tem um conhecimento intermediário da linguagem Haskell e está preparado para aprender persistência de dados, desenvolvimento web e concorrência, por exemplo, sem muita dificuldade.

UM PEQUENO INTERPRETADOR

Até aqui, usamos o Haskell para escrever funções e programas que rodam no "mundo real". Neste capítulo, vamos fazer algo um pouco diferente e muito divertido: escrever um programa que **interpreta outro programa**. Ou seja, vamos construir uma minilinguagem de programação dentro do Haskell e um interpretador para ela.

Esse exercício é clássico no mundo da programação funcional, pois mostra como o Haskell é uma ferramenta poderosa para se trabalhar com linguagens, compiladores e *Domain Specific Languages* (DSLs). E, de quebra, vamos usar o sistema de tipos do Haskell de uma maneira que vai, provavelmente, mudar a forma como você enxerga tipos. Nada de muito sofisticado, prometo; o objetivo é plantar a semente.

Nossa linguagenzinha será uma versão minúscula do *cálculo lambda* com apenas dois tipos de valores: inteiros e booleanos. Ela terá literais, soma, multiplicação, comparação (`<=`) e o `if` . É pouco, mas o suficiente para que apareçam programas "certos" e programas "errados", e é justamente essa fronteira que vamos explorar.

10.1 REPRESENTANDO PROGRAMAS COMO DADOS

A primeira ideia importante é: **um programa é apenas um dado**. Podemos representar uma expressão da nossa linguagem como um valor de um tipo Haskell, usando os conceitos de *sum type* e recursão vistos no *capítulo 3*. Esse tipo costuma ser chamado de **AST** (*Abstract Syntax Tree*, ou árvore sintática abstrata).

```
data Expr
  = IntLit Int
  | BoolLit Bool
  | Add Expr Expr
  | Mul Expr Expr
  | Leq Expr Expr      -- menor ou igual
  | If Expr Expr Expr
deriving Show
```

Note que o tipo `Expr` é recursivo, assim como a `Arvore` do *capítulo 5*: um `Add` carrega duas outras `Expr`, e o `If` carrega três (a condição, o ramo verdadeiro e o ramo falso). Com isso, conseguimos escrever programas da nossa linguagem como valores Haskell. Por exemplo, a expressão matemática $3 + 4 * 5$ fica assim:

```
Add (IntLit 3) (Mul (IntLit 4) (IntLit 5))
```

E um programa que devolve `10` se `2 <= 3` e `20` caso contrário fica:

```
If (Leq (IntLit 2) (IntLit 3)) (IntLit 10) (IntLit 20)
```

10.2 AVALIANDO PROGRAMAS (COM CHECAGEM EM TEMPO DE EXECUÇÃO)

Montar a árvore é só metade do trabalho. Precisamos de uma função que **avalia** (executa) a expressão e devolve seu valor. O problema é: o valor pode ser um inteiro ou um booleano. Por isso, criamos um tipo para representar o resultado:

```
data Valor = VInt Int | VBool Bool deriving (Show, Eq)
```

Aqui mora o perigo. Nada nos impede de montar uma árvore sem sentido, como `Add (BoolLit True) (IntLit 1)` (somar um booleano com um inteiro) ou `If (IntLit 3) ...` (usar um número na condição do `if`). São programas *mal formados*, e nosso interpretador precisa detectá-los. Vamos fazer isso devolvendo um `Either String Valor : Left` para um erro de tipo e `Right` para um valor válido. Lembre-se de que `Either` é uma Mônada, então podemos usar a notação `do` do capítulo 8.

```
-- exige que um Valor seja inteiro, senão devolve um erro de tipo
asInt :: Valor -> Either String Int
asInt (VInt n) = Right n
asInt (VBool _) = Left "erro de tipo: esperava um inteiro, encont
rei um booleano"

asBool :: Valor -> Either String Bool
asBool (VBool b) = Right b
asBool (VInt _) = Left "erro de tipo: esperava um booleano, enco
ntrei um inteiro"

eval :: Expr -> Either String Valor
eval (IntLit n) = Right (VInt n)
eval (BoolLit b) = Right (VBool b)
eval (Add a b) = VInt <$> binInt (+) a b
eval (Mul a b) = VInt <$> binInt (*) a b
eval (Leq a b) = do x <- eval a >=> asInt
                    y <- eval b >=> asInt
                    Right (VBool (x <= y))
```

```

eval (If c t e) = do cond <- eval c >=> asBool
                  if cond then eval t else eval e

binInt :: (Int -> Int -> Int) -> Expr -> Expr -> Either String Int

binInt op a b = do x <- eval a >=> asInt
                  y <- eval b >=> asInt
                  Right (op x y)

```

As funções `asInt` e `asBool` são as "portei­ras" que garantem o tipo certo em cada posição. O `eval a >=> asInt` significa "avalie `a` e, se der certo, exija que o resultado seja um inteiro". Como `Either` é Mônada, o `>=>` já cuida de propagar um eventual `Left` (o primeiro erro encontrado encerra tudo), exatamente como vimos com o `Maybe`.

Vamos avaliar dois **programas bons** no GHCi:

```

*Interp> eval (Add (IntLit 3) (Mul (IntLit 4) (IntLit 5)))
Right (VInt 23)

*Interp> eval (If (Leq (IntLit 2) (IntLit 3)) (IntLit 10) (IntLit
  20))
Right (VInt 10)

```

Como esperado, $3 + 4 * 5 = 23$ e, como $2 \leq 3$ é verdadeiro, o `if` devolve `10`. Agora vamos avaliar dois **programas ruins**:

```

*Interp> eval (Add (BoolLit True) (IntLit 1))
Left "erro de tipo: esperava um inteiro, encontrei um booleano"

*Interp> eval (If (IntLit 3) (IntLit 1) (IntLit 2))
Left "erro de tipo: esperava um booleano, encontrei um inteiro"

```

Funcionou! Nosso interpretador rejeitou os dois programas mal formados, devolvendo uma mensagem de erro em vez de explodir. Essa é uma maneira honesta de escrever um interpretador, e é assim que muitas linguagens dinâmicas (como

Python e JavaScript) tratam os tipos: o erro só aparece **quando o programa roda**.

10.3 TIPOS COMO EXEMPLO DE ELEGÂNCIA: GADTS

Aqui chega a parte legal. Você notou que tivemos de escrever toda a checagem de tipos na mão (as funções `asInt` e `asBool`) e que os erros só aparecem em tempo de execução? Será que o Haskell, com seu sistema de tipos brilhante, não consegue fazer melhor? Será que dá para fazer com que um programa mal formado **nem sequer compile**?

A resposta é **sim**, e a ferramenta para isso se chama **GADT** (*Generalized Algebraic Data Type*). Trata-se de uma extensão da linguagem, associada ao *pragma* `{-# LANGUAGE GADTs #-}` (ela já vem ativada na edição de linguagem `GHC2024`, que veremos no próximo capítulo).

Em um `data` comum, todos os *value constructors* devolvem o mesmo tipo. No nosso `Expr`, tanto `IntLit` quanto `BoolLit` devolvem `Expr`, sem distinção. Um GADT nos deixa escrever a **assinatura completa de cada construtor** e, o mais importante, **refinar o parâmetro de tipo** no retorno de cada um. Veja a nova versão de `Expr`, agora com um parâmetro `a` que diz qual é o tipo do valor que a expressão produz:

```
{-# LANGUAGE GADTs #-}
```

```
data Expr a where
  IntLit  :: Int  -> Expr Int
  BoolLit :: Bool -> Expr Bool
  Add    :: Expr Int -> Expr Int -> Expr Int
```

```

Mul :: Expr Int -> Expr Int -> Expr Int
Leq :: Expr Int -> Expr Int -> Expr Bool
If  :: Expr Bool -> Expr a -> Expr a -> Expr a

```

Leia com calma cada linha. `IntLit` recebe um `Int` e devolve uma `Expr Int` (uma expressão *cujo valor é um inteiro*). `BoolLit` devolve uma `Expr Bool`. O `Add` só aceita duas `Expr Int` e produz uma `Expr Int`. O `Leq` recebe duas `Expr Int` mas produz uma `Expr Bool` (a comparação). E o `If` exige que a condição seja uma `Expr Bool` e que os dois ramos tenham o **mesmo** tipo `a`, devolvendo esse `a`. O tipo está, literalmente, codificando as regras da nossa linguagem.

A consequência disso é maravilhosa: o `eval` fica trivial, sem nenhuma checagem manual e sem `Either`, pois o tipo já garante que tudo se encaixa. Ele devolve diretamente o valor `a`:

```

eval :: Expr a -> a
eval (IntLit n) = n
eval (BoolLit b) = b
eval (Add a b) = eval a + eval b
eval (Mul a b) = eval a * eval b
eval (Leq a b) = eval a <= eval b
eval (If c t e) = if eval c then eval t else eval e

```

Repare que `eval (Add a b) = eval a + eval b` simplesmente soma, sem medo, pois o tipo `Add :: Expr Int -> Expr Int -> Expr Int` garante que `eval a` e `eval b` são inteiros. O compilador faz a prova por nós!

Os programas bons continuam funcionando, e agora o `eval` devolve o valor puro, sem embrulho:

```

*InterpG> eval (Add (IntLit 3) (Mul (IntLit 4) (IntLit 5)))
23

*InterpG> eval (If (Leq (IntLit 2) (IntLit 3)) (IntLit 10) (IntLi

```

```
t 20))  
10
```

```
*InterpG> :t Add (IntLit 1) (IntLit 2)  
Add (IntLit 1) (IntLit 2) :: Expr Int
```

E o programa ruim? Aquele `Add (BoolLit True) (IntLit 1)` que, na versão anterior, só dava erro ao rodar? Agora ele **nem compila**:

```
*InterpG> eval (Add (BoolLit True) (IntLit 1))  
  
<interactive>:1:12: error: [GHC-83865]  
  • Couldn't match type 'Bool' with 'Int'  
    Expected: Expr Int  
    Actual:   Expr Bool  
  • In the first argument of 'Add', namely '(BoolLit True)'
```

Pare um instante e admire isso. O erro de tipo foi pego **antes mesmo de o programa rodar**, em tempo de compilação. Não há como executar um programa mal formado, porque ele não existe: o compilador se recusa a construí-lo. Essa é a promessa do *type-safety* que mencionamos lá na introdução do livro, levada ao extremo. Com GADTs, transformamos uma regra da nossa linguagem em uma garantia estática, e o trabalho de verificar tipos, que fizemos na mão na seção anterior, é feito de graça pelo GHC.

TOME CUIDADO!

GADTs são uma ferramenta poderosa, mas não saia colocando-os em tudo. Eles brilham justamente em situações como a de um interpretador, em que queremos que tipos diferentes de valores convivam em uma mesma estrutura e que o compilador garanta as regras. Para tipos do dia a dia, o bom e velho `data` continua sendo a melhor escolha. Use a ferramenta certa para cada problema.

10.4 EXERCÍCIOS

10.1) Adicione um construtor `Sub Expr Expr` (subtração) à versão dinâmica do `Expr` e trate-o na função `eval`.

10.2) Adicione à versão dinâmica um construtor `And Expr Expr` (o "e" lógico), que só deve operar sobre booleanos. Avalie um programa bom e um ruim no `GHCi` e mostre as saídas.

10.3) Adicione um construtor `Div Expr Expr` (divisão inteira) à versão dinâmica. Além do erro de tipo, ele deve devolver um `Left "divisão por zero"` quando o divisor for `0`. Aproveite que `eval` já vive na Mônada `Either`.

10.4) Na versão com GADTs, adicione o construtor `Sub :: Expr Int -> Expr Int -> Expr Int` e trate-o em `eval`. Tente, no `GHCi`, escrever `eval (Sub (BoolLit True) (IntLit 1))` e observe a mensagem de erro de compilação.

10.5) Ainda na versão com GADTs, adicione um construtor

`And :: Expr Bool -> Expr Bool -> Expr Bool`. Em seguida, explique por escrito por que a expressão `And (IntLit 1) (BoolLit True)` não compila.

10.6) (Opcional) Escreva uma função `mostra :: Expr a -> String` para a versão com GADTs, que transforma uma expressão de volta em uma `String` legível, por exemplo, `mostra (Add (IntLit 3) (IntLit 4))` deve devolver `"(3 + 4)"`.

10.5 CONCLUSÃO

Neste capítulo, construímos uma minilinguagem de programação e dois interpretadores para ela. O primeiro, usando um `data` comum e a Mônada `Either`, faz a checagem de tipos em tempo de execução, devolvendo `Left` para programas mal formados. O segundo, usando GADTs, sobe a checagem para o sistema de tipos: programas mal formados deixam de existir, pois sequer compilam.

Esse contraste mostra, na prática, por que tantos programadores se apaixonam pelo Haskell. O sistema de tipos deixa de ser um chato que reclama e vira um parceiro que escreve provas por você. Vimos também, de leve, a extensão GADTs, que vamos reencontrar no próximo capítulo. Prepare-se, pois agora vamos sair do interpretador e colocar uma aplicação Haskell de verdade na web.

HASKELL NA WEB COM SERVANT

Chegamos ao grande final. Ao longo deste livro, você aprendeu o paradigma funcional, o sistema de tipos, *funtores*, Mônadas e a Mônada IO . Agora vamos juntar tudo isso para fazer o que mais empolga: colocar uma aplicação Haskell **de verdade na web**.

E aqui vai a melhor parte. Em quase toda linguagem, uma API web é um campo minado de erros que só aparecem em produção: uma rota que devolve o tipo errado, um parâmetro que não foi convertido, um *endpoint* documentado de um jeito e implementado de outro. Em Haskell, com a biblioteca **Servant**, a sua API é um **tipo**. E, como você já sabe, no Haskell o que o tipo promete, o compilador cobra. Se o servidor não respeitar exatamente o contrato da API, o programa **não compila**. Acabaram os *bugs* do tipo: "esqueci de tratar essa rota".

Esse é o tipo de garantia que faz empresas como GitHub, Meta, Standard Chartered e diversas *fintechs* e *startups* escolherem Haskell para os pontos mais críticos de seus sistemas. Você está prestes a sentir esse poder na pele.

Neste capítulo, vamos do zero: configurar o projeto, organizar módulos, dar uma espiada em GADTs e, então, construir e rodar um servidor web com dois *endpoints* GET . Vamos lá!

11.1 SEÇÃO 1: CONFIGURANDO O PROJETO COM CABAL

Até agora, compilamos arquivos soltos com `ghc` e brincamos no `GHCi`. Para um projeto de verdade, com dependências externas (como o `Servant`), usamos um **gerenciador de projetos**. O mais tradicional é o `cabal`, que já veio instalado junto com o `GHCup` lá no *capítulo 2*. Comece criando uma pasta e rodando o `cabal init` dentro dela:

```
$ mkdir minha-api && cd minha-api
$ cabal init --interactive
```

O `cabal init` faz algumas perguntas: o tipo de projeto (escolha **Executable**), o nome do pacote, a versão, a licença, o seu nome (ele costuma pegar do `git`), a versão do Cabal a ser usada e a edição de linguagem. Ao final, ele gera um arquivo de configuração com a extensão `.cabal`. Esse arquivo é o coração do projeto, e ele se parece com isto:

```
cabal-version:      3.0
name:               minha-api
version:            0.1.0.0
license:            BSD-2-Clause
build-type:         Simple
```

```
executable minha-api
  main-is:          Main.hs
  hs-source-dirs:   app
  build-depends:    base
  default-language: GHC2021
  ghc-options:      -Wall
```

Vamos entender cada parte:

- `cabal-version: 3.0` deve ser **sempre a primeira linha** do arquivo, e indica a versão da sintaxe do Cabal. Use `3.0` (ou superior).
- `executable minha-api` declara que vamos gerar um executável com esse nome.
- `main-is: Main.hs` aponta o arquivo que contém a função `main` (a mesma do *capítulo 9*).
- `hs-source-dirs: app` diz em qual pasta estão os arquivos `.hs`.
- `build-depends:` é a lista de **dependências**. Por enquanto, só a `base` (a biblioteca padrão). É aqui que vamos adicionar o `Servant` em instantes.
- `default-language: GHC2021` é a **edição de linguagem**. Uma edição é um conjunto pronto de extensões da linguagem ligadas de uma vez só. A `GHC2021` é a padrão desde o `GHC 9.2` e reúne cerca de 50 extensões consagradas. Há também a `GHC2024` (disponível a partir do `GHC 9.10`), que liga, entre outras, as extensões `DataKinds` e `GADTs`, que usaremos aqui. À época da escrita deste livro, o `cabal init` ainda gera projetos com `GHC2021` por padrão; se você tem o `GHC 9.10` ou superior, vale a pena trocar para `GHC2024`, que já vem com várias extensões modernas ligadas.

Para compilar e rodar o projeto, usamos:

```
$ cabal build
$ cabal run
```

TOME CUIDADO!

O `cabal` lê a lista de dependências do arquivo `.cabal` e baixa, da internet, tudo o que for preciso (do repositório de pacotes Hackage). Na primeira vez, rode `cabal update` para atualizar o índice de pacotes. A compilação inicial, que baixa e compila as dependências, pode demorar alguns minutos. Tenha paciência: isso só acontece uma vez.

11.2 SEÇÃO 2: MÓDULOS E COMO ATUALIZAR O ARQUIVO .CABAL

Um programa de verdade não cabe em um único arquivo. Vamos organizar o código em **módulos**. Você já viu módulos desde o *capítulo 2*: aquela linha `module Main where` no topo dos arquivos. Agora vamos criar nossos próprios. Um módulo é um arquivo `.hs` cujo nome corresponde ao caminho do arquivo. Por exemplo, um módulo chamado `Api.Tipos` deve morar no arquivo `src/Api/Tipos.hs` e começar assim:

```
module Api.Tipos where

-- definições do módulo aqui
```

A grande vantagem de modularizar é separar responsabilidades e controlar o que fica visível para fora. Repare que, além de `executable`, podemos ter uma seção `library` no `.cabal`. A biblioteca guarda o código reaproveitável, e o executável apenas a consome. Atualizamos o arquivo `.cabal` assim:

```

library
  exposed-modules:  Api.Tipos
  other-modules:    Api.Interno
  hs-source-dirs:   src
  build-depends:    base
  default-language: GHC2024

executable minha-api
  main-is:          Main.hs
  hs-source-dirs:   app
  build-depends:    base, minha-api  -- depende da nossa própr
ia biblioteca
  default-language: GHC2024

```

Pontos importantes:

- `exposed-modules`: lista os módulos **públicos** da biblioteca (a API que outros podem importar).
- `other-modules`: lista os módulos **internos**, que ficam escondidos do mundo externo, algo parecido com o `private` da orientação a objetos.
- Para o executável usar a biblioteca, basta adicionar o **nome do próprio pacote** (`minha-api`) ao seu `build-depends`.

Toda vez que você criar um novo módulo, lembre-se de adicioná-lo a `exposed-modules` ou `other-modules`. Esquecer disso é o erro de iniciante número um com Cabal, e o sintoma é uma mensagem de "módulo não encontrado".

11.3 SEÇÃO 3: UMA ESPIADA EM GADTS

No capítulo anterior, conhecemos os **GADTs** (*Generalized Algebraic Data Types*) ao construir nosso interpretador. Vale recapitular a ideia, pois ela é a porta de entrada para entender por que o Servant é tão especial.

Um `data` comum tem todos os construtores devolvendo o mesmo tipo. Um GADT, ligado pela extensão `{-# LANGUAGE GADTs #-}` (já incluída na edição `GHC2024`), nos deixa escrever a assinatura completa de cada construtor e **refinar o tipo de retorno**:

```
{-# LANGUAGE GADTs #-}
```

```
data Expr a where
  IntLit  :: Int  -> Expr Int
  BoolLit :: Bool -> Expr Bool
  Add     :: Expr Int -> Expr Int -> Expr Int
```

A lição que fica é que, em Haskell, **tipos carregam informação** e podem codificar regras. O `Expr Int` não é só "uma expressão"; é "uma expressão cujo valor é um inteiro", e o compilador usa isso para barrar bobagens.

O `Servant` leva essa ideia a outro patamar: ele faz **programação no nível dos tipos**. Em vez de um valor `Expr a`, vamos escrever um tipo inteiro que descreve a API.

Para isso, além de GADTs no espírito, entra em cena a extensão `DataKinds`, que permite usar valores (como o texto `"hello"`) lá no nível dos tipos. Não se assuste com a sintaxe a seguir; o importante é a intuição: **a descrição da API é um tipo**.

11.4 SEÇÃO 4: O SERVANT E A EXPLICAÇÃO DOS SEUS TIPOS

O **Servant** é uma biblioteca para construir APIs web em Haskell de forma *type-safe*. À época da escrita deste livro, está na versão `0.20.x`. Sua ideia central, que é genial, é a seguinte: **você descreve a sua API como um tipo Haskell**, e a biblioteca usa esse

tipo para garantir que o servidor (e até clientes) estejam corretos. Para usá-lo, primeiro adicionamos as dependências ao `.cabal`. Vamos precisar de `servant` e `servant-server` (as bibliotecas principais do Servant), `warp` (o servidor HTTP de alta performance, um dos mais rápidos do mundo) e `wai` (a interface padrão de aplicações web em Haskell):

```
executable minha-api
  main-is:           Main.hs
  hs-source-dirs:    app
  build-depends:     base
                    , servant          >= 0.20 && < 0.21
                    , servant-server  >= 0.20 && < 0.21
                    , warp
                    , wai
  default-language: GHC2024
  default-extensions: DataKinds, TypeOperators
  ghc-options:       -threaded -rtsopts
```

A opção `-threaded` é **obrigatória** para rodar o servidor: o `warp` usa o *runtime* com múltiplas *threads* do GHC para atender várias requisições ao mesmo tempo. Se você esquecer dela, o projeto até compila, mas o servidor falha ao iniciar com uma mensagem do tipo *"requires linking against the threaded runtime"*.

As extensões `DataKinds` e `TypeOperators` são **essenciais**: a primeira permite os literais de texto no nível de tipo (como `"hello"`), e a segunda permite usar operadores como `>` e `<|>` em tipos. Se você usar a edição `GHC2021`, ainda precisa ligá-las; com `GHC2024`, o `DataKinds` já vem ligado.

Agora, a parte mais bonita. Uma API no Servant é descrita com alguns "blocos de montar" que vivem no nível dos tipos:

- `>` (lê-se "então") **encadeia** os pedaços de uma rota. Por exemplo, `"sum" > Capture "x" Int > Get`

- '[PlainText] String' significa: o caminho `/sum` , **então** um pedaço capturado como `Int` , **então** um `GET` .
- `:<|>` (lê-se "ou") **combina** vários *endpoints* em uma única API.
- `Get '[PlainText] String'` significa um *endpoint* do tipo `HTTP GET` que devolve um `String` no formato texto puro (`PlainText`). Poderíamos usar `'[JSON]` para devolver JSON, por exemplo.
- `Capture "x" Int` captura um pedaço do caminho da URL e o converte para `Int` . É o equivalente *type-safe* daqueles `/:id` que você vê em outras linguagens, só que aqui o tipo `Int` é garantido pelo compilador.

Juntando tudo, eis a nossa API com dois *endpoints*:

```
type API =
    "hello" => Get '[PlainText] String
    :<|> "sum" => Capture "x" Int => Capture "y" Int => Get '[Plain
Text] String
```

Leia em voz alta: "A API é: o caminho `/hello` que responde a um `GET` com texto; **ou** o caminho `/sum` , então um `Int` , então outro `Int` , que responde a um `GET` com texto". Essa é, literalmente, a documentação da sua API e o seu contrato com o compilador, tudo na mesma linha.

Não existe a possibilidade de a documentação ficar desatualizada em relação ao código, porque elas são a mesma coisa.

11.5 SEÇÃO 5: IMPLEMENTANDO OS ENDPOINTS GET

Descrita a API, falta dizer **o que cada *endpoint* faz**. No Servant, para cada *endpoint* do tipo `API`, fornecemos um *handler* (uma função que produz a resposta). O conjunto dos *handlers* tem o tipo `Server API`, e eles são combinados com o mesmo `:<|>` que usamos na API. A ordem precisa bater exatamente com a ordem dos *endpoints* no tipo, **e o compilador verifica isso para você**.

Os *handlers* vivem na Mônada `Handler`, que nada mais é do que uma `IO` turbinada para o contexto web (lembra das Mônadas do capítulo 8? Aqui elas aparecem trabalhando de verdade). Como nossos *handlers* são simples, usamos `pure` para colocar o valor na Mônada, exatamente como aprendemos.

```
server :: Server API
server = helloH :<|> sumH
  where
    helloH :: Handler String
    helloH = pure "Olá, Haskell na web com Servant!"

    sumH :: Int -> Int -> Handler String
    sumH x y = pure (show (x + y))
```

Analise com carinho:

- `helloH` é o *handler* do `/hello`. Ele não recebe nada e devolve uma `String` dentro de `Handler`. É só um "olá".
- `sumH` é o *handler* do `/sum`. Como a API capturou dois `Int` com `Capture`, o *handler* **recebe esses dois inteiros como parâmetros** — repare como o tipo `Int -> Int -> Handler String` reflete exatamente os dois `Capture` da

API. Ele soma os dois e devolve o resultado como texto, usando o `show` do *capítulo 5*.

Agora, o detalhe que faz a mágica acontecer: se você trocar a ordem dos *handlers*, ou fizer `sumH` receber apenas um `Int`, ou devolver um tipo que não bate, o **programa não compila**. O `Servant` usa o tipo `API` para conferir que cada *handler* tem exatamente a forma certa. Em outras linguagens, isso seria um teste de integração (ou um *bug* em produção); em Haskell, é o compilador, de graça, em toda compilação.

11.6 SEÇÃO 6: O SERVIDOR E O PONTO DE ENTRADA (MAIN)

Falta pouco! Precisamos transformar nosso `server` em uma aplicação web de verdade e colocá-la para rodar. O `Servant` tem a função `serve`, que recebe uma prova do tipo da `API` (um `Proxy`) e os *handlers*, e produz uma `Application` (o tipo padrão do `wai`). Em seguida, o `warp` coloca essa aplicação no ar com a função `run`, que recebe a porta e a aplicação.

O `Proxy` é apenas uma forma de "entregar o tipo `API`" para a função `serve`, já que em Haskell tipos não são valores. Pense nele como um bilhetezinho dizendo "o tipo é este aqui".

```
module Main (main) where

import Data.Proxy (Proxy(..))
import Network.Wai.Handler.Warp (run)
import Servant

-- (aqui entram o type API, o server e os handlers das seções ant
eriores)
```

```
api :: Proxy API
api = Proxy

app :: Application
app = serve api server

main :: IO ()
main = do
    putStrLn "Servidor no ar em http://localhost:8080"
    run 8080 app
```

A função `main`, nosso velho conhecido de tipo `IO ()`, anuncia o endereço e chama `run 8080 app`, que sobe o servidor na porta `8080`. Para rodar tudo:

```
$ cabal run
Servidor no ar em http://localhost:8080
```

Com o servidor no ar, abra outro terminal e teste os *endpoints* (aqui usando o `curl`, mas você pode usar o navegador):

```
$ curl http://localhost:8080/hello
Olá, Haskell na web com Servant!

$ curl http://localhost:8080/sum/3/4
7

$ curl http://localhost:8080/sum/100/250
350
```

Funcionou! Em pouquíssimas linhas, subimos uma API web *type-safe*, com um servidor que é um dos mais rápidos do mundo, e com a garantia, dada pelo compilador, de que o servidor respeita exatamente o contrato da API.

E olha que beleza: experimente, no *endpoint* da soma, acessar `http://localhost:8080/sum/3/abc`. O Servant **automaticamente** devolve um erro `400 Bad Request`, porque `abc` não é um `Int`. Você não escreveu uma linha de validação

para isso; o tipo `Capture "x" Int` cuidou de tudo. Essa é a economia de código, além da redução de *bugs* que tornam Haskell uma escolha de elite para sistemas que não podem falhar.

TOME CUIDADO!

Este foi o "esqueleto" mínimo de uma aplicação *Servant*, propositalmente enxuto para caber em um capítulo introdutório. O ecossistema é muito mais rico: *endpoints* `POST` com `ReqBody`, respostas em `JSON` (com a biblioteca `aeson`), autenticação, geração automática de documentação (`Swagger/OpenAPI`) e até a geração automática de **clientes** a partir do mesmo tipo `API`. Tudo isso reaproveitando o tipo que você já escreveu.

11.7 EXERCÍCIOS

11.1) Crie, do zero, um projeto com `cabal init` e reproduza o servidor deste capítulo. Rode-o e teste os dois *endpoints* com o `curl` ou pelo navegador.

11.2) Adicione um terceiro *endpoint* `GET /mult/:x/:y` que devolve a multiplicação de dois inteiros. Lembre-se de atualizar **três** lugares: o tipo `API`, o `server` e os *handlers*.

11.3) Adicione um *endpoint* `GET /ola/:nome` que captura uma `String` e devolve `"Olá, <nome>!"`. (Dica: use `Capture "nome" String`.)

11.4) O que acontece se você inverter a ordem dos *handlers* em

server (colocar `sumH :<|> helloH`)? Tente compilar e explique a mensagem de erro. O que isso revela sobre a relação entre o tipo `API` e os *handlers*?

11.5) Reorganize o projeto em módulos: mova o tipo `API` e os *handlers* para um módulo `Api` em `src/Api.hs`, declare uma *library* no arquivo `.cabal` com `Api` em `exposed-modules` e deixe o `Main.hs` apenas importando `Api` e chamando o servidor.

11.6) (Opcional) Pesquise como devolver `JSON` em vez de texto puro. Crie um tipo `data Resposta = Resposta { resultado :: Int }`, derive `Generic` e `ToJSON` (da biblioteca `aeson`) e faça o *endpoint* da soma devolver `Get '[JSON] Resposta`.

11.8 CONCLUSÃO

Que jornada! Neste capítulo, saímos do interpretador e construímos uma aplicação web real em Haskell. Configuramos um projeto com Cabal, organizamos o código em módulos e bibliotecas, recapitulamos GADTs e, finalmente, usamos o Servant para descrever uma API **como um tipo** e implementá-la com a garantia do compilador.

Mais do que aprender uma biblioteca, espero que você tenha sentido a virada de chave: em Haskell, o sistema de tipos não atrapalha, ele protege. Aquilo que em outras linguagens é teste, documentação manual e reza, aqui é responsabilidade do compilador. É por isso que, uma vez que um programa Haskell compila, ele costuma simplesmente **funcionar**.

Você começou este livro sem saber o que era uma função pura e termina capaz de colocar uma API *type-safe* no ar. A semente da programação funcional foi plantada. Agora é com você: regue-a com prática, explore o ecossistema, e leve um pouquinho desse rigor e dessa elegância para qualquer linguagem em que você trabalhe. Bem-vindo ao mundo Haskell. Ele é tão bonito por dentro quanto parece.

APÊNDICE

O código usado nos miniprojetos dos capítulos está disponível na íntegra aqui.

```
{-# LANGUAGE BangPatterns #-}
module Main where

import System.Directory
import Control.Monad
import Control.Applicative

data Cargo = Estagiario | Programador | Coordenador | Gerente deriving (Read, Show)

data Pessoa = Pessoa {cargo :: Cargo, nome :: String} deriving (Show, Read)

data Indice a = Indice {indice :: Int, dados :: a} deriving (Show, Read)

data Projeto = Projeto {nomeProjeto :: String,
                        budget :: Double,
                        envolvidos :: [Int]} deriving (Read, Show)

instance Functor Indice where
    fmap f (Indice i dados) = Indice i (f dados)

(|>) :: a -> (a -> b) -> b
(|>) x f = f x

infixl 9 |>
```

```

class ToJSON a where
    toJSON :: a -> String

instance ToJSON a => ToJSON (Indice a) where
    toJSON p = "{id: " ++ show (indice p) ++ ", dados: " ++ (toJS
ON $ dados p) ++ "}"

instance ToJSON Pessoa where
    toJSON p = "{nome: \"" ++ (nome p) ++
        "\", cargo: \"" ++ show (cargo p) ++
        "\", salario: " ++ show (verSalario p) ++ "}"

instance ToJSON Projeto where
    toJSON p = "{nome: \"" ++ (nomeProjeto p) ++
        "\", orçamento: \"" ++ show (budget p) ++
        "\", envolvidos: " ++ show (envolvidos p) ++ "}"

instance Semigroup Projeto where
    (Projeto nome1 budget1 env1) <> (Projeto nome2 budget2 env2)
    =
        Projeto (nome1 ++ ", " ++ nome2) (budget1 + budget2) (en
v1 ++ env2)

instance Monoid Projeto where
    mempty = Projeto "" 0 []

verSalario :: Pessoa -> Double
verSalario (Pessoa Estagiario _) = 1500
verSalario (Pessoa Programador _) = 5750.15
verSalario (Pessoa Coordenador _) = 8000
verSalario (Pessoa Gerente _) = 10807.20

verFolha :: Pessoa -> String
verFolha p = "{nome: \"" ++ (nome p) ++
    "\", cargo: \"" ++ show (cargo p) ++
    "\", salario: " ++ show (verSalario p) ++ "}"

contratar :: Cargo -> (String -> Pessoa)
contratar cargo = Pessoa cargo

promover :: Pessoa -> Pessoa
promover (Pessoa Estagiario n) = Pessoa Programador n
promover (Pessoa Programador n) = Pessoa Coordenador n
promover (Pessoa Coordenador n) = Pessoa Gerente n

```

```

promover (Pessoa _ n) = Pessoa Gerente n

mediaSalarial :: [Pessoa] -> Double
mediaSalarial ps = (foldl calculo 0 ps) / (fromIntegral $ length
ps)
    where
        calculo salario pessoa = salario + verSala
rio pessoa

contratarVariosEstag :: [String] -> [Pessoa]
contratarVariosEstag ps = map (contratar Estagiario) ps

pesquisarPorNome :: String -> [Pessoa] -> [Pessoa]
pesquisarPorNome nomePesq ps = filter (\(Pessoa _ n) -> nomePesq
== n) ps

rotinaPromocao :: Pessoa -> String
rotinaPromocao p = p
    |> promover
    |> verFolha

cadastroPessoa :: IO ()
cadastroPessoa = do
    !pessId <- fmap (length . lines) (readFile "func.dat")
    pess <- Pessoa <$> (putStrLn "Cargo: " >> readLn) <*> (putStr
Ln "Nome: " >> getLine)
    appendFile "func.dat" (show (Indice (1+pessId) pess) ++ "\n")
    putStrLn "Usuário cadastrado com sucesso!"

cadastroProjeto :: IO ()
cadastroProjeto = do
    !projId <- fmap (length . lines) (readFile "projetos.dat")
    proj <- Projeto <$> (putStrLn "Nome: " >> getLine) <*> (putSt
rLn "Orçamento: " >> readLn) <*> (putStrLn "Envolvidos: " >> read
Ln)
    appendFile "projetos.dat" (show (Indice (1+projId) proj) ++ "
\n")
    putStrLn "Projeto cadastrado com sucesso!"

buscaPessoas :: IO ()
buscaPessoas = do
    putStrLn "Digite o id do funcionário"
    pessId <- readLn
    pess <- buscaPorId pessId "func.dat" :: IO (Maybe (Indice Pes
soa))

```

```

    case pess of
      Nothing -> putStrLn "Erro..."
      Just p -> print p

buscaProjetos :: IO ()
buscaProjetos = do
  putStrLn "Digite o id do projeto"
  projId <- readLn
  proj <- buscaPorId projId "projetos.dat" :: IO (Maybe (Indice
Projeto))
  case proj of
    Nothing -> putStrLn "Erro..."
    Just (Indice i (Projeto nm bd env)) -> do
      print $ "id :" ++ show i
      print $ "nome :" ++ nm
      print $ "orçamento :" ++ show bd
      forM_ env $ \envId -> do
        pess <- buscaPorId envId "func.dat" :: IO (Maybe (Indice
Pessoa))
        case pess of
          Nothing -> print "Pessoa inválida"
          Just p -> print p

buscaPorId :: (Read a, Show a) => Int -> String -> IO (Maybe (Ind
ice a))
buscaPorId tid arq = do
  todosReg <- fmap lines (readFile arq)
  todosConv <- return $ map read todosReg
  res <- return $ filter (\ips -> indice ips == tid) todosConv
  case res of
    [] -> return Nothing
    _ -> return $ Just (head res)

todosSalarios :: IO ()
todosSalarios = do
  putStrLn "Listando salários..."
  todasPessoas <- fmap lines (readFile "func.dat")
  todasPess <- return $ map read todasPessoas :: IO [Indice Pes
soa]
  forM_ todasPess $ \(Indice _ pessoa) ->
    print $ "Nome : " ++ (nome pessoa) ++ ", Salário: " ++ sh
ow (verSalario pessoa)

exportarPessoas :: IO ()
exportarPessoas = do

```

```

writeFile "func.json" ""
todasPessoas <- fmap lines (readFile "func.dat")
todasPess <- return $ map read todasPessoas :: IO [Indice Pes
soa]
mapM_ (\x -> appendFile "func.json" (toJSON x ++ "\n")) todas
Pess

menu :: IO Int
menu = do
    putStrLn "Lambda Systems v1.0"
    putStrLn "Digite uma opção... "
    putStrLn "1- Cadastro de Pessoas"
    putStrLn "2- Cadastro de Projetos"
    putStrLn "3- Busca de Pessoas"
    putStrLn "4- Busca de Projetos"
    putStrLn "5- Total gasto por projeto"
    putStrLn "6- Ver salários"
    putStrLn "7- Exportar Pessoas (JSON)"
    putStrLn "8- Exportar Projetos (JSON)"
    opcao <- readLn
    if opcao < 0 || opcao > 8 then do
        putStrLn "Opção inválida"
        menu
    else
        return opcao

main :: IO ()
main = do
    let projetos = "projetos.dat"
        pessoas = "func.dat"
    existeProj <- doesFileExist projetos
    existeFunc <- doesFileExist pessoas
    when (not existeProj) (writeFile projetos "")
    when (not existeFunc) (writeFile pessoas "")
    opcao <- menu
    case opcao of
        1 -> cadastroPessoa
        2 -> cadastroProjeto
        3 -> buscaPessoas
        4 -> buscaProjetos
        6 -> todosSalarios
        7 -> exportarPessoas
        _ -> putStrLn "Opção ainda não implementada"

```

REFERÊNCIAS

HUGHES, J. *Why Functional Programming Matters*. Research Topics in Functional Programming, Addison-Wesley, 1990, pp 17-42. Disponível em: <https://www.cs.kent.ac.uk/people/staff/dat/miranda/whyfp90.pdf>.

LIPOVACA, M. *Learn You a Haskell for Great Good! A Beginner's Guide*. 2011.

MESTANOGULLARI, A.; HAHN, S.; ARNI, J. K.; LÖH, A. *Type-level Web APIs with Servant*: an exercise in domain-specific generic programming. Proceedings of the 11th ACM SIGPLAN Workshop on Generic Programming (WGP), 2015. Disponível em: <https://dl.acm.org/doi/10.1145/2808098.2808099>.

O'GRADY, Stephen. *The RedMonk Programming Language Rankings: January 2016*. Fev. 2016. Disponível em: <http://redmonk.com/sogrady/2016/02/19/language-rankings-1-16/>.

O'GRADY, Stephen. *The RedMonk Programming Language Rankings: January 2026*. Abr. 2026. Disponível em: <https://redmonk.com/sogrady/2026/04/14/language-rankings-1-26/>.

O'SULLIVAN, B.; GOERZEN, J.; STEWART, D. *Real World Haskell: Code You Can Believe*. O'Reilly, 2008.

SERVANT. *Servant — A Type-Level Web DSL*: documentação oficial. Disponível em: <https://docs.servant.dev/> e <https://www.servant.dev/>.

SNOYMAN, M. *Developing Web Applications with Haskell and Yesod*. O'Reilly, 2012.

SPIVAK, D. *Category Theory for the Sciences*. MIT Press, 2014.

PEYTON JONES, S.; HUDAK, P.; WADLER, P.; HUGHES, J. *A history of Haskell: being lazy with class*. Proceedings of the III ACM SIGPLAN, Jun. 2007.

PEYTON JONES, S. *Haskell and Erlang: growing up together*. Erlang factory meeting, 2009.

PEYTON JONES, S. *Tackling the Awkward Squad: monadic input/output, concurrency,*

exceptions, and foreign-language calls in Haskell. Maio, 2002. Disponível em: https://www.researchgate.net/publication/2472798_Tackling_the_Awkward_Squad_monadic_inputoutput_concurrency_exceptions_and_foreign-language_calls_in_Haskell.